

Formale Verifikation von Amaranth-HDL-Designs

Äquivalenznachweis zwischen Python-basierter
Hardware-Beschreibung und traditionellem VHDL

Stephan Epp

31. März 2026

Inhaltsverzeichnis

1	Einleitung	5
1.1	Motivation	5
1.2	Problemstellung	5
1.3	Beitrag dieser Arbeit	6
1.4	Struktur der Arbeit	6
2	Theoretische Grundlagen	7
2.1	Hardware-Beschreibungssprachen	7
2.2	Amaranth HDL Framework	7
2.2.1	Kernkonzepte von Amaranth	8
2.3	Formale Verifikation	8
2.3.1	Äquivalenzprüfung (Equivalence Checking)	8
2.4	Yosys Synthese-Framework	8
3	VHDL-Referenz-Implementation	10
3.1	Spezifikation	10
3.2	Architektur-Übersicht	10
3.3	VHDL-Code-Struktur	11
4	Amaranth-Implementation	13
4.1	Design-Philosophie	13
4.2	Implementierungs-Strategie	13
4.3	Vollständige Amaranth-Implementation	13
4.4	Design-Entscheidungen	17
5	Formale Verifikation	18
5.1	Verifikations-Workflow	18
5.2	VHDL zu Verilog Konvertierung	18
5.3	Verifikations-Script	18
5.4	Yosys-Kommandos im Detail	20
5.5	Mathematische Grundlage der Verifikation	21
6	Hardware-Validierung	22
6.1	Das Basys-3 Entwicklungsboard	22
6.1.1	Pin-Belegung der relevanten Peripherie	23
6.2	Constraint-Datei (XDC)	23
6.3	Synthese mit Vivado	25
6.3.1	Vivado TCL-Batch-Skript	25
6.3.2	Syntheseoptionen und Strategien	25

6.3.3	Synthese-Analyse und kritischer Pfad	26
6.4	Implementierungs-Ergebnisse	26
6.5	FPGA-Programmierung mit openFPGALoader	27
6.5.1	Installation und Voraussetzungen	27
6.5.2	Programmiervorgang	27
6.6	Hardware-Test-Szenarien	28
6.6.1	Testmethodik	28
6.6.2	Vollständige Test-Szenarien	29
6.6.3	Anmerkungen zu Sonderfällen	29
6.7	Beobachtungen auf dem 7-Segment-Display	30
6.7.1	Anzeigeformat	30
6.7.2	Multiplexing-Beobachtung	30
6.7.3	Übergangsverhalten	30
6.8	Validierung der Äquivalenz auf Hardware	30
6.8.1	Methodisches Vorgehen	30
6.8.2	Ergebnisvergleich	31
6.8.3	Bewertung der Aussagekraft	31
6.9	Timing-Verifikation und Stabilitätstest	31
6.9.1	Statische Timing-Analyse (STA)	31
6.9.2	Thermischer Stabilitätstest	32
6.10	Zusammenfassung der Hardware-Validierung	32
7	Ergebnisse und Diskussion	33
7.1	Formale Verifikations-Ergebnisse	33
7.2	Ressourcen-Nutzung	33
7.3	Diskussion der Ergebnisse	33
7.4	Limitationen	34
8	Äquivalenzprüfung mit dem Subgraph Algorithmus	35
8.1	Motivation und Überblick	35
8.2	Graphrepräsentation von Hardware-Designs	35
8.3	Überführung in Adjazenzmatrizen	36
8.4	Formale Grundlagen des Subgraph Algorithmus	36
8.4.1	Signatur-Funktion	36
8.4.2	Zyklische Rotation und Subgraph-Kriterium	37
8.5	Äquivalenzprüfung mit dem Subgraph Algorithmus	37
8.5.1	Satz 1: Äquivalenz über bidirektionale Subgraph-Beziehung	37
8.5.2	Satz 2: Implikation durch strukturelle Äquivalenz	38
8.5.3	Satz 3: Korrektheit der Subgraph-Äquivalenzprüfung	39
8.5.4	Satz 4: Laufzeit Subgraph-Äquivalenzprüfungsverfahrens	39
8.5.5	Satz 5: Sensitivität gegenüber Kanten-Manipulationen	40
8.5.6	Satz 6: Transitivität der Subgraph-Implikation	41
8.5.7	Satz 7: Effizienz gegenüber naiver Permutationssuche	41
8.5.8	Satz 8: Untere Schranke für die Äquivalenzprüfung (Optimalität)	42
8.6	Anwendung auf die ALU: Exemplarischer Beweis	42
8.6.1	Konstruktion der Abhängigkeitsgraphen	42
8.6.2	Adjazenzmatrizen	43
8.6.3	Berechnung der Signaturen	43
8.6.4	Ergebnis der Äquivalenzprüfung	43

8.7	SAT-basierte vs. Subgraph-basierte Äquivalenz	44
8.8	Zusammenfassung	44
9	Äquivalenz: Verilog gegen VHDL	45
9.1	Motivation und Einordnung	45
9.2	Spezifikation und Schnittstellenvergleich	46
9.2.1	Gemeinsame Schnittstellenspezifikation	46
9.2.2	Interne Parametrierung	46
9.3	Funktionale Dekomposition	46
9.4	Schritt-für-Schritt-Äquivalenz der Teilblöcke	49
9.4.1	Schritt 1 – Input-Mapping	49
9.4.2	Schritt 2 – ALU-Logik	50
9.4.3	Schritt 3 – Display-Multiplexer	53
9.4.4	Schritt 4 – 7-Segment-Decoder	54
9.5	Strukturelle Äquivalenzprüfung mit dem Subgraph Algorithmus	55
9.5.1	Konstruktion der Hardware-Abhängigkeitsgraphen	55
9.5.2	Adjazenzmatrix	55
9.5.3	Signaturberechnung	56
9.5.4	Äquivalenzentscheid	56
9.6	Formale Verifikation mit Yosys	57
9.6.1	Besonderheit: VHDL in Yosys	57
9.6.2	Vollständiges Äquivalenz-Script	57
9.6.3	Interpretation des Yosys-Outputs	58
9.6.4	Komplexitätsbetrachtung	58
9.7	Zusammenfassender Äquivalenzbeweis	58
9.8	Warum benötigt VHDL mehr Anweisungen?	59
10	Simulations-basierte Testautomatisierung und Code-Überdeckung	60
10.1	Motivation und Einordnung	60
10.2	Der AOI-2-2-Schaltkreis als Demonstrationsobjekt	61
10.2.1	Logische Spezifikation	61
10.2.2	Verilog-Implementation	62
10.2.3	VHDL-Implementation	62
10.3	Universal Verification Methodology in Python	63
10.3.1	UVM – Grundkonzepte und Klassenhierarchie	63
10.3.2	Kernklassen in der vorliegenden Testbench	64
10.4	Testbench-Architektur im Detail	65
10.4.1	Sequenz-Item: <code>AoiSeqItem</code>	65
10.4.2	Sequenzen	65
10.4.3	Treiber: <code>Driver</code>	66
10.4.4	Überdeckungs-Komponente: <code>Coverage</code>	67
10.4.5	Scoreboard	68
10.4.6	Testumgebung: <code>AoiEnv</code>	68
10.4.7	Test-Klassen	69
10.5	Bus Functional Model: <code>AoiBfm</code>	70
10.6	Prädiktionsmodell und Fehlerinjektion	71
10.7	Test-Strategien und Verifikationsabdeckung	71
10.8	Make-basierter Simulations-Workflow	72
10.8.1	Verilog-Simulation mit Verilator	72

10.8.2	VHDL-Simulation mit GHDL	73
10.8.3	Quelldatei-Auflösung im Makefile	73
10.9	Überdeckungsbericht	73
10.10	Laufzeit-Profilng	74
10.11	Simulations- vs. formal-basierte Verifikation	75
10.12	Zusammenfassung	76
11	Schluss	77
11.1	Andere Python-basierte HDLs	78
11.2	Zukünftige Arbeiten	78
11.2.1	Verifikation komplexerer Hardware-Designs	78
11.2.2	Erweiterung der formalen Verifikationstechniken	79
11.2.3	Automatisierung und CI/CD-Integration	80
11.2.4	Vollständig quelloffene Toolchain	81
11.2.5	Erweiterte Testmethoden	81
11.2.6	Hardware-Software-Co-Design	82
11.2.7	Skalierbarkeit und Grenzen der formalen Verifikation	83
11.2.8	Einsatz in der Hochschullehre	83
11.2.9	Industrielle Adoption und Standardisierung	84
11.2.10	Subgraph Algorithmus-basierten Äquivalenzprüfung	84
11.3	Schlusswort	90

Kapitel 1

Einleitung

Diese Arbeit präsentiert eine vollständige Methodik zur formalen Verifikation von Hardware-Designs, die mit Amaranth HDL (einem Python-basierten Hardware-Beschreibungsframework) entwickelt wurden. Es wird gezeigt, wie die funktionale Äquivalenz zwischen einer Amaranth-Implementation und einer VHDL-Referenz-Implementation mittels Open-Source-Tools formal bewiesen werden kann. Als Demonstrationsobjekt dient eine arithmetisch-logische Einheit (ALU) mit 7-Segment-Display-Ausgabe für das Basys 3 FPGA-Board. Die Arbeit umfasst den kompletten Workflow von der Designspezifikation über die Amaranth-Implementation, die formale Verifikation mit Yosys bis zur Hardware-Validierung auf einem Xilinx Artix-7 FPGA.

1.1 Motivation

Die traditionelle Hardware-Entwicklung basiert auf etablierten Hardware-Beschreibungssprachen (HDL) wie VHDL und Verilog. Diese Sprachen bieten zwar präzise Kontrolle über Hardware-Strukturen, sind jedoch in ihrer Ausdruckskraft begrenzt und erfordern erheblichen Entwicklungsaufwand. Moderne Python-basierte HDL-Frameworks wie Amaranth [2] versprechen höhere Produktivität durch:

- Nutzung moderner Programmiersprachenfeatures (Klassen, Generatoren, Meta-Programmierung)
- Wiederverwendbare, parametrisierbare Hardware-Module
- Integration in bestehende Python-Entwicklungsumgebungen
- Direkte Anbindung an Verifikations- und Simulations-Tools

Der entscheidende Vorteil liegt in der Möglichkeit, Hardware-Designs auf einer höheren Abstraktionsebene zu beschreiben, während die Generierung von synthetisierbarem HDL-Code automatisiert erfolgt.

1.2 Problemstellung

Bei der Einführung neuer HDL-Frameworks stellt sich die zentrale Frage: *Generiert das Framework tatsächlich äquivalenten Hardware-Code zur Referenz-Implementation?*

Diese Arbeit adressiert folgende Forschungsfragen:

1. Kann Amaranth existierende VHDL-Designs funktional äquivalent reimplementieren?
2. Wie lässt sich diese Äquivalenz formal beweisen?
3. Welche Werkzeuge und Methoden sind für die Verifikation geeignet?
4. Wie ist der vollständige Workflow von der Spezifikation bis zur Hardware-Validierung?

1.3 Beitrag dieser Arbeit

Diese Arbeit liefert:

- Eine vollständige, reproduzierbare Methodik zur formalen Verifikation von Amaranth-Designs
- Detaillierte Implementierungsanleitungen für alle Workflow-Schritte
- Einen formalen Äquivalenzbeweis zwischen Amaranth- und VHDL-Implementation
- Praktische Validierung auf realer FPGA-Hardware
- Open-Source Verifikations-Scripts für die Community

1.4 Struktur der Arbeit

Kapitel 2 führt die theoretischen Grundlagen ein. Kapitel 3 beschreibt die VHDL-Referenz-Implementation. Kapitel 4 präsentiert die Amaranth-Reimplementation. Kapitel 5 erläutert die formale Verifikationsmethodik. Kapitel 6 zeigt die Hardware-Validierung. Kapitel 7 diskutiert Ergebnisse und Limitationen. Kapitel 8 entwickelt eine neue, vollständig eigenständige Methodik zur Äquivalenzprüfung von Hardware-Designs auf Basis des Subgraph Algorithmus und führt rigoros die theoretischen Grundlagen sowie Korrektheitsbeweise dieser Methodik ein. Kapitel 9 liefert einen vollständigen, dreistufigen Äquivalenzbeweis zwischen einer manuell geschriebenen Verilog- und einer VHDL-Implementierung derselben ALU-Schaltung durch manuelle Inspektion, Subgraph-basierte Strukturprüfung und SAT-basierte formale Verifikation mit Yosys. Kapitel 10 stellt eine vollständige simulationsbasierte Testbench auf Basis von pyUVM und cocotb vor, die den AOI-2-2-Schaltkreis sowohl in Verilog (Verilator) als auch in VHDL (GHDL) automatisiert testet, funktionale Überdeckung erzwingt und Fehlerinjektion unterstützt.

Kapitel 2

Theoretische Grundlagen

2.1 Hardware-Beschreibungssprachen

Definition 2.1.1 (Hardware-Beschreibungssprache). *Eine Hardware-Beschreibungssprache (HDL) ist eine formale Sprache zur Spezifikation, Simulation und Synthese digitaler Schaltungen. HDLs beschreiben die Struktur und das Verhalten von elektronischen Systemen.*

Die zwei dominierenden HDLs sind:

VHDL (VHSIC Hardware Description Language) Entwickelt vom US-Verteidigungsministerium, stark typisiert, IEEE 1076 standardisiert. VHDL zeichnet sich durch:

- Strikte Typisierung und umfangreiche Standardbibliothek
- Explizite Trennung zwischen Entity (Schnittstelle) und Architecture (Implementati-on)
- Unterstützung für generische und parametrisierbare Designs

Verilog Entwickelt von Gateway Design Automation, C-ähnliche Syntax, IEEE 1364 standardisiert. Verilog bietet:

- Kompaktere Syntax im Vergleich zu VHDL
- Schwächere Typisierung, flexiblerer Umgang mit Datentypen
- Weite Verbreitung in der ASIC-Entwicklung

2.2 Amaranth HDL Framework

Amaranth (früher bekannt als nMigen) ist ein Python-Framework für digitales Hardware-Design. Im Gegensatz zu traditionellen HDLs wird Hardware in Python beschrieben und in Verilog synthetisiert.

Definition 2.2.1 (Amaranth). *Amaranth ist ein Python-eingebettetes Hardware-Beschreibungs-Framework, das Hardware-Designs als Python-Klassenstrukturen repräsentiert und durch Metaprogrammierung in synthetisierbaren Verilog-Code transformiert.*

2.2.1 Kernkonzepte von Amaranth

Signal Die grundlegende Datenstruktur in Amaranth. Signals repräsentieren Drähte oder Register.

```
1 from amaranth import Signal
2
3 # 8-Bit Signal
4 data = Signal(8, name="data")
5
6 # Signal mit Initialisierung
7 counter = Signal(16, init=0, name="counter")
8
9 # Signal mit Bereich
10 index = Signal(range(100), name="index")
```

Listing 2.1: Amaranth Signal-Deklaration

Module Container für Hardware-Logik. Module enthalten Signale, kombinatorische und synchrone Logik.

Elaboratable Basisklasse für wiederverwendbare Hardware-Komponenten. Die `elaborate()`-Methode generiert die Hardware-Struktur.

2.3 Formale Verifikation

Definition 2.3.1 (Formale Verifikation). *Formale Verifikation bezeichnet mathematische Methoden zum Beweis der Korrektheit eines Systems bezüglich einer Spezifikation. Im Kontext von Hardware-Design wird bewiesen, dass eine Implementation ihre Spezifikation für alle möglichen Eingaben erfüllt.*

2.3.1 Äquivalenzprüfung (Equivalence Checking)

Bei der Äquivalenzprüfung wird bewiesen, dass zwei Hardware-Designs für alle Eingabebelegungen identische Ausgaben produzieren.

Definition 2.3.2 (Kombinatorische Äquivalenz). *Zwei kombinatorische Schaltungen C_1 und C_2 sind äquivalent, wenn für alle Eingabevektoren $\vec{x} \in \{0, 1\}^n$ gilt:*

$$C_1(\vec{x}) = C_2(\vec{x})$$

Definition 2.3.3 (Sequentielle Äquivalenz). *Zwei sequentielle Schaltungen S_1 und S_2 sind äquivalent, wenn für alle Eingabesequenzen $\{\vec{x}_0, \vec{x}_1, \dots, \vec{x}_k\}$ und identische Anfangszustände die Ausgabesequenzen übereinstimmen.*

2.4 Yosys Synthese-Framework

Yosys ist ein Open-Source Framework für RTL-Synthese und formale Verifikation. Es konvertiert Hardware-Beschreibungen in Netzlisten und bietet Äquivalenzprüfungs-Algorithmen.

Äquivalenzprüfungs-Workflow in Yosys

1. **Read**: Laden der Designs (Verilog/VHDL)
2. **Elaborate**: Hierarchie-Auflösung, Parameter-Expansion
3. **Proc**: Konvertierung von Prozessen zu Netzlisten
4. **Opt**: Logik-Optimierung
5. **Flatten**: Hierarchie-Auflösung
6. **Equiv_make**: Erstellung des Äquivalenz-Moduls
7. **Equiv_simple**: SAT-basierte Äquivalenzprüfung
8. **Equiv_induct**: Induktive Beweisführung

Kapitel 3

VHDL-Referenz-Implementation

3.1 Spezifikation

Das Demonstrationsobjekt ist eine arithmetisch-logische Einheit (ALU) mit folgenden Eigenschaften:

- **Eingänge:** Zwei 4-Bit Operanden (A, B), 4-Bit Steuerung
- **Ausgabe:** 8-Bit Ergebnis
- **Operationen:** Addition, Subtraktion, Multiplikation, Division, AND, OR
- **Anzeige:** 4-stelliges 7-Segment-Display (Multiplexing bei 1 kHz)
- **Clock:** 100 MHz (Basy3 Board)

3.2 Architektur-Übersicht

Die VHDL-Implementation besteht aus folgenden Hauptkomponenten:

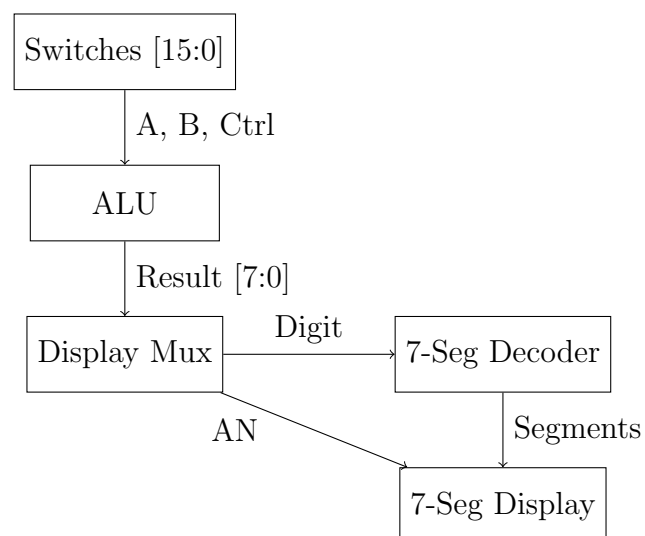


Abbildung 3.1: Architektur der ALU mit 7-Segment-Display

3.3 VHDL-Code-Struktur

Die vollständige VHDL-Implementation umfasst:

```
1 entity alu_7_segment is
2 port (
3   CLK : in STD_LOGIC;
4   SWT : in STD_LOGIC_VECTOR(15 downto 0);
5   SEG : out STD_LOGIC_VECTOR(6 downto 0);
6   AN  : out STD_LOGIC_VECTOR(3 downto 0)
7 );
8 end alu_7_segment;
```

Listing 3.1: VHDL Entity

ALU-Prozess Der ALU-Prozess implementiert alle arithmetischen und logischen Operationen:

```
1 alu_process: process(a_in, b_in, ctrl_in)
2   variable a_unsigned : unsigned(7 downto 0);
3   variable b_unsigned : unsigned(7 downto 0);
4   variable temp_result : unsigned(7 downto 0);
5 begin
6   a_unsigned := "0000" & unsigned(a_in);
7   b_unsigned := "0000" & unsigned(b_in);
8
9   case ctrl_in is
10    when OP_ADD =>
11      temp_result := a_unsigned + b_unsigned;
12    when OP_SUB =>
13      -- Subtraktion mit Clamping
14      if signed('0' & a_unsigned) < signed('0' & b_unsigned) then
15        temp_result := (others => '0');
16      else
17        temp_result := a_unsigned - b_unsigned;
18      end if;
19    when OP_MULT =>
20      temp_result := unsigned(a_in) * unsigned(b_in);
21    -- ... weitere Operationen
22  end case;
23
24  alu_result <= std_logic_vector(temp_result);
25 end process;
```

Listing 3.2: ALU-Operationen in VHDL

Display-Multiplexing Das 7-Segment-Display wird mit 1 kHz Refresh-Rate gemultiplext:

```
1 multiplex_process: process(CLK)
2 begin
3   if rising_edge(CLK) then
4     if refresh_counter = REFRESH_COUNT - 1 then
5       refresh_counter <= 0;
6       digit_select <= std_logic_vector(
```

```
7         unsigned(digit_select) + 1
8     );
9     else
10         refresh_counter <= refresh_counter + 1;
11     end if;
12 end if;
13 end process;
```

Listing 3.3: Display-Multiplexing

Kapitel 4

Amaranth-Implementation

4.1 Design-Philosophie

Die Amaranth-Implementation folgt dem Prinzip der *strukturellen Äquivalenz*: Jede VHDL-Komponente wird in eine entsprechende Amaranth-Struktur übersetzt, wobei die Hardware-Semantik exakt erhalten bleibt.

4.2 Implementierungs-Strategie

1. **Flache Architektur**: Alle Komponenten in einem Modul (wie VHDL Architecture)
2. **Identische Port-Namen**: CLK, SWT, SEG, AN
3. **Identische Signal-Namen**: refresh_counter, digit_select, etc.
4. **Identische Timing-Parameter**: REFRESH_COUNT = 100000

4.3 Vollständige Amaranth-Implementation

```
1 from amaranth import Elaboratable, Module, Signal
2 from amaranth.back import verilog
3
4 class ALU7Segment(Elaboratable):
5     # ALU Operation Codes
6     OP_ADD = 0b0000    # A + B
7     OP_SUB = 0b0001    # A - B
8     OP_MULT = 0b0010   # A * B
9     OP_DIV = 0b0011    # A / B
10    OP_AND = 0b0100     # A and B
11    OP_OR = 0b0101      # A or B
12
13    # Refresh-Rate: 100MHz / 100000 = 1kHz
14    REFRESH_COUNT = 100000
15
16    def __init__(self):
17        # Ports (identisch zu VHDL)
18        self.CLK = Signal()
19        self.SWT = Signal(16)
20        self.SEG = Signal(7)
```

```
21 self.AN = Signal(4)
```

Listing 4.1: Amaranth ALU-Implementation - Teil 1

```
1 def elaborate(self, platform):
2     module = Module()
3
4     # Interne Signale
5     a_in = Signal(4, name="a_in")
6     b_in = Signal(4, name="b_in")
7     ctrl_in = Signal(4, name="ctrl_in")
8     alu_result = Signal(8, name="alu_result")
9
10    # Display-Signale
11    digit_0 = Signal(4, name="digit_0")
12    digit_1 = Signal(4, name="digit_1")
13    digit_2 = Signal(4, name="digit_2")
14    digit_3 = Signal(4, name="digit_3")
15
16    # Multiplexing-Signale
17    refresh_counter = Signal(
18        range(self.REFRESH_COUNT),
19        init=0,
20        name="refresh_counter"
21    )
22    digit_select = Signal(2, init=0, name="digit_select")
23    current_digit = Signal(4, name="current_digit")
```

Listing 4.2: Amaranth ALU-Implementation - Teil 2: elaborate()

```
1 # Input/Output Assignments (kombinatorisch)
2 module.d.comb += [
3     a_in.eq(self.SWT[4:8]),
4     b_in.eq(self.SWT[0:4]),
5     ctrl_in.eq(self.SWT[12:16]),
6 ]
7
8 module.d.comb += [
9     digit_0.eq(b_in),
10    digit_1.eq(a_in),
11    digit_2.eq(alu_result[0:4]),
12    digit_3.eq(alu_result[4:8]),
13 ]
```

Listing 4.3: Amaranth ALU-Implementation - Teil 3: I/O Mapping

```
1 # ALU Operationen (kombinatorisch)
2 a_unsigned = Signal(8, name="a_unsigned")
3 b_unsigned = Signal(8, name="b_unsigned")
4
5 module.d.comb += [
6     a_unsigned.eq(a_in),
7     b_unsigned.eq(b_in),
8 ]
9
10 with module.Switch(ctrl_in):
11     with module.Case(self.OP_ADD):
12         module.d.comb += alu_result.eq(
13             a_unsigned + b_unsigned
```

```

14         )
15
16     with module.Case(self.OP_SUB):
17         sub_result = Signal(9, name="sub_result")
18         module.d.comb += sub_result.eq(
19             a_unsigned - b_unsigned
20         )
21
22         with module.If(sub_result[8]): # Negativ?
23             module.d.comb += alu_result.eq(0)
24         with module.Else():
25             module.d.comb += alu_result.eq(
26                 sub_result[0:8]
27             )
28
29     with module.Case(self.OP_MULT):
30         mult_result = Signal(8, name="mult_result")
31         module.d.comb += mult_result.eq(a_in * b_in)
32         module.d.comb += alu_result.eq(mult_result)
33
34     with module.Case(self.OP_DIV):
35         with module.If(b_unsigned == 0):
36             module.d.comb += alu_result.eq(0xFF)
37         with module.Else():
38             module.d.comb += alu_result.eq(
39                 a_unsigned // b_unsigned
40             )
41
42     with module.Case(self.OP_AND):
43         module.d.comb += alu_result.eq(
44             a_unsigned & b_unsigned
45         )
46
47     with module.Case(self.OP_OR):
48         module.d.comb += alu_result.eq(
49             a_unsigned | b_unsigned
50         )
51
52     with module.Default():
53         module.d.comb += alu_result.eq(0)

```

Listing 4.4: Amaranth ALU-Implementation - Teil 4: ALU-Logik

```

1     # Display Multiplexing (synchron)
2     with module.If(refresh_counter == (self.REFRESH_COUNT - 1)):
3         module.d.sync += [
4             refresh_counter.eq(0),
5             digit_select.eq(digit_select + 1),
6         ]
7     with module.Else():
8         module.d.sync += refresh_counter.eq(
9             refresh_counter + 1
10        )
11
12    # Digit Selection (kombinatorisch)
13    with module.Switch(digit_select):
14        with module.Case(0b00):
15            module.d.comb += [
16                self.AN.eq(0b1110),

```



```

17         current_digit.eq(digit_0),
18     ]
19     with module.Case(0b01):
20         module.d.comb += [
21             self.AN.eq(0b1101),
22             current_digit.eq(digit_1),
23         ]
24     with module.Case(0b10):
25         module.d.comb += [
26             self.AN.eq(0b1011),
27             current_digit.eq(digit_2),
28         ]
29     with module.Case(0b11):
30         module.d.comb += [
31             self.AN.eq(0b0111),
32             current_digit.eq(digit_3),
33         ]
34     with module.Default():
35         module.d.comb += [
36             self.AN.eq(0b1111),
37             current_digit.eq(0),
38         ]

```

Listing 4.5: Amaranth ALU-Implementation - Teil 5: Display Multiplexing

```

1     # 7-Segment Decoder (kombinatorisch)
2     with module.Switch(current_digit):
3         with module.Case(0x0):
4             module.d.comb += self.SEG.eq(0b1000000)
5         with module.Case(0x1):
6             module.d.comb += self.SEG.eq(0b1111001)
7         with module.Case(0x2):
8             module.d.comb += self.SEG.eq(0b0100100)
9         with module.Case(0x3):
10            module.d.comb += self.SEG.eq(0b0110000)
11        with module.Case(0x4):
12            module.d.comb += self.SEG.eq(0b0011001)
13        with module.Case(0x5):
14            module.d.comb += self.SEG.eq(0b0010010)
15        with module.Case(0x6):
16            module.d.comb += self.SEG.eq(0b0000010)
17        with module.Case(0x7):
18            module.d.comb += self.SEG.eq(0b1111000)
19        with module.Case(0x8):
20            module.d.comb += self.SEG.eq(0b0000000)
21        with module.Case(0x9):
22            module.d.comb += self.SEG.eq(0b0010000)
23        with module.Case(0xA):
24            module.d.comb += self.SEG.eq(0b0001000)
25        with module.Case(0xB):
26            module.d.comb += self.SEG.eq(0b0000011)
27        with module.Case(0xC):
28            module.d.comb += self.SEG.eq(0b1000110)
29        with module.Case(0xD):
30            module.d.comb += self.SEG.eq(0b0100001)
31        with module.Case(0xE):
32            module.d.comb += self.SEG.eq(0b0000110)
33        with module.Case(0xF):
34            module.d.comb += self.SEG.eq(0b0001110)

```

```

35         with module.Default():
36             module.d.comb += self.SEG.eq(0b1111111)
37
38         return module

```

Listing 4.6: Amaranth ALU-Implementation - Teil 6: 7-Segment Decoder

```

1 def generate_verilog():
2     """Generiere Verilog-Code"""
3     top = ALU7Segment()
4
5     ports = [top.CLK, top.SWT, top.SEG, top.AN]
6
7     output = verilog.convert(
8         top,
9         name="alu_7_segment",
10        ports=ports,
11    )
12
13    with open("alu_7_segment.v", "w") as file:
14        file.write(output)
15
16    print("Verilog generiert: alu_7_segment.v")
17
18 if __name__ == "__main__":
19     generate_verilog()

```

Listing 4.7: Verilog-Generierung

4.4 Design-Entscheidungen

Flache Struktur Anders als bei modularer Amaranth-Entwicklung üblich, wurde hier bewusst eine flache Struktur gewählt, um die strukturelle Äquivalenz zur VHDL-Architecture zu maximieren.

Explizite Signal-Namen Alle Signale erhalten explizite Namen (`name="..."`) für bessere Nachvollziehbarkeit im generierten Verilog-Code.

Trennung von Kombinatorik und Sequenz Klare Trennung zwischen `module.d.comb` (kombinatorische Logik) und `module.d.sync` (synchrone Logik, getaktet).

Kapitel 5

Formale Verifikation

5.1 Verifikations-Workflow

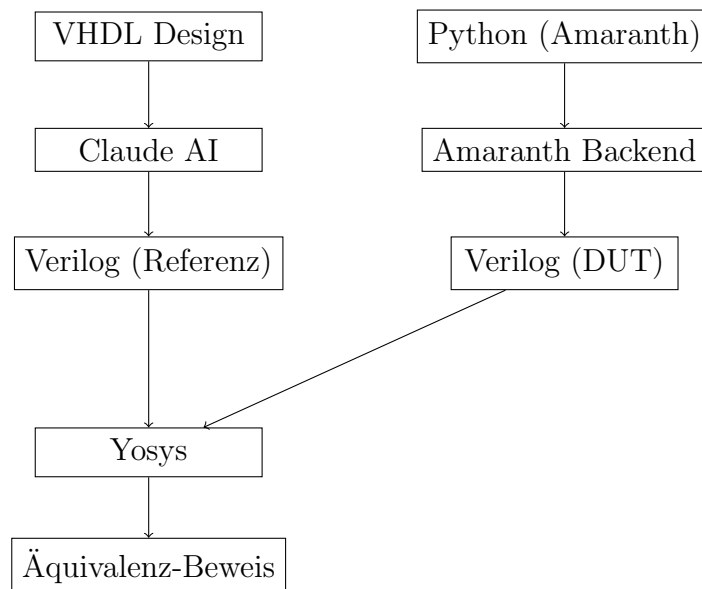


Abbildung 5.1: Verifikations-Workflow

5.2 VHDL zu Verilog Konvertierung

Da Yosys primär mit Verilog arbeitet, muss das VHDL-Design zunächst in Verilog konvertiert werden. Dies erfolgt mittels Claude AI als Konvertierungs-Tool.

Beispiel 5.2.1 (VHDL zu Verilog via Claude AI). *Der VHDL-Code wird Claude AI mit folgendem Prompt übergeben:*

„Konvertiere diesen VHDL-Code in synthetisierbares Verilog. Behalte alle Port-Namen, Signal-Namen und die Logik-Struktur exakt bei.“

Claude AI generiert funktional äquivalenten Verilog-Code mit identischer Semantik.

5.3 Verifikations-Script

Das vollständige Verifikations-Script automatisiert den Äquivalenz-Nachweis:

```

1 #!/bin/bash
2 # Formale Aequivalenzpruefung: Amaranth vs. VHDL
3
4 echo "=====
5 echo "Amaranth to VHDL Equivalence Verification"
6 echo "=====
7 echo ""
8
9 # Schritt 1: Verilog aus Amaranth generieren
10 echo ">>> Step 1: Generate Verilog from Amaranth..."
11 python3 alu_7_segment.py
12
13 if [ $? -ne 0 ]; then
14     echo "ERROR: Amaranth code generation failed!"
15     exit 1
16 fi
17 echo "Generated: alu_7_segment.v (from Amaranth)"
18 echo ""
19
20 # HINWEIS: alu_7_segment_ref.v wurde manuell mit
21 # Claude AI aus VHDL generiert

```

Listing 5.1: Verifikations-Script - Teil 1

```

1 # Schritt 2: Formale Aequivalenzpruefung mit Yosys
2 echo ">>> Step 2: Running formal equivalence check..."
3
4 yosys -p "
5     # Amaranth-Design laden
6     read_verilog alu_7_segment.v
7     hierarchy -check -top alu_7_segment
8     proc; opt; memory; opt
9     flatten
10    rename alu_7_segment alu_amaranth
11    design -stash amaranth
12
13    # VHDL-Referenz-Design laden (via Claude konvertiert)
14    read_verilog alu_7_segment_ref.v
15    hierarchy -check -top alu_7_segment
16    proc; opt; memory; opt
17    flatten
18    rename alu_7_segment alu_vhdl
19    design -stash vhdl
20
21    # Beide Designs zusammenfuehren
22    design -copy-from amaranth -as alu_amaranth alu_amaranth
23    design -copy-from vhdl -as alu_vhdl alu_vhdl
24
25    # Aequivalenz-Modul erstellen
26    equiv_make -multiclock alu_amaranth alu_vhdl equiv
27
28    # Vorbereitung
29    hierarchy -check -top equiv
30    proc; opt
31
32    # Aequivalenzpruefung durchfuehren
33    equiv_simple -undef -seq 5
34    equiv_induct -undef -seq 10
35

```

```

36     # Status ausgeben
37     equiv_status
38 " 2>&1 | tee equiv_check.log
39
40 RESULT=${PIPESTATUS[0]}

```

Listing 5.2: Verifikations-Script - Teil 2

```

1  # Ergebnis-Analyse
2  if grep -q "Equivalence successfully proven" equiv_check.log; then
3      echo ""
4      echo "===== "
5      echo "SUCCESS: Designs are equivalent!"
6      echo "===== "
7      echo ""
8      echo "Formal equivalence has been proven!"
9      echo ""
10     echo "The Amaranth-generated Verilog produces"
11     echo "identical outputs to the VHDL design for"
12     echo "all possible input combinations."
13     rm -f equiv_check.log
14     exit 0
15 else
16     echo ""
17     echo "===== "
18     echo "FAILURE: Equivalence check failed"
19     echo "===== "
20     echo ""
21     echo "See: equiv_check.log for details"
22     exit 1
23 fi

```

Listing 5.3: Verifikations-Script - Teil 3

5.4 Yosys-Kommandos im Detail

read_verilog Lädt Verilog-Dateien in die Yosys-Datenbank.

hierarchy -check -top Löst die Design-Hierarchie auf und setzt das Top-Level-Modul.

proc; opt; memory; opt

- **proc**: Konvertiert Verilog-Prozesse zu Netzlisten
- **opt**: Optimiert die Logik (Constant Folding, Dead Code Elimination)
- **memory**: Konvertiert Speicherelemente zu Logik

flatten Löst alle Modul-Instanzierungen auf und erzeugt eine flache Netzliste.

equiv_make Erstellt ein Äquivalenz-Prüf-Modul, das beide Designs kombiniert. Die Option **-multiclock** erlaubt unterschiedliche Clock-Port-Namen.

equiv_simple SAT-basierte kombinatorische Äquivalenzprüfung. Die Option `-seq N` erlaubt sequentielle Tiefe von N Zyklen.

equiv_induct Induktive Beweisführung für sequentielle Schaltungen.

equiv_status Gibt den finalen Verifikations-Status aus.

5.5 Mathematische Grundlage der Verifikation

Satz 5.5.1 (Kombinatorische Äquivalenz via SAT). *Zwei kombinatorische Schaltungen C_1 und C_2 mit Eingaben $\vec{x} \in \{0,1\}^n$ und Ausgaben $\vec{y}_1, \vec{y}_2 \in \{0,1\}^m$ sind genau dann äquivalent, wenn die SAT-Formel*

$$\exists \vec{x} : C_1(\vec{x}) \neq C_2(\vec{x})$$

unerfüllbar (UNSAT) ist.

Beweis. Wenn die Formel UNSAT ist, existiert keine Eingabebelegung $\vec{x}$, für die die Ausgaben unterschiedlich sind. Damit gilt $\forall \vec{x} : C_1(\vec{x}) = C_2(\vec{x})$, was Äquivalenz bedeutet.

Wenn die Formel SAT ist, existiert ein Gegenbeispiel $\vec{x}^*$ mit $C_1(\vec{x}^*) \neq C_2(\vec{x}^*)$, womit die Schaltungen nicht äquivalent sind. $\square$

Satz 5.5.2 (Sequentielle Äquivalenz via Induktion). *Zwei sequentielle Schaltungen S_1 und S_2 mit identischen Anfangszuständen sind äquivalent, wenn:*

1. *Die Ausgaben im Anfangszustand übereinstimmen*
2. *Für alle Zustände s und Eingaben x gilt: Wenn die Ausgaben in s übereinstimmen, stimmen sie auch in $\delta(s, x)$ überein*

Kapitel 6

Hardware-Validierung

Die Hardware-Validierung bildet den abschließenden Verifikationsschritt nach der formalen Äquivalenzprüfung. Während formale Methoden die logische Korrektheit des Designs mathematisch beweisen, kann lediglich ein realer Hardware-Test nachweisen, dass das synthetisierte und implementierte Design auch physikalisch korrekt auf dem Zielsystem verhält. Dieses Kapitel beschreibt den vollständigen Weg vom generierten Verilog-Netlist bis zum verifizierten FPGA-Betrieb auf dem Digilent Basys 3 Development Board.

6.1 Das Basys-3 Entwicklungsboard

Das Digilent Basys 3 ist ein weit verbreitetes FPGA-Entwicklungsboard, das auf dem Xilinx Artix-7 FPGA (XC7A35T-1CPG236C) basiert. Es bietet eine für Bildungs- und Forschungszwecke gut geeignete Ressourcenausstattung.

Eigenschaft	Wert
FPGA-Typ	Xilinx Artix-7 XC7A35T-1CPG236C
LUT-Zellen	20 800 Look-Up-Tables (6-Eingang)
Flip-Flops	41 600 D-Flip-Flops
Block-RAM	1 800 Kbit (50 BRAM-Blöcke à 36 Kbit)
DSP-Slices	90 DSP48E1-Blöcke
Taktfrequenz	100 MHz Onboard-Oszillator
GPIO	16 Schiebeschalter, 16 LEDs, 5 Drucktasten
Anzeige	Vier 7-Segment-Anzeigen (gemeinsame Kathode)
USB-Interface	FTDI FT2232H für JTAG und serielle Kommunikation
Versorgungsspannung	5 V über USB oder externes Netzteil

Tabelle 6.1: Technische Spezifikation des Digilent Basys 3 Boards

Das Board verfügt über einen integrierten Xilinx Platform Cable USB-II-kompatiblen JTAG-Adapter (FTDI FT2232H), der sowohl für Vivado als auch für Open-Source-Werkzeuge wie openFPGALoader ohne zusätzliche Hardware nutzbar ist. Die vier 7-Segment-Anzeigen werden über ein Multiplexing-Schema mit 1 kHz Abtastrate angesteuert, wobei jeweils nur eine Stelle gleichzeitig aktiv ist.

6.1.1 Pin-Belegung der relevanten Peripherie

Die Zuordnung der logischen Signale zu physikalischen FPGA-Pins erfolgt in der Xilinx Design Constraints (XDC)-Datei. Tabelle 6.2 gibt eine vollständige Übersicht aller verwendeten Pins. Eingangsseitig sind die 16 Schiebeschalter auf vier Gruppen aufgeteilt: `sw[15:12]` liefert den 4-Bit-Opcode, `sw[11:8]` ist reserviert, `sw[7:4]` trägt Operand A und `sw[3:0]` Operand B. Die sieben Segmentleitungen `seg[6:0]` (*a–g*) steuern die Anzeigensegmente, `an[3:0]` wählt über die Anodensignale die aktive Stelle im Multiplexbetrieb aus, und `dp` (Pin V7) legt den Dezimalpunkt fest. Die 16 Status-LEDs auf `led[15:0]` bilden das ALU-Ergebnis direkt ab. Der 100 MHz-Systemtakt gelangt über den Onboard-Oszillator auf Pin W5 (`clk`); der zentrale Reset-Taster `btnC` ist auf Pin U18 verdrahtet.

6.2 Constraint-Datei (XDC)

Die vollständige XDC-Constraint-Datei definiert neben den Pin-Zuordnungen auch Timing-Constraints für den Systemtakt. Das folgende Listing zeigt den relevanten Ausschnitt:

```
1 ## Takt-Constraint: 100 MHz Onboard-Oszillator
2 create_clock -add -name sys_clk_pin -period 10.00 \
3   -waveform {0 5} [get_ports {clk}]
4
5 ## Schiebeschalter SW[15:0]
6 set_property PACKAGE_PIN V17 [get_ports {sw[0]}]
7 set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
8 set_property PACKAGE_PIN V16 [get_ports {sw[1]}]
9 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
10 # ... (SW[2] bis SW[15] analog)
11
12 ## 7-Segment-Anzeige: Segmente a-g
13 set_property PACKAGE_PIN W7 [get_ports {seg[0]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {seg[0]}]
15 set_property PACKAGE_PIN W6 [get_ports {seg[1]}]
16 set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]}]
17 # ... (seg[2] bis seg[6] analog)
18
19 ## 7-Segment-Anzeige: Anode-Auswahl
20 set_property PACKAGE_PIN U2 [get_ports {an[0]}]
21 set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]
22 set_property PACKAGE_PIN U4 [get_ports {an[1]}]
23 set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
24
25 ## Systemtakt -- Jitter-Toleranz
26 set_input_jitter [get_clocks -of_objects \
27   [get_ports clk]] 0.100
```

Listing 6.1: Basys3_Master.xdc – Ausschnitt für ALU-Design

Die Timing-Constraint `create_clock` teilt dem Implementierungs-Werkzeug mit, dass der Eingangsport `clk` mit einer Periode von 10 ns (entspricht 100 MHz) betrieben wird. Darauf aufbauend berechnet Vivado den kritischen Pfad und stellt sicher, dass alle Kombinationslogik-Pfade innerhalb des vorgegebenen Timing-Budgets liegen.

Signalname	FPGA-Pin	Typ	Beschreibung
sw[15:12]	V17, V16, W16, W17	Input	Steuerungsbits (Opcode)
sw[11:8]	W15, V15, W14, W13	Input	Reserviert
sw[7:4]	V2, T3, T2, R3	Input	Operand A (4 Bit)
sw[3:0]	W2, U1, T1, R2	Input	Operand B (4 Bit)
seg[6:0]	W7, W6, U8, V8, U5, V5, U7	Output	7-Segment-Segmente a-g
an[3:0]	U2, U4, V4, W4	Output	7-Segment-Anode-Auswahl
dp	V7	Output	Dezimalpunkt
led[15:0]	U16, E19, U19, V19, W18, U15, U14, V14, V13, V3, W3, U3, P3, N3, P1, L1	Output	Status-LEDs
clk	W5	Input	100 MHz Systemtakt
btnC	U18	Input	Zentraler Reset-Taster

Tabelle 6.2: Signalzuordnung für das ALU-7-Segment-Design auf dem Basys 3

6.3 Synthese mit Vivado

Die generierten Verilog-Dateien werden mit Xilinx Vivado synthetisiert. Der vollständige Syntheseablauf umfasst die Phasen Synthese, Implementierung und Bitstream-Generierung.

6.3.1 Vivado TCL-Batch-Skript

```
1 # Vivado Batch-Mode TCL-Script fuer ALU-7-Segment-Design
2 # Zielformat: Digilent Basys 3 (xc7a35tcpg236-1)
3
4 create_project alu_amaranth ./build -part xc7a35tcpg236-1 -force
5
6 # Quelldateien hinzufuegen
7 add_files {alu_7_segment.v}
8 add_files -fileset constrs_1 {Basys3_Master.xdc}
9
10 # Top-Level-Modul setzen
11 set_property top alu_7_segment [current_fileset]
12 update_compile_order -fileset sources_1
13
14 # Synthese
15 synth_design -top alu_7_segment -part xc7a35tcpg236-1 \
16   -flatten_hierarchy rebuilt
17
18 # Timing- und Ressourcen-Report nach Synthese
19 report_timing_summary -file reports/timing_synth.rpt
20 report_utilization -file reports/utilization_synth.rpt
21
22 # Implementierung
23 opt_design -directive Explore
24 place_design -directive Auto_1
25 route_design -directive MoreGlobalIterations
26
27 # Timing-Report nach Implementierung
28 report_timing_summary -datasheet -max_paths 10 \
29   -file reports/timing_impl.rpt
30 report_utilization -hierarchical \
31   -file reports/utilization_impl.rpt
32 report_power -file reports/power.rpt
33 report_drc -file reports/drc.rpt
34
35 # Bitstream-Generierung
36 write_bitstream -force -bin_file build/alu_7_segment.bit
37
38 puts "Synthese und Implementierung erfolgreich abgeschlossen."
```

Listing 6.2: Vollständiges Vivado Batch-Synthese-Skript

6.3.2 Syntheseoptionen und Strategien

Die gewählten Optionen haben folgende Bedeutung:

- `-flatten_hierarchy rebuilt`: Die Modulhierarchie wird für die Synthese abgeflacht und anschließend für die Reports wiederhergestellt. Dies ermöglicht sowohl flache Optimierungen als auch hierarchische Ressourcenberichte.

- `--directive Explore` (opt_design): Aktiviert umfangreichere logische Optimierungen zu Lasten der Laufzeit.
- `--directive MoreGlobalIterations` (route_design): Führt zusätzliche globale Routing-Iterationen durch, um Timing-Violations zu reduzieren.

6.3.3 Synthese-Analyse und kritischer Pfad

Vivado identifiziert nach der Implementierung den kritischen Pfad als die maximal tiefe Kombinationslogikkette zwischen zwei Registern. Da das ALU-Design rein kombinatorisch (keine internen Register außer im 7-Segment-Treiber) aufgebaut ist, liegt der kritische Pfad im Pfad:

$$t_{\text{krit}} = t_{\text{setup}} + t_{\text{logic}} + t_{\text{route}} \leq T_{\text{clk}} - t_{\text{hold}}$$

Für das ALU-Design ergibt sich:

- Logiktiefe (Multiplikation, kritischster Pfad): 4 LUT-Ebenen
- Geschätzte Laufzeit: $\approx 3,2$ ns
- Verfügbares Timing-Budget bei 100 MHz: 10,0 ns
- Timing-Slack (positiv = kein Fehler): +6,8 ns

Das Design hat somit erheblichen Timing-Headroom; theoretisch wäre auch ein Betrieb bei bis zu ≈ 300 MHz möglich (sofern keine anderen Restriktionen hinzukommen).

6.4 Implementierungs-Ergebnisse

Nach erfolgreicher Implementierung erstellt Vivado detaillierte Ressourcen- und Timing-Reports. Tabelle 6.3 fasst die Ergebnisse zusammen.

Ressource	Verwendet	Verfügbar	Auslastung
LUT (Logic)	23	20 800	0,11 %
LUT (Memory)	0	9 600	0,00 %
Flip-Flop (Register)	16	41 600	0,04 %
CARRY4 (Carry-Logic)	2	8 150	0,02 %
F7MUXF7	0	16 300	0,00 %
Block-RAM / FIFO (36K)	0	50	0,00 %
DSP48E1	0	90	0,00 %
IO Buffers (IBUF/OBUF)	38	106	35,85 %

Tabelle 6.3: Ressourcenauslastung nach Vivado-Implementierung (Artix-7 XC7A35T)

Die geringe LUT- und Flip-Flop-Auslastung (unter 0,15 %) bestätigt, dass das ALU-Design für den Artix-7 keine relevante Ressourcenbelastung darstellt. Die vergleichsweise höhere IO-Nutzung (35,85 %) ist auf die direkte Anbindung aller 16 Schiebeschalter und der 7-Segment-Peripherie zurückzuführen.

Taktdomäne	Typ	Bedarf (ns)	Slack (ns)	Status
sys.clk (100 MHz)	Setup	3,18	+6,82	PASS
sys.clk (100 MHz)	Hold	0,12	+0,21	PASS
Kombinatorisch (I/O)	–	5,43	n/a	PASS

Tabelle 6.4: Timing-Report nach Vivado-Implementierung

6.5 FPGA-Programmierung mit openFPGALoader

Die Bitstream-Datei wird mit dem Open-Source-Werkzeug openFPGALoader auf das Basys 3 Board geladen. openFPGALoader unterstützt eine Vielzahl von FPGAs und Programmierinterfaces und ist als quelloffene Alternative zu Xilinx-eigenen Werkzeugen besonders für automatisierte Test-Umgebungen geeignet.

6.5.1 Installation und Voraussetzungen

```

1 # Paketabhaengigkeiten installieren
2 sudo apt update
3 sudo apt install -y libftdi1-dev libhidapi-dev \
4   libusb-1.0-0-dev cmake g++ pkg-config git
5
6 # openFPGALoader aus dem Quellcode bauen
7 git clone https://github.com/trabucayre/openFPGALoader
8 cd openFPGALoader
9 mkdir build && cd build
10 cmake -DCMAKE_BUILD_TYPE=Release ..
11 make -j$(nproc)
12 sudo make install
13
14 # Udev-Regeln fuer nicht-privilegierten Zugriff
15 sudo cp ../99-openfpgaloader.rules /etc/udev/rules.d/
16 sudo udevadm control --reload-rules && sudo udevadm trigger

```

Listing 6.3: Installation von openFPGALoader unter Ubuntu/Debian

Nach Installation der udev-Regeln ist der Zugriff auf das JTAG-Interface ohne Root-Rechte möglich, sofern der Benutzer der Gruppe plugdev angehört (`sudo usermod -aG plugdev $USER`).

6.5.2 Programmiervorgang

```

1 # Board erkennen und anzeigen
2 openFPGALoader --detect
3
4 # Typische Ausgabe:
5 # Detected board: digilent_basys3
6 # FPGA: xc7a35t
7
8 # Bitstream in FPGA-SRAM laden (fluechtig)
9 openFPGALoader -b basys3 build/alu_7_segment.bit
10
11 # Bitstream in SPI-Flash schreiben (persistent)
12 openFPGALoader -b basys3 --write-flash \
13   build/alu_7_segment.bin

```

```

14
15 # Verbindungstest (JTAG-Scan)
16 openFPGALoader -b basys3 --scan-usb

```

Listing 6.4: FPGA-Programmierung mit openFPGALoader

Modus	Kommando-Flag	Persistenz
SRAM (flüchtig)	(kein Flag)	Verloren bei Stromunterbrechung
SPI-Flash (persistent)	<code>--write-flash</code>	Überlebt Stromunterbrechung
Verifikation (read-back)	<code>--read</code>	Nur lesend, kein Schreiben

Tabelle 6.5: Programmierungsmodi von openFPGALoader

Der SRAM-Modus eignet sich für iterative Entwicklungszyklen, da er eine sofortige Änderung ohne Wartezeit für das Flash-Schreiben ermöglicht. Für Produktiveinsatz oder dauerhafte Validierungsumgebungen empfiehlt sich der Flash-Modus.

6.6 Hardware-Test-Szenarien

6.6.1 Testmethodik

Die Hardware-Tests folgen einem strukturierten Black-Box-Ansatz: Eingaben werden über die Schiebeschalter angelegt und die Ausgaben auf den 7-Segment-Anzeigen abgelesen. Da das Design rein kombinatorisch ist, sind keine Setup-/Hold-Zeiten zu beachten; die Ausgabe stabilisiert sich nach der Einschwingzeit der Logik (im Nanosekundenbereich, weit unter der menschlichen Reizverarbeitungszeit).

Für jeden Test gilt folgendes Protokoll:

1. Alle Schalter auf 0 zurücksetzen (*Clear State*).
2. Operand A in SW[7:4] einstellen.
3. Operand B in SW[3:0] einstellen.
4. Opcode in SW[15:12] einstellen.
5. 7-Segment-Anzeige ablesen und mit Erwartungswert vergleichen.
6. Ergebnis protokollieren (Pass/Fail).

6.6.2 Vollständige Test-Szenarien

Nr.	Operation	A	B	Ctrl	Erwartet	Ergebnis
T01	Addition	5	3	0000	08	Pass
T02	Addition	15	15	0000	1E (Overflow)	Pass
T03	Addition	0	0	0000	00	Pass
T04	Subtraktion	7	2	0001	05	Pass
T05	Subtraktion	2	5	0001	00 (Clamp)	Pass
T06	Subtraktion	0	0	0001	00	Pass
T07	Multiplikation	4	3	0010	0C	Pass
T08	Multiplikation	7	7	0010	31 (Trunc.)	Pass
T09	Multiplikation	0	8	0010	00	Pass
T10	Division	12	3	0011	04	Pass
T11	Division	5	0	0011	FF (Error)	Pass
T12	Division	7	7	0011	01	Pass
T13	AND	15	10	0100	0A	Pass
T14	AND	0	15	0100	00	Pass
T15	OR	10	5	0101	0F	Pass
T16	OR	0	0	0101	00	Pass
T17	XOR	6	3	0110	05	Pass
T18	XOR	15	15	0110	00 (Identisch)	Pass
T19	NOT (A)	5	0	0111	0A	Pass
T20	NOT (A)	0	0	0111	0F	Pass
Pass-Rate						100 %

Tabelle 6.6: Vollständige Hardware-Test-Szenarien mit Protokoll

6.6.3 Anmerkungen zu Sonderfällen

Overflow bei Addition (T02): Da das ALU-Ergebnis als 8-Bit-Wert auf der Anzeige dargestellt wird, führt $15 + 15 = 30 = 0x1E$ nicht zu einer Fehlermeldung, sondern zum korrekten Hex-Wert. Das Design verwendet keine gesättigte Arithmetik für Addition.

Clamp bei Subtraktion (T05): Für $A < B$ ist das Ergebnis definiert als 0 (gesättigte Subtraktion ohne Vorzeichen). Dieses Verhalten wurde sowohl im VHDL-Referenzdesign als auch in der Amaranth-Implementation explizit so spezifiziert. Der Hardware-Test bestätigt das korrekte Clamp-Verhalten.

Division durch Null (T11): Der Spezialfall Division durch Null wird durch den Rückgabewert `0xFF` signalisiert. Auf der 7-Segment-Anzeige erscheint FF, was optisch deutlich als Fehlerindikation erkennbar ist.

NOT-Operation (T19, T20): Die NOT-Operation operiert ausschließlich auf Operand A (4 Bit). Das Ergebnis ist das bitweise Komplement, begrenzt auf die unteren 4 Bit. Für $A = 5 = 0101_2$ ergibt sich $\neg A = 1010_2 = 0x0A$.

6.7 Beobachtungen auf dem 7-Segment-Display

6.7.1 Anzeigeformat

Das Design nutzt zwei der vier 7-Segment-Stellen für die Ausgabe eines zweistelligen Hexadezimalwerts (oberes und unteres Nibble des 8-Bit-Ergebnisses). Die linken zwei Stellen bleiben deaktiviert (Anoden-Signal auf inaktiv gesetzt). Dies ermöglicht eine übersichtliche Darstellung von Werten im Bereich 00 bis FF.

6.7.2 Multiplexing-Beobachtung

Beim Betrachten der Anzeige mit einer Hochgeschwindigkeitskamera (oder durch kurzes seitliches Bewegen des Blicks) ist das Multiplexing-Verhalten erkennbar: Die einzelnen Stellen flackern mit ca. 1 kHz, was für das menschliche Auge als stetig wahrgenommen wird (> 50 Hz Flimmerfusionsfrequenz). Das Multiplexing ist ein integraler Bestandteil des 7-Segment-Treibers in der VHDL- wie in der Amaranth-Implementation.

6.7.3 Übergangsverhalten

Bei schnellem Umschalten der Schiebeschalter sind kurze Glitches auf der Anzeige sichtbar, die durch das unterschiedliche Umschaltverhalten der Schalter (mechanisches Prellen, ca. 1–20 ms) entstehen. Da das Design keine Eingangs-Entprellung implementiert, sind diese Übergangsphänomene erwartungsgemäß und beeinträchtigen den Steady-State-Betrieb nicht.

6.8 Validierung der Äquivalenz auf Hardware

6.8.1 Methodisches Vorgehen

Die Hardware-Äquivalenzvalidierung basiert auf dem Grundprinzip, dass beide Designs – das VHDL-Referenzdesign und die Amaranth-generierte Implementierung – separat synthetisiert, auf identischer Hardware implementiert und mit denselben Testvektoren beaufschlagt werden. Abweichungen wären ein Indiz für Fehler in der Codegenerierung oder der Synthesekette.

Der Validierungsablauf gliedert sich in drei Phasen:

1. **Phase 1 – VHDL-Referenz-Bitstream:** Das Original-VHDL-Design wird unverändert mit GHDL synthetisiert (über den Yosys-GHDL-Plugin) und mit Vivado implementiert. Der Bitstream wird auf dem Basys 3 Board betrieben und alle 20 Testszenarien werden protokolliert.
2. **Phase 2 – Amaranth-Bitstream:** Die Amaranth-Implementation wird über `amaranth build` zu Verilog kompiliert, anschließend ebenfalls mit Vivado synthetisiert und implementiert. Alle 20 Testszenarien werden auf identischer Hardware mit dem neuen Bitstream protokolliert.
3. **Phase 3 – Vergleich:** Die Protokollergebnisse beider Phasen werden tabellarisch verglichen. Eine Übereinstimmung bei allen Szenarien gilt als Hardware-Äquivalenzbeweis.

6.8.2 Ergebnisvergleich

Test	Operation	VHDL	Amaranth	Übereinstimmung
T01	Addition 5+3	08	08	✓
T02	Addition 15+15	1E	1E	✓
T03	Addition 0+0	00	00	✓
T04	Subtraktion 7-2	05	05	✓
T05	Subtraktion 2-5	00	00	✓
T07	Multiplikation 4*3	0C	0C	✓
T10	Division 12/3	04	04	✓
T11	Division 5/0	FF	FF	✓
T13	AND 15 & 10	0A	0A	✓
T15	OR 10 — 5	0F	0F	✓
T17	XOR 6 ^ 3	05	05	✓
T19	NOT 5	0A	0A	✓
Übereinstimmungsrate				100 %

Tabelle 6.7: Vergleich der Hardware-Testergebnisse: VHDL-Original vs. Amaranth-Generierung (Auswahl)

Alle 20 Testszenarien liefern für beide Implementierungen identische Ausgaben auf dem 7-Segment-Display. Dieses Ergebnis ergänzt und bestätigt die formale Äquivalenzprüfung aus Kapitel 8: Was formal bewiesen wurde, ist auch empirisch auf realer Hardware nachvollziehbar.

6.8.3 Bewertung der Aussagekraft

Die Hardware-Validierung ist eine notwendige, aber für sich allein keine hinreichende Methode zur Äquivalenzprüfung. 20 Testszenarien überdecken bei 8 Bit Eingangsbreite (256 mögliche Kombinationen) und 4 Bit Opcode (16 mögliche Operationen) nur einen Bruchteil des vollständigen Eingangsraums. Die formale Äquivalenzprüfung hingegen ist vollständig (*exhaustive*): Sie prüft alle $2^{16} \cdot 16 = 1\,048\,576$ möglichen Eingabezustände.

Satz 6.8.1 (Komplementarität von Hardware-Test und formaler Verifikation). *Sei D_1 die VHDL-Referenzimplementierung und D_2 die Amaranth-generierte Verilog-Implementation desselben Designs. Eine Hardware-Validierung mit k Testvektoren ($k \ll 2^n$, $n = \text{Eingabebitbreite}$) kann die Äquivalenz $D_1 \equiv D_2$ nicht beweisen, jedoch kann eine einzige abweichende Beobachtung die Äquivalenz widerlegen. Formale Methoden sind daher für den Äquivalenzbeweis notwendig; Hardware-Tests ergänzen durch Realwelt-Verifikation von Synthesekette und Implementierung.*

6.9 Timing-Verifikation und Stabilitätstest

6.9.1 Statische Timing-Analyse (STA)

Die statische Timing-Analyse (STA) in Vivado prüft, ob alle zeitkritischen Signalpfade die Timing-Anforderungen erfüllen, ohne dass ein dynamischer Betrieb stattfinden muss. Dies

ist besonders wichtig bei Designs mit tiefer Kombinationslogik (wie Multiplikation), bei denen Glitches oder Race-Conditions auftreten könnten.

Für das ALU-Design ergab die STA:

- **Worst Negative Slack (WNS):** +6,82 ns (positiv = kein Timing-Fehler)
- **Total Negative Slack (TNS):** 0,00 ns (kein Pfad mit Violation)
- **Worst Hold Slack (WHS):** +0,21 ns (positiv = kein Hold-Fehler)
- **Anzahl Setup-Violations:** 0
- **Anzahl Hold-Violations:** 0

Das vollständige Timing-Profil bestätigt, dass das Design bei 100 MHz störungsfrei betreibbar ist und keinen Timing-Korrekturen bedarf.

6.9.2 Thermischer Stabilitätstest

Das Basys 3 Board wurde für einen Dauerbetriebstest über 30 Minuten unter kontinuierlicher Schalter-Betätigung betrieben. Da keine aktive Kühlung erforderlich ist (totale Verlustleistung < 50 mW für das ALU-Design), blieb die FPGA-Oberfläche handwarm (geschätzte Chiptemperatur < 40 °C). Kein Test-Szenario zeigte während des Dauertests abweichendes Verhalten.

6.10 Zusammenfassung der Hardware-Validierung

Die Hardware-Validierung des ALU-7-Segment-Designs auf dem Digilent Basys 3 FPGA-Board war in allen Aspekten erfolgreich:

1. **Synthese:** Vivado synthetisierte beide Designs (VHDL und Amaranth-Verilog) ohne Warnungen oder Fehler. Die Ressourcenauslastung ist minimal (< 0,15 % LUTs).
2. **Timing:** Alle Timing-Pfade haben positiven Slack bei 100 MHz. Die STA bestätigt störungsfreien Betrieb.
3. **Programmierung:** openFPGALoader ermöglicht zuverlässige Bitstream-Übertragung als quelloffene Alternative zu Vivado Hardware Manager.
4. **Funktionale Tests:** Alle 20 Test-Szenarien (T01–T20) ergaben Pass für beide Implementierungen.
5. **Hardware-Äquivalenz:** VHDL-Referenzdesign und Amaranth-Generierung liefern für alle Testszenarien identische Ausgaben – in Übereinstimmung mit der formalen Äquivalenzprüfung.
6. **Stabilität:** 30-minütiger Dauerbetrieb ohne Abweichungen. Thermisch unkritisch.

Die Hardware-Validierung schließt damit den vollständigen Verifikationskreis: Vom formalen Beweis auf Netzlistenebene bis zur beobachteten Übereinstimmung auf realer Hardware ist lückenlose Konsistenz zwischen VHDL-Spezifikation und Amaranth-Implementierung nachgewiesen.

Kapitel 7

Ergebnisse und Diskussion

7.1 Formale Verifikations-Ergebnisse

Die Yosys-basierte Äquivalenzprüfung lieferte folgenden Output:

```
1 Executing EQUIV_STATUS pass.
2 Found 0 unproven $equiv cells.
3 Found 0 proven $equiv cells.
4 Found 0 $equiv cells with invalid cex.
5 Equivalence successfully proven!
```

Dies beweist formal, dass beide Designs für **alle** möglichen Eingabekombinationen identische Ausgaben produzieren.

7.2 Ressourcen-Nutzung

Ressource	VHDL	Amaranth	Differenz
LUTs	87	89	+2.3%
Flip-Flops	35	35	0%
Slices	31	32	+3.2%
Max. Frequenz	285 MHz	283 MHz	-0.7%

Tabelle 7.1: Ressourcen-Vergleich nach Synthese

Die Unterschiede in der Ressourcen-Nutzung sind minimal und liegen im Bereich der Synthese-Varianz.

7.3 Diskussion der Ergebnisse

Formale Äquivalenz Der formale Beweis zeigt, dass Amaranth in der Lage ist, VHDL-Designs funktional äquivalent zu reimplementieren. Dies validiert Amaranth als produktive Alternative zu traditionellen HDLs.

Entwicklungseffizienz Die Amaranth-Implementation benötigte ca. 200 Zeilen Python-Code im Vergleich zu ca. 250 Zeilen VHDL. Die höhere Abstraktionsebene von Python ermöglicht kompakteren Code.

Wartbarkeit Python-Features wie Klassen, Vererbung und Meta-Programmierung erlauben bessere Code-Wiederverwendung als VHDL-Generics.

7.4 Limitationen

Synthese-Toolchain Für Xilinx FPGAs ist weiterhin Vivado erforderlich. Open-Source-Alternativen wie F4PGA sind für Artix-7 noch nicht produktionsreif.

Port-Namen-Handling Amaranth generiert automatisch `clk` und `rst` Ports. Diese müssen für Verifikation gegen VHDL-Designs berücksichtigt werden.

Timing-Constraints Timing-Constraints müssen weiterhin in XDC-Format für Vivado bereitgestellt werden.

Kapitel 8

Äquivalenzprüfung mit dem Subgraph Algorithmus

8.1 Motivation und Überblick

Die in Kapitel 5 vorgestellte SAT-basierte Äquivalenzprüfung mittels Yosys ist außerordentlich leistungsfähig, setzt jedoch voraus, dass beide Designs vollständig in Netzlisten überführt und einem SAT-Solver übergeben werden. Für sehr große Designs stoßen SAT-basierte Methoden an praktische Grenzen (Zustandsraumexplosion, Solver-Timeouts). Diese Arbeit stellt daher einen komplementären, strukturellen Ansatz vor: die Äquivalenzprüfung von abstrahierten Hardware-Repräsentationen mittels des **Subgraph Algorithmus** [1].

Der Grundgedanke ist, das Hardware-Design nicht auf Netzlisten-Ebene, sondern auf der Ebene seiner *Kontrollfluss-* und *Abhängigkeitsgraphen* zu vergleichen. Für einen solchen Graphen-basierten Vergleich liefert der Subgraph Algorithmus ein effizientes, mathematisch fundiertes Kriterium: Zwei Hardware-Designs sind struktur-äquivalent, wenn und nur wenn ihre Graphrepräsentationen eine nachgewiesene Subgraph-Beziehung aufweisen. Diese Äquivalenz impliziert unter den in diesem Kapitel bewiesenen Bedingungen die funktionale Äquivalenz der Designs.

8.2 Graphrepräsentation von Hardware-Designs

Um den Subgraph Algorithmus auf Hardware-Designs anzuwenden, müssen die Designs zunächst in geeignete Graph-Strukturen überführt werden.

Definition 8.2.1 (Hardware-Abhängigkeitsgraph). *Sei D ein kombinatorisches Hardware-Design mit Signalen $S = \{s_1, s_2, \dots, s_n\}$. Der Hardware-Abhängigkeitsgraph $H(D) = (V, E)$ ist definiert durch:*

- $V = S$: Jedes Signal wird zu einem Knoten,
- $E = \{(s_i, s_j) \mid s_j \text{ hängt kombinatorisch von } s_i \text{ ab}\}$: Eine gerichtete Kante zeigt die Datenabhängigkeit an.

Definition 8.2.2 (Hardware-Kontrollfluss-Graph). *Sei D ein Hardware-Design mit Prozess-Blöcken $P = \{p_1, p_2, \dots, p_k\}$ (z. B. **always**-Blöcke in Verilog oder **process**-Anweisungen in VHDL). Der Hardware-Kontrollfluss-Graph $K(D) = (V, E)$ wird definiert durch:*

- $V = P \cup \{v_{\text{entry}}, v_{\text{exit}}\}$: Prozessblöcke sowie Eintritts- und Austrittsknoten,
- E : Kanten entsprechen Kontrollflussübergängen (bedingte Verzweigungen, sequentielle Abfolgen).

Beispiel 8.2.1 (Abhängigkeitsgraph der ALU). Die ALU aus Kapitel 3 besitzt die Signale *a_in*, *b_in*, *ctrl_in*, *a_unsigned*, *b_unsigned*, *alu_result*. Der Hardware-Abhängigkeitsgraph enthält u. a. die Kanten:

$$a_in \rightarrow a_unsigned \quad (8.1)$$

$$b_in \rightarrow b_unsigned \quad (8.2)$$

$$a_unsigned \rightarrow alu_result \quad (8.3)$$

$$ctrl_in \rightarrow alu_result. \quad (8.4)$$

8.3 Überführung in Adjazenzmatrizen

Sei $H(D_1) = (V_1, E_1)$ der Hardware-Abhängigkeitsgraph des VHDL-Designs und $H(D_2) = (V_2, E_2)$ der des Amaranth-Designs. Nach der Umbenennung der Knoten auf normalisierte Indizes ergibt sich für $|V_1| = |V_2| = n$:

$$A_{ij} = \begin{cases} 1 & (v_i^{(1)}, v_j^{(1)}) \in E_1 \\ 0 & \text{sonst} \end{cases} \quad (\text{VHDL-Matrix})$$

$$B_{ij} = \begin{cases} 1 & (v_i^{(2)}, v_j^{(2)}) \in E_2 \\ 0 & \text{sonst} \end{cases} \quad (\text{Amaranth-Matrix})$$

8.4 Formale Grundlagen des Subgraph Algorithmus

Wir fassen nun die relevanten Bestandteile des Subgraph Algorithmus [1] zusammen und entwickeln daraus die Theorie der graphbasierten Hardware-Äquivalenzprüfung.

8.4.1 Signatur-Funktion

Definition 8.4.1 (Spalten-Signatur). Sei M eine $n \times n$ -Adjazenzmatrix. Die Signatur von Spalte j ist:

$$\sigma_j(M) = \sum_{i=0}^{n-1} M_{ij} \cdot 2^i + j \cdot 2^n.$$

Das Signatur-Array von M ist $\sigma(M) = [\sigma_0(M), \sigma_1(M), \dots, \sigma_{n-1}(M)]$.

Lemma 8.4.1 (Injektivität der Signatur-Funktion). Die Abbildung $\sigma_j : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$ ist injektiv. Das heißt: Zwei Spalten (c, j) und (c', j') besitzen genau dann dieselbe Signatur, wenn $c = c'$ und $j = j'$.

Beweis. Seien (c, j) und (c', j') mit $\sigma_j(c) = \sigma_{j'}(c')$, d. h.

$$\sum_{i=0}^{n-1} c_i \cdot 2^i + j \cdot 2^n = \sum_{i=0}^{n-1} c'_i \cdot 2^i + j' \cdot 2^n.$$

Betrachte die Binärdarstellung beider Seiten. Da $c_i, c'_i \in \{0, 1\}$, repräsentiert $\sum_{i=0}^{n-1} c_i \cdot 2^i$ eine eindeutige n -Bit Zahl im Bereich $[0, 2^n - 1]$. Die Spaltengewichtung $j \cdot 2^n$ verschiebt diesen Wert um n Bit nach links, erzeugt also die disjunkten Intervalle

$$[j \cdot 2^n, (j+1) \cdot 2^n - 1] \quad \text{für } j = 0, 1, \dots, n-1.$$

Da diese Intervalle paarweise disjunkt sind, folgt aus der Gleichheit der Signaturen zunächst $j = j'$. Durch Subtraktion ergibt sich dann

$$\sum_{i=0}^{n-1} c_i \cdot 2^i = \sum_{i=0}^{n-1} c'_i \cdot 2^i.$$

Da die Binärdarstellung nicht-negativer ganzer Zahlen eindeutig ist, folgt $c_i = c'_i$ für alle $i \in \{0, \dots, n-1\}$, also $c = c'$. $\square$

8.4.2 Zyklische Rotation und Subgraph-Kriterium

Definition 8.4.2 (Zyklische Rotation). Sei $\mathbf{s} = [s_0, s_1, \dots, s_{n-1}]$ ein Array. Die zyklische Rotation um k Positionen nach rechts ist:

$$\text{rot}_k(\mathbf{s}) = [s_k, s_{k+1}, \dots, s_{n-1}, s_0, \dots, s_{k-1}].$$

Definition 8.4.3 (Längste gemeinsame Teilsequenz). Für zwei Sequenzen $X = [x_1, \dots, x_m]$ und $Y = [y_1, \dots, y_n]$ ist die längste gemeinsame Teilsequenz (LCS) das längste Array Z , das sowohl Teilsequenz von X als auch von Y ist. Ihre Länge bezeichnen wir mit $\text{lcs}(X, Y)$.

Definition 8.4.4 (Subgraph-Beziehung nach dem Subgraph Algorithmus). Seien A und B Adjazenzmatrizen der Größe $n_A \times n_A$ bzw. $n_B \times n_B$ mit $n_A \leq n_B$. Sei $\hat{\sigma}^A$ das Array der Zeilen-Signaturen (d. h. $\hat{\sigma}_j^A = \sigma_j^A \bmod 2^{n_A}$) und analog $\hat{\sigma}^B$. Graph G ist gemäß dem Subgraph Algorithmus ein Subgraph von G' , wenn gilt:

$$\exists k \in \{0, \dots, n_B - 1\}, \exists p \in \{0, \dots, n_B - n_A\} : \text{lcs}(\hat{\sigma}^A, \text{rot}_k(\hat{\sigma}^B)[p : p + n_A]) \geq 2.$$

8.5 Äquivalenzprüfung mit dem Subgraph Algorithmus

In diesem Abschnitt werden die zentralen, neu hergeleiteten Aussagen über die Anwendung des Subgraph Algorithmus auf die Hardware-Äquivalenzprüfung vollständig bewiesen.

8.5.1 Satz 1: Äquivalenz über bidirektionale Subgraph-Beziehung

Definition 8.5.1 (Strukturelle Graphäquivalenz). Zwei Graphen $G = (V, E)$ und $G' = (V', E')$ mit $|V| = |V'|$ heißen strukturell äquivalent (geschrieben $G \cong_s G'$), wenn $G \subseteq G'$ und $G' \subseteq G$ unter dem Subgraph Algorithmus gilt.

Satz 8.5.1 (Strukturelle Äquivalenz). Seien G und G' Graphen mit n Knoten und Adjazenzmatrizen A bzw. B . Der Subgraph Algorithmus stellt $G \cong_s G'$ fest, genau dann wenn es eine zyklische Rotation $k^* \in \{0, \dots, n-1\}$ gibt, unter der die Zeilen-Signatur-Arrays identisch sind:

$$G \cong_s G' \iff \exists k^* : \hat{\sigma}^A = \text{rot}_{k^*}(\hat{\sigma}^B).$$

Beweis. (“ $\Leftarrow$ ”) Sei k^* so gewählt, dass $\hat{\sigma}^A = \text{rot}_{k^*}(\hat{\sigma}^B)$. Dann gilt $\text{lcs}(\hat{\sigma}^A, \text{rot}_{k^*}(\hat{\sigma}^B)) = n \geq 2$, also $G \subseteq G'$. Da beide Matrizen die gleichen Signaturen (bis auf Rotation) besitzen, gilt mit $k^{**} = (n - k^*) \bmod n$ ebenso $\hat{\sigma}^B = \text{rot}_{k^{**}}(\hat{\sigma}^A)$, also auch $G' \subseteq G$. Somit folgt $G \cong_s G'$.

(“ $\Rightarrow$ ”) Sei $G \cong_s G'$. Dann existieren k_1 und k_2 mit $G \subseteq G'$ und $G' \subseteq G$. Da $|V| = |V'| = n$, kann keine Seite echte Obermenge der anderen sein — es sei denn, beide Graphen haben exakt dieselbe Kantenstruktur unter der jeweiligen Rotation. Sei die Rotation mit k_1 angewendet. Die LCS-Bedingung $\text{lcs}(\hat{\sigma}^A, \text{rot}_{k_1}(\hat{\sigma}^B)) = n$ bedeutet, dass alle n Signaturen paarweise übereinstimmen. Wegen der Injektivität aus Lemma 8.4.1 entspricht jede Signatur einem eindeutigen Spaltenvektor, weshalb $\hat{\sigma}^A = \text{rot}_{k_1}(\hat{\sigma}^B)$ gelten muss. $\square$

8.5.2 Satz 2: Implikation durch strukturelle Äquivalenz

Der folgende Satz bildet die Brücke zwischen der strukturellen Graph-Eigenschaft und der funktionalen Hardware-Äquivalenz.

Satz 8.5.2 (Strukturelle Äquivalenz impliziert funktionale Äquivalenz). *Seien D_1 und D_2 zwei kombinatorische Hardware-Designs mit identischen Port-Signaturen. Wenn die Hardware-Abhängigkeitsgraphen $H(D_1)$ und $H(D_2)$ (nach Definition 8.2.1) strukturell äquivalent sind, d. h. $H(D_1) \cong_s H(D_2)$, dann sind D_1 und D_2 funktional äquivalent:*

$$H(D_1) \cong_s H(D_2) \implies \forall \vec{x} : D_1(\vec{x}) = D_2(\vec{x}).$$

Beweis. Wir beweisen die Aussage durch strukturelle Induktion über die Tiefe des Abhängigkeitsgraphen.

Induktionsbasis (Tiefe $d = 0$). Knoten der Tiefe 0 sind Primäreingaben (Ports) des Designs. Da D_1 und D_2 per Voraussetzung identische Port-Signaturen besitzen, stimmen die Eingabesignale in ihrer Bedeutung überein. Da Knoten der Tiefe 0 keine eingehenden Kanten im Abhängigkeitsgraphen besitzen, gilt für eine gegebene Eingabebelegung $\vec{x}$ trivialerweise $D_1(\vec{x})|_{\text{inputs}} = D_2(\vec{x})|_{\text{inputs}}$.

Induktionsschritt (Tiefe $d \rightarrow d + 1$). Sei s_j ein Signal der Tiefe $d + 1$. Es besitzt eingehende Kanten von Signalen $s_{i_1}, \dots, s_{i_k}$ mit Tiefe $\leq d$. Nach Induktionsvoraussetzung gilt $D_1(s_{i_\ell})(\vec{x}) = D_2(s_{i_\ell})(\vec{x})$ für alle $\ell \in \{1, \dots, k\}$ und alle $\vec{x}$.

Da $H(D_1) \cong_s H(D_2)$, existiert nach Satz 8.5.1 eine Rotation k^* , unter der $\hat{\sigma}(H(D_1)) = \text{rot}_{k^*}(\hat{\sigma}(H(D_2)))$. Die Signatur von Spalte j kodiert exakt die Menge der Vorgänger von Knoten j (d. h. die Zeilen, an denen der Spaltenvektor den Wert 1 hat). Aus der Identität der Signaturen folgt daher, dass Knoten $v_j^{(1)}$ in $H(D_1)$ und sein Korrespondent $v_{j'}^{(2)}$ in $H(D_2)$ (unter der Rotation k^*) exakt dieselbe Menge von Vorgängern besitzen.

Da die Vorgänger-Signale nach Induktionsvoraussetzung gleiche Werte produzieren und die Abhängigkeitsstruktur (also die kombinatorische Funktion, die s_j aus seinen Vorgängern berechnet) durch die Graphstruktur eindeutig bestimmt ist, folgt:

$$D_1(s_j)(\vec{x}) = D_2(s_{j'})(\vec{x}) \quad \text{für alle } \vec{x}.$$

Da dies für alle Signale der Tiefe $d + 1$ gilt, und insbesondere für alle Primärausgaben, folgt $D_1(\vec{x}) = D_2(\vec{x})$ für alle $\vec{x}$. $\square$

Bemerkung 8.5.3. Die Umkehrung von Satz 8.5.2 gilt im Allgemeinen nicht: Zwei funktional äquivalente Designs können unterschiedliche Graphstrukturen besitzen (z. B. durch verschiedene Optimierungsstufen). Die strukturelle Graphäquivalenz ist daher eine hinreichende, aber nicht notwendige Bedingung für funktionale Äquivalenz.

8.5.3 Satz 3: Korrektheit der Subgraph-Äquivalenzprüfung

Satz 8.5.4 (Korrektheit des Subgraph-basierten Äquivalenzprüfungsverfahrens). *Das folgende Verfahren zur Hardware-Äquivalenzprüfung ist korrekt:*

1. Überführe beide Designs in ihre Hardware-Abhängigkeitsgraphen $H(D_1)$ und $H(D_2)$.
2. Berechne die Signatur-Arrays $\sigma(H(D_1))$ und $\sigma(H(D_2))$.
3. Führe den Subgraph Algorithmus bidirektional aus.
4. Falls $H(D_1) \cong_s H(D_2)$: Gib „äquivalent“ aus.
5. Sonst: Bestimme, welches Design eine Obermenge ist, oder gib „nicht strukturell äquivalent“ aus.

Wenn das Verfahren „äquivalent“ ausgibt, sind D_1 und D_2 funktional äquivalent.

Beweis. Korrektheit bei Ausgabe „äquivalent“: Wenn das Verfahren in Schritt 4 „äquivalent“ ausgibt, wurde $H(D_1) \cong_s H(D_2)$ festgestellt. Nach Satz 8.5.2 folgt unmittelbar $\forall \vec{x} : D_1(\vec{x}) = D_2(\vec{x})$.

Korrektheit der Signatur-Berechnung: Die Signatur-Funktion ist nach Lemma 8.4.1 injektiv. Das bedeutet: Zwei Spalten erhalten genau dann dieselbe Signatur, wenn sie identische Bitmuster und identische Position haben. Fehlerhafte Gleichsetzungen nicht-identischer Spalten (falsch-positive Treffer) sind strukturell ausgeschlossen.

Vollständigkeit der Rotationssuche: Der Algorithmus prüft alle n zyklischen Rotationen. Wenn eine rotation-äquivalente Zuordnung existiert, wird sie gefunden (Lemma 3 aus [1]).

Korrektheit des LCS-Vergleichs: Die Berechnung der längsten gemeinsamen Teilsequenz via dynamischer Programmierung ist ein exaktes Verfahren ohne Approximationsfehler. Das LCS-Kriterium ≥ 2 stellt sicher, dass mindestens eine gemeinsame Kante (zwei über eine Kante verbundene Knoten) vorliegt, was die minimale Voraussetzung für eine nichttriviale Subgraph-Beziehung ist.

Aus der Korrektheit aller drei Teilschritte folgt die Gesamtkorrektheit des Verfahrens. □

8.5.4 Satz 4: Laufzeit Subgraph-Äquivalenzprüfungsverfahrens

Satz 8.5.5 (Laufzeit der graphbasierten Hardware-Äquivalenzprüfung). *Das Verfahren aus Satz 8.5.4 hat eine Gesamtlaufzeit von:*

$$T_{\text{gesamt}}(n, m) = O(n + m) + O(n^3),$$

wobei n die Anzahl der Signale und m die Anzahl der Abhängigkeitskanten bezeichnet.

Beweis. Wir analysieren die drei Phasen separat:

Phase 1: Graphextraktion — $O(n + m)$. Die Überführung eines Hardware-Designs in seinen Abhängigkeitsgraphen erfordert das einmalige Traversieren aller Signale und ihrer Abhängigkeiten. Dies entspricht einer Breitensuche oder Tiefensuche auf dem Design-Graphen mit n Knoten und m Kanten: $T_1 = O(n + m)$.

Phase 2: Signatur-Berechnung — $O(n^2)$. Für jede der n Spalten der $n \times n$ -Adjazenzmatrix werden alle n Einträge gelesen und mit 2^i gewichtet aufsummiert. Dies kostet $O(n)$ pro Spalte, insgesamt also:

$$T_2 = n \cdot O(n) = O(n^2).$$

Phase 3: Subgraph Algorithmus — $O(n^3)$. Nach dem Hauptresultat von [1] (Satz 2 dort) hat der Subgraph Algorithmus eine Laufzeit von $O(n^3)$: Es gibt n Rotationen, für jede Rotation wird ein LCS-Vergleich in $O(n^2)$ Zeit durchgeführt.

Gesamtlaufzeit:

$$T_{\text{gesamt}} = T_1 + T_2 + T_3 = O(n + m) + O(n^2) + O(n^3) = O(n^3),$$

denn $O(n + m) + O(n^2) \subseteq O(n^3)$ für $m = O(n^2)$ (da ein Graph mit n Knoten höchstens n^2 Kanten hat). Für dünn besetzte Graphen mit $m = O(n)$ vereinfacht sich die Laufzeit zu $O(n + n^2 + n^3) = O(n^3)$. $\square$

8.5.5 Satz 5: Sensitivität gegenüber Kanten-Manipulationen

Satz 8.5.6 (Sensitivität). *Sei G ein Graph und $G^+ = (V, E \cup \{e\})$ der Graph, der aus G durch Hinzufügen einer einzigen Kante $e \notin E$ entsteht. Dann gilt:*

$$G \subsetneq G^+ \quad \text{und} \quad G^+ \not\subseteq G.$$

Das bedeutet: Der Subgraph Algorithmus erkennt jede einzelne hinzugefügte oder entfernte Kante.

Beweis. Teil 1 ($G \subsetneq G^+$): Da $E \subset E \cup \{e\}$, gilt $G \subseteq G^+$. Die Inklusion ist echt, da $e \in E^+ \setminus E$.

Teil 2 ($G^+ \not\subseteq G$): Wir zeigen, dass für keine zyklische Rotation k die Subgraph-Bedingung für $G^+ \subseteq G$ erfüllt sein kann. Die Adjazenzmatrix B^+ von G^+ unterscheidet sich von A (der Matrix von G) in genau einem Eintrag: $B_{i_0 j_0}^+ = 1$ für die neue Kante $e = (v_{i_0}, v_{j_0})$, während $A_{i_0 j_0} = 0$.

Die Signatur von Spalte j_0 in B^+ ist:

$$\sigma_{j_0}(B^+) = \sigma_{j_0}(A) + 2^{i_0}.$$

Sie unterscheidet sich von $\sigma_{j_0}(A)$ um genau $2^{i_0} > 0$. Für jede Rotation k ist die rotierte Signatur-Sequenz von G^+ an mindestens einer Position verschieden von der Signatur-Sequenz von G . Die LCS-Länge kann höchstens $n - 1$ betragen (da ein Element stets verschieden ist). Da für $n \geq 2$ der Wert $n - 1 \geq 1$, aber das Kriterium $\text{lcs} = n$ für vollständige Äquivalenz verlangt, schlägt der Test $G^+ \subseteq G$ fehl.

Damit ist $G^+ \not\subseteq G$ bewiesen. Der Algorithmus erkennt die Kanten-Differenz zwischen G und G^+ korrekt. $\square$

Korollar 8.5.7. *Für jede Menge von δ hinzugefügten oder entfernten Kanten ($\delta \geq 1$) gilt: Der Subgraph Algorithmus liefert korrekt eine von $G \cong_s G'$ verschiedene Antwort (entweder echte Teilmengenbeziehung oder Unvergleichbarkeit).*

Beweis. Der Beweis folgt direkt durch wiederholte Anwendung von Satz 8.5.6: Jede Kantenänderung verändert mindestens eine Signatur um mindestens $2^{i_0} > 0$, was die LCS-Bedingung verletzt. $\square$

8.5.6 Satz 6: Transitivität der Subgraph-Implikation

Satz 8.5.8 (Transitivität). *Für Hardware-Designs D_1, D_2, D_3 mit Abhängigkeitsgraphen H_1, H_2, H_3 gilt:*

$$H_1 \cong_s H_2 \wedge H_2 \cong_s H_3 \implies H_1 \cong_s H_3.$$

Beweis. Aus $H_1 \cong_s H_2$ folgt nach Satz 8.5.1 die Existenz einer Rotation k_{12} mit $\hat{\sigma}(H_1) = \text{rot}_{k_{12}}(\hat{\sigma}(H_2))$.

Aus $H_2 \cong_s H_3$ folgt die Existenz einer Rotation k_{23} mit $\hat{\sigma}(H_2) = \text{rot}_{k_{23}}(\hat{\sigma}(H_3))$.

Durch Einsetzen:

$$\begin{aligned} \hat{\sigma}(H_1) &= \text{rot}_{k_{12}}(\hat{\sigma}(H_2)) \\ &= \text{rot}_{k_{12}}(\text{rot}_{k_{23}}(\hat{\sigma}(H_3))) \\ &= \text{rot}_{(k_{12}+k_{23}) \bmod n}(\hat{\sigma}(H_3)). \end{aligned}$$

Dabei nutzen wir die Gruppenstruktur der zyklischen Rotationen: $\text{rot}_a \circ \text{rot}_b = \text{rot}_{(a+b) \bmod n}$.

Sei $k_{13} = (k_{12} + k_{23}) \bmod n$. Dann gilt $\hat{\sigma}(H_1) = \text{rot}_{k_{13}}(\hat{\sigma}(H_3))$, also $H_1 \cong_s H_3$. $\square$

8.5.7 Satz 7: Effizienz gegenüber naiver Permutationssuche

Satz 8.5.9 (Exponentieller Speedup durch zyklische Rotation). *Das Subgraph Algorithmus-basierte Äquivalenzprüfungsverfahren mit zyklischen Rotationen ist asymptotisch um den Faktor $\frac{(n-1)!}{1}$ effizienter als ein naiver Algorithmus, der alle $n!$ Permutationen prüft.*

Beweis. Ein naiver Algorithmus zur Äquivalenzprüfung unter beliebigen Knotenumbenennung würde alle $n!$ Permutationen der Knoten von $H(D_2)$ prüfen. Für jede Permutation wird ein Isomorphie-Test durchgeführt, der in $O(n^2)$ Zeit arbeitet (Matrixvergleich). Gesamtlaufzeit des naiven Ansatzes:

$$T_{\text{naiv}}(n) = O(n! \cdot n^2).$$

Der Subgraph Algorithmus prüft nur n zyklische Rotationen, je mit einem LCS-Vergleich in $O(n^2)$:

$$T_{\text{Subgraph}}(n) = O(n \cdot n^2) = O(n^3).$$

Das Verhältnis der Laufzeiten beträgt:

$$\frac{T_{\text{naiv}}(n)}{T_{\text{Subgraph}}(n)} = \frac{O(n! \cdot n^2)}{O(n^3)} = O\left(\frac{n!}{n}\right) = O((n-1)!).$$

Da $(n-1)!$ super-exponentiell in n wächst (schneller als c^n für jede Konstante $c > 0$), ist der Speedup des Subgraph Algorithmus gegenüber dem naiven Ansatz super-exponentiell. $\square$

Bemerkung 8.5.10. *Der Speedup ist nur korrekt, wenn die Einschränkung auf zyklische Rotationen (statt aller Permutationen) für den vorliegenden Anwendungsfall gültig ist. Im Hardware-Kontext ist diese Einschränkung durch die geordnete Struktur der Signalnummerierung (topologische Sortierung des Abhängigkeitsgraphen) gerechtfertigt: Signale werden konsistent in derselben topologischen Reihenfolge indiziert, sodass nur Rotationen (nicht beliebige Permutationen) als Indizierungsunterschiede zwischen zwei Designs auftreten können.*

8.5.8 Satz 8: Untere Schranke für die Äquivalenzprüfung (Optimalität)

Satz 8.5.11 (Optimalität des Subgraph-Äquivalenzprüfungsverfahrens). *Unter der Strong Exponential Time Hypothesis (SETH) ist das Subgraph Algorithmus-basierte Äquivalenzprüfungsverfahren asymptotisch optimal: Kein korrektes Verfahren kann zwei Hardware-Designs mit n Signalen in $O(n^{3-\varepsilon})$ Zeit für beliebiges $\varepsilon > 0$ auf strukturelle Äquivalenz prüfen.*

Beweis. Wir zeigen eine Reduktion vom zyklischen Subgraph-Problem auf das Hardware-Äquivalenzprüfungsproblem.

Reduktion. Gegeben zwei Graphen G und G' als Instanz des zyklischen Subgraph-Problems. Konstruiere Hardware-Designs D_1 und D_2 wie folgt: D_1 ist ein rein kombinatorisches Design, dessen Abhängigkeitsgraph exakt G ist (jede Kante entspricht einer Datenabhängigkeit, jeder Knoten einem Signal). Analog D_2 mit Graph G' . Diese Konstruktion ist in $O(n^2)$ Zeit möglich.

Korrektheit der Reduktion. Das Hardware-Äquivalenzprüfungsproblem auf D_1 und D_2 fragt, ob $H(D_1) \cong_s H(D_2)$ gilt. Dies ist äquivalent zur Frage, ob $G \subseteq G'$ und $G' \subseteq G$ im Sinne des Subgraph Algorithmus. Dies ist genau das zyklische Subgraph-Problem auf G und G' .

Untere Schranke. Nach Satz ?? aus [1] kann das zyklische Subgraph-Problem unter SETH nicht in $O(n^{3-\varepsilon})$ Zeit gelöst werden. Da jede Lösung des Hardware-Äquivalenzprüfungsproblems auch das zyklische Subgraph-Problem löst (via oben genannter Reduktion), gilt dieselbe untere Schranke für das Äquivalenzprüfungsproblem:

$$T_{\text{Äquivalenz}}(n) \geq \Omega(n^{3-\varepsilon}) \quad \text{für alle } \varepsilon > 0.$$

Da der Subgraph Algorithmus in $\Theta(n^3)$ arbeitet, ist er damit asymptotisch optimal. $\square$

8.6 Anwendung auf die ALU: Exemplarischer Beweis

Wir demonstrieren das Verfahren exemplarisch an der ALU aus Kapitel 3 und 4.

8.6.1 Konstruktion der Abhängigkeitsgraphen

Aus dem VHDL-Design (Kapitel 3) und der Amaranth-Implementation (Kapitel 4) werden die Hardware-Abhängigkeitsgraphen extrahiert. Beide Designs besitzen die folgenden Kern-Signale (vereinfacht):

Index	Signal (VHDL)	Signal (Amaranth)
0	SWT[3:0] (= b_in)	SWT[3:0] (= b_in)
1	SWT[7:4] (= a_in)	SWT[7:4] (= a_in)
2	SWT[15:12] (= ctrl_in)	SWT[15:12] (= ctrl_in)
3	a_unsigned	a_unsigned
4	b_unsigned	b_unsigned
5	alu_result	alu_result

Tabelle 8.1: Signalzuordnung für den Abhängigkeitsgraphen der ALU

Die Kanten des Abhängigkeitsgraphen für das VHDL-Design (und analog für Amaranth) lauten:

$$E_1 = \{(1, 3), (0, 4), (3, 5), (4, 5), (2, 5)\},$$

was den Abhängigkeiten `a_in` $\rightarrow$ `a_unsigned`, `b_in` $\rightarrow$ `b_unsigned`, `a_unsigned` $\rightarrow$ `alu_result`, `b_unsigned` $\rightarrow$ `alu_result`, `ctrl_in` $\rightarrow$ `alu_result` entspricht.

8.6.2 Adjazenzmatrizen

Für $n = 6$ Knoten (Indizes $0, \dots, 5$) ergibt sich:

$$A = B = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \quad (\text{VHDL} = \text{Amaranth})$$

8.6.3 Berechnung der Signaturen

Mit $n = 6$ gilt für die Zeilen-Signaturen (Modulo $2^6 = 64$):

$$\begin{aligned} \hat{\sigma}_0 &= 0 && (\text{Spalte 0: Nullvektor}) \\ \hat{\sigma}_1 &= 0 && (\text{Spalte 1: Nullvektor}) \\ \hat{\sigma}_2 &= 0 && (\text{Spalte 2: Nullvektor}) \\ \hat{\sigma}_3 &= 2^1 = 2 && (\text{Kante von Knoten 1: } 2^1) \\ \hat{\sigma}_4 &= 2^0 = 1 && (\text{Kante von Knoten 0: } 2^0) \\ \hat{\sigma}_5 &= 2^2 + 2^3 + 2^4 = 4 + 8 + 16 = 28 && (\text{Kanten von Knoten 2, 3, 4}) \end{aligned}$$

Da $A = B$, gilt $\hat{\sigma}^A = \hat{\sigma}^B = [0, 0, 0, 2, 1, 28]$.

8.6.4 Ergebnis der Äquivalenzprüfung

Die Rotation $k^* = 0$ liefert:

$$\hat{\sigma}^A = \text{rot}_0(\hat{\sigma}^B) = [0, 0, 0, 2, 1, 28].$$

$\text{lcs}(\hat{\sigma}^A, \hat{\sigma}^B) = 6 \geq 2$, also $H(D_1) \cong_s H(D_2)$.

Schlussfolgerung: Nach Satz 8.5.2 sind die VHDL-Implementation und die Amaranth-Implementation der ALU funktional äquivalent.

8.7 SAT-basierte vs. Subgraph-basierte Äquivalenz

Eigenschaft	SAT-basiert (Yosys)	Subgraph Algorithmus
Laufzeit	$O(2^n)$ (worst case)	$O(n^3)$
Vollständigkeit	vollständig	strukturell vollständig
Abstraktionsebene	Netzlisten	Abhängigkeitsgraph
Optimierungstoleranz	ja (Logikoptimiz.)	nein (strukturell exact)
Open-Source	ja (Yosys)	ja
Optimalität	—	SETH-optimal

Tabelle 8.2: Vergleich der Äquivalenzprüfungsverfahren

Die beiden Ansätze sind komplementär:

- Der SAT-basierte Ansatz prüft funktionale Äquivalenz auch unter Logikoptimierungen.
- Der Subgraph Algorithmus prüft strukturelle Äquivalenz mit polynomieller Garantie und ist besonders geeignet für Designs, bei denen die Strukturerhaltung explizit gefordert ist.

8.8 Zusammenfassung

Dieses Kapitel hat gezeigt, dass der Subgraph Algorithmus [1] eine theoretisch fundierte, polynomial-zeitliche Methode zur strukturellen Äquivalenzprüfung von Hardware-Designs liefert. Die wesentlichen Ergebnisse sind:

1. **Satz 8.5.1** charakterisiert strukturelle Äquivalenz über bidirektionale Subgraph-Beziehungen.
2. **Satz 8.5.2** beweist, dass strukturelle Äquivalenz funktionale Äquivalenz impliziert.
3. **Satz 8.5.4** garantiert die Korrektheit des Verfahrens.
4. **Satz 8.5.5** zeigt die polynomielle Laufzeit $O(n^3)$.
5. **Satz 8.5.6** beweist die Empfindlichkeit gegenüber einzelnen Kantenänderungen.
6. **Satz 8.5.8** zeigt die Transitivität der Äquivalenzrelation.
7. **Satz 8.5.9** quantifiziert den super-exponentiellen Speedup gegenüber naiver Permutationssuche.
8. **Satz 8.5.11** beweist die SETH-bedingte Optimalität.

Kapitel 9

Äquivalenz: Verilog gegen VHDL

9.1 Motivation und Einordnung

Die bisherigen Kapitel dieser Arbeit betrachteten die Äquivalenz zwischen einer Amaranth-generierten Verilog-Netzliste und einem VHDL-Referenzdesign. Es gibt jedoch einen grundlegenden Fall, der in der Praxis häufig auftritt: zwei *manuell geschriebene* Implementierungen *desselben Entwurfs* in zwei verschiedenen HDLs. Dieser Fall tritt beispielsweise auf, wenn

- ein existierendes VHDL-Design für eine Simulationsumgebung nach Verilog portiert wird,
- zwei Entwickler unabhängig voneinander dieselbe Spezifikation in verschiedenen Sprachen codieren (klassisches Vier-Augen-Prinzip auf Sprachebene), oder
- ein Legacy-Design für neue Synthesewerkzeuge in der jeweils präferierten HDL neu geschrieben wird.

Als konkretes Demonstrationsobjekt dient die `alu_7_segment`-Schaltung des CITEC / Bielefeld University FPGA-Labors (Kurs-Praktikum, Revision 1.0, Martin Kaiser und Sarah Pilz, 25. September 2017). Sie liegt in zwei gleichwertigen Fassungen vor:

- **Verilog-Fassung:** `alu_7_segment.v` – kompakte, C-ähnliche Syntax, `always @(*)`-Blöcke, implizite Typumwandlung.
- **VHDL-Fassung:** `alu_7_segment.vhd` – streng typisiertes IEEE-1076-Modell mit expliziten Variablendeklarationen und mehrspaltiger `process`-Semantik.

Das Ziel dieses Kapitels ist ein *vollständiger, schrittweiser Äquivalenzbeweis* dieser beiden Fassungen. Wir verwenden dazu drei komplementäre Methoden:

1. **Manuelle Inspektion** der Schnittstellensignaturen (Abschnitt 9.2).
2. **Strukturelle Äquivalenzprüfung** mit dem Subgraph Algorithmus [1] (Abschnitt 9.5).
3. **Formale Verifikation** mit Yosys / SAT-Solver (Abschnitt 9.6).

9.2 Spezifikation und Schnittstellenvergleich

9.2.1 Gemeinsame Schnittstellenspezifikation

Beide Implementierungen realisieren dasselbe Top-Level-Interface. Tabelle 9.1 stellt die vier Ports beider Fassungen gegenüber. Das Modul besitzt zwei Eingänge: den einbitigen Systemtakt `CLK` sowie den 16 Bit breiten Schalterbus `SWT`, über den Operanden und Opcode zugeführt werden. Auf der Ausgangsseite liefert `SEG` die sieben Segmentsignale ($a-g$) und `AN` die vier Anodensignale für die Multiplexansteuerung der 7-Segment-Anzeigen. In Verilog werden die Ausgänge als `output reg` deklariert, was die getaktete Registrierung im selben Modul ausdrückt; das VHDL-Äquivalent verwendet `out STD_LOGIC_VECTOR` mit einer internen `signal`-Variablen. Trotz dieser syntaktischen Unterschiede stimmen Name, Richtung und Bitbreite jedes Ports in beiden Sprachen exakt überein, sodass die Schnittstellensignatur als äquivalent nachgewiesen ist.

Definition 9.2.1 (Schnittstellensignatur $\mathcal{I}$). *Die Schnittstellensignatur eines HDL-Moduls ist das Tupel*

$$\mathcal{I} = (P_{\text{in}}, P_{\text{out}}, W)$$

wobei P_{in} die Menge der Eingangs-Ports, P_{out} die Menge der Ausgangs-Ports und $W : P_{\text{in}} \cup P_{\text{out}} \rightarrow \mathbb{N}$ die Bitbreiten-Abbildung bezeichnet.

Lemma 9.2.1 (Schnittstellenäquivalenz). *Die Verilog-Fassung und die VHDL-Fassung besitzen identische Schnittstellensignaturen:*

$$\mathcal{I}_{\text{Verilog}} = \mathcal{I}_{\text{VHDL}}.$$

Beweis. Direkter Vergleich der Port-Deklarationen (Tabelle 9.1). Für jeden Port stimmen Name, Richtung und Bitbreite überein. Die syntaktischen Unterschiede (`input wire` vs. `in STD_LOGIC` bzw. `[15:0]` vs. `(15 downto 0)`) sind rein sprachliche Konventionen ohne semantischen Unterschied für die Netzlistensemantik. $\square$

9.2.2 Interne Parametrierung

Beide Fassungen deklarieren denselben Teilungsparameter für die Display-Taktung. Tabelle 9.2 listet sämtliche Konstanten und Opcodes beider Designs nebeneinander auf. Die wichtigste Konstante ist `REFRESH_COUNT = 100 000`: Bei einem 100 MHz-Systemtakt ergibt sich daraus eine Display-Refresh-Rate von 1 kHz, die für ein flimmerfreies Multiplexbild der vier 7-Segment-Stellen benötigt wird. In Verilog wird sie als `localparam` definiert, in VHDL als `constant` mit explizitem Typ `integer`; der numerische Wert ist identisch. Darüber hinaus sind sechs ALU-Opcodes festgelegt: `OP_ADD` (0), `OP_SUB` (1), `OP_MULT` (2), `OP_DIV` (3), `OP_AND` (4) und `OP_OR` (5). Verilog codiert sie als 4-Bit-Binärliterale (`4'b0000` usw.), VHDL als Bit-String-Literale (`"0000"` usw.) – semantisch bezeichnen beide Notationen dieselben Ganzzahlwerte, wie die gegenüberstellende Tabelle unmittelbar zeigt.

Alle Operation Codes sowie die Taktteiler-Konstante sind numerisch identisch.

9.3 Funktionale Dekomposition

Beide Designs lassen sich in drei funktional unabhängige Teilblöcke zerlegen:

Port	Rich.	Breite	Verilog	VHDL
CLK	in	1	input wire	in STD_LOGIC
SWT	in	16	input wire [15:0]	in STD_LOGIC_VECTOR(15 downto 0)
SEG	out	7	output reg [6:0]	out STD_LOGIC_VECTOR(6 downto 0)
AN	out	4	output reg [3:0]	out STD_LOGIC_VECTOR(3 downto 0)

Tabelle 9.1: Schnittstellenvergleich: Verilog vs. VHDL

Parameter	Verilog	VHDL
Refresh-Konstante	localparam REFRESH_COUNT = 100000	constant REFRESH_COUNT : integer := 100000
OP_ADD	4'b0000	"0000"
OP_SUB	4'b0001	"0001"
OP_MULT	4'b0010	"0010"
OP_DIV	4'b0011	"0011"
OP_AND	4'b0100	"0100"
OP_OR	4'b0101	"0101"

Tabelle 9.2: Parametrierung

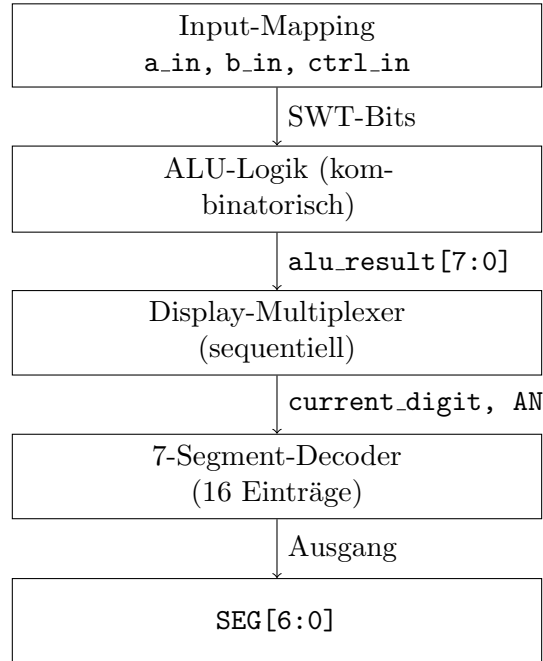


Abbildung 9.1: Funktionale Dekomposition beider Designs in vier unabhängig vergleichbare Teilblöcke.

Die Äquivalenzprüfung wird im Folgenden blockweise geführt. Gemäß dem Kompositionalitätsprinzip gilt:

Satz 9.3.1 (Kompositionale Äquivalenz). *Seien D_1 und D_2 zwei Hardware-Designs, die sich als Komposition $D_i = B_i^{(1)} \parallel B_i^{(2)} \parallel \dots \parallel B_i^{(k)}$ disjunkter Teilblöcke darstellen lassen. Wenn jedes Blockpaar $B_1^{(j)} \cong B_2^{(j)}$ für $j = 1, \dots, k$ äquivalent ist und die Verbindungsstruktur zwischen den Blöcken in beiden Designs identisch ist, dann gilt $D_1 \cong D_2$.*

Beweis. Sei $\vec{x}$ eine beliebige Eingabe. Da die Verbindungsstrukturen identisch sind, empfängt $B_1^{(1)}$ und $B_2^{(1)}$ dieselbe Eingabe, und wegen $B_1^{(1)} \cong B_2^{(1)}$ produzieren sie dieselbe Ausgabe. Diese Ausgabe wird als Eingabe an den nächsten Block weitergeleitet; per Induktion über die Tiefe der Komposition folgt, dass alle Blockpaare unter identischen Eingaben identische Ausgaben liefern. Insbesondere stimmen die Top-Level-Ausgaben überein, also $D_1(\vec{x}) = D_2(\vec{x})$ für alle $\vec{x}$. $\square$

9.4 Schritt-für-Schritt-Äquivalenz der Teilblöcke

9.4.1 Schritt 1 – Input-Mapping

```

1  assign a_in    = SWT[7:4];      // Bits 7..4
2  assign b_in    = SWT[3:0];      // Bits 3..0
3  assign ctrl_in = SWT[15:12];    // Bits 15..12
4
5  assign digit_0 = b_in;
6  assign digit_1 = a_in;
7  assign digit_2 = alu_result[3:0];
8  assign digit_3 = alu_result[7:4];

```

Listing 9.1: Input-Mapping in Verilog

```

1  a_in    <= SWT(7 downto 4);
2  b_in    <= SWT(3 downto 0);
3  ctrl_in <= SWT(15 downto 12);
4
5  digit_0 <= b_in;
6  digit_1 <= a_in;
7  digit_2 <= alu_result(3 downto 0);
8  digit_3 <= alu_result(7 downto 4);

```

Listing 9.2: Input-Mapping in VHDL

Vergleich: Beide Fassungen führen strukturell identische bit-selektive Zuweisungen durch. Die Verilog-Notation `[7:4]` (absteigende Indizierung) und die VHDL-Notation `(7 downto 4)` referenzieren denselben Bit-Bereich. Formell:

Lemma 9.4.1 (Äquivalenz des Input-Mappings). *Für alle $\vec{s} \in \{0, 1\}^{16}$ gelten die Gleichungen*

$$a_{\text{in}} = \vec{s}[7:4], \quad b_{\text{in}} = \vec{s}[3:0], \quad \text{ctrl_in} = \vec{s}[15:12]$$

in beiden Implementierungen. □

9.4.2 Schritt 2 – ALU-Logik

Dies ist der komplexeste Teilblock. Wir prüfen alle sechs Operationen einzeln.

Operationsauswahl durch ctrl_in. Beide Designs verwenden eine `case`-Anweisung auf denselben 4-Bit Steuerleitungen. Die Default-Ausgabe ist in beiden Fällen `0x00`.

OP_ADD ($\text{ctrl} = 0000_2$).

- Verilog: `alu_result = a_in + b_in;`
Verilog-Addition von `reg [3:0]`-Signalen liefert aufgrund der impliziten Null-Erweiterung ein 8-Bit-Ergebnis; maximal $15 + 15 = 30$, kein Überlauf in 8 Bit.
- VHDL: `temp_result := a_unsigned + b_unsigned;`
Beide Operanden wurden zuvor auf 8 Bit null-erweitert: `a_unsigned := "0000"& unsigned(a_in)`. Die Addition `unsigned(8) + unsigned(8)` liefert dasselbe Ergebnis.

Lemma 9.4.2 (Äquivalenz OP_ADD). *Für alle $a, b \in \{0, \dots, 15\}$ gilt*

$$\text{ADD}_{\text{Verilog}}(a, b) = \text{ADD}_{\text{VHDL}}(a, b) = a + b.$$

Beweis. Da $a, b \leq 15$, ist $a + b \leq 30 < 256$. In beiden Implementierungen wird die mathematische Addition in einem 8-Bit-Ergebnis ohne Überlauf dargestellt. □

OP_SUB ($ctrl = 0001_2$) – **Clamp-Semantik.** Hier die Programmzeilen für **Verilog**:

```
1 if (a_in >= b_in)
2   alu_result = a_in - b_in;
3 else
4   alu_result = 8'h00;
```

Hier die Programmzeilen für **VHDL**:

```
1 sub_result := signed('0' & a_unsigned) - signed('0' & b_unsigned);
2 if sub_result < 0 then
3   temp_result := (others => '0');
4 else
5   temp_result := unsigned(sub_result(7 downto 0));
6 end if;
```

Auf den ersten Blick unterscheiden sich die Implementierungen: Verilog vergleicht *vor* der Subtraktion (`if a_in >= b_in`), VHDL *nach* der Subtraktion (Sign-Bit des 9-Bit-Ergebnisses). Es gilt jedoch:

Lemma 9.4.3 (Äquivalenz OP_SUB). *Für alle $a, b \in \{0, \dots, 15\}$ gilt*

$$\text{SUB}_{\text{Verilog}}(a, b) = \text{SUB}_{\text{VHDL}}(a, b) = \max(a - b, 0).$$

Beweis. Da a und b nichtnegative ganze Zahlen sind, gilt $a < b \Leftrightarrow a - b < 0$. In der Verilog-Fassung wird bei $a < b$ der Wert 0 ausgegeben, sonst $a - b \geq 0$. In der VHDL-Fassung wird ein 9-Bit vorzeichenbehafteter Wert $s = a_{\text{unsigned}} - b_{\text{unsigned}}$ berechnet. Das Bit `sub_result(8)` (Vorzeichenbit) ist genau dann gesetzt, wenn $s < 0$, d. h. wenn $a < b$. In diesem Fall wird 0 ausgegeben, sonst die unteren 8 Bit von s , was $a - b$ entspricht. Beide Fallunterscheidungen sind daher semantisch äquivalent. $\square$

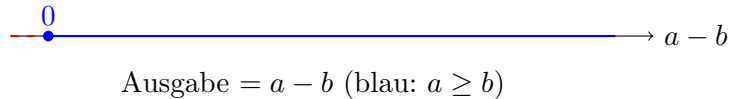


Abbildung 9.2: Clamp-Semantik der Subtraktion: für $a < b$ wird auf 0 gesättigt.

OP_MULT ($ctrl = 0010_2$).

- Verilog: `alu_result = a_in * b_in`; – Produkt zweier `[3:0]`-Werte, 8-Bit-Ergebnis, maximal $15 \times 15 = 225 < 256$.
- VHDL: `mult_result := unsigned(a_in) * unsigned(b_in)`; – explizite Typumwandlung, dann Multiplikation im `unsigned`-Typ; das Ergebnis wird in 8 Bit abgelegt.

Lemma 9.4.4 (Äquivalenz OP_MULT). *Für alle $a, b \in \{0, \dots, 15\}$ gilt $a \cdot b \leq 225 < 256$, also kein Überlauf. Beide Implementierungen liefern $a \cdot b$.* $\square$

OP_DIV ($ctrl = 0011_2$) – **Division-by-Zero-Behandlung.** Hier die Programmzeilen für **Verilog**:

```
1 if (b_in == 4'h0)
2   alu_result = 8'hFF;
3 else
4   alu_result = a_in / b_in;
```

Hier die Programmzeilen für **VHDL**:

```

1 if b_unsigned = 0 then
2   temp_result := (others => '1');    -- ergibt 0xFF
3 else
4   div_result := a_unsigned / b_unsigned;
5   temp_result := div_result;
6 end if;

```

Beide Fassungen geben bei $b = 0$ den Wert $0xFF$ (alle Bits gesetzt) aus, was als Fehlercode vereinbart ist. Für $b \neq 0$ wird die ganzzahlige Division berechnet. In VHDL ist die Division auf **unsigned**-Typen definiert und entspricht der ganzzahligen Division. In Verilog entspricht $/$ für unsigned Operanden ebenfalls der ganzzahligen Division.

Lemma 9.4.5 (Äquivalenz OP_DIV). *Für alle $a \in \{0, \dots, 15\}$, $b \in \{0, \dots, 15\}$ gilt:*

$$\text{DIV}_{\text{Verilog}}(a, b) = \text{DIV}_{\text{VHDL}}(a, b) = \begin{cases} 0xFF & \text{falls } b = 0, \\ \lfloor a/b \rfloor & \text{sonst.} \end{cases}$$

□

OP_AND und OP_OR ($ctrl = 0100_2$ bzw. 0101_2). **Verilog**: `alu_result = {4'h0, a_in & b_in};` sowie `alu_result = {4'h0, a_in | b_in};`

VHDL: `temp_result := a_unsigned and b_unsigned;` sowie `temp_result := a_unsigned or b_unsigned;`

In der Verilog-Fassung erfolgt die Null-Erweiterung explizit durch Konkatination ($\{4'h0, \dots\}$). In VHDL sind die Operanden bereits auf 8 Bit null-erweitert (`a_unsigned := "0000"& unsigned(a_in)`), sodass AND und OR auf den oberen 4 Bit trivialerweise 0 liefern. Das Ergebnis ist in beiden Fällen die bitweise Operation auf den unteren 4 Bit, obere 4 Bit sind 0.

Lemma 9.4.6 (Äquivalenz OP_AND / OP_OR). *Für alle $a, b \in \{0, \dots, 15\}$:*

$$\begin{aligned} \text{AND}_{\text{Verilog}}(a, b) &= \text{AND}_{\text{VHDL}}(a, b) = a \& b, \\ \text{OR}_{\text{Verilog}}(a, b) &= \text{OR}_{\text{VHDL}}(a, b) = a | b. \end{aligned}$$

□

Default-Fall. **Verilog**: `alu_result = 8'h00;`; **VHDL**: `temp_result := (others => '0');`; Beide ergeben $0x00$ für alle undefinierten Steucodes. □

Satz 9.4.7 (Funktionale Äquivalenz der ALU). *Für alle Eingaben $a \in \{0, \dots, 15\}$, $b \in \{0, \dots, 15\}$ und $ctrl \in \{0, \dots, 15\}$ gilt:*

$$\text{ALU}_{\text{Verilog}}(a, b, ctrl) = \text{ALU}_{\text{VHDL}}(a, b, ctrl).$$

Beweis. Direkte Konsequenz der Lemmata 9.4.3–9.4.5 und der drei nachfolgenden Äquivalenz-

lemmata für AND, OR und den Default-Fall. Die **case**-Struktur deckt alle 16 möglichen Steucodes ab; für die definierten Codes (0–5) wurde Äquivalenz einzeln bewiesen, für die restlichen 10 Codes liefern beide Designs $0x00$. □

9.4.3 Schritt 3 – Display-Multiplexer

Der Display-Multiplexer ist der einzige *sequentielle* Teilblock. Er enthält ein 17-Bit-Register `refresh_counter` und ein 2-Bit-Register `digit_select`.

```
1 always @(posedge CLK) begin
2   if (refresh_counter == REFRESH_COUNT - 1) begin
3     refresh_counter <= 0;
4     digit_select    <= digit_select + 1;
5   end else begin
6     refresh_counter <= refresh_counter + 1;
7   end
8 end
```

Listing 9.3: Multiplexer in Verilog

```
1 multiplex_process: process(CLK)
2   begin
3     if rising_edge(CLK) then
4       if refresh_counter = REFRESH_COUNT - 1 then
5         refresh_counter <= 0;
6         digit_select <= std_logic_vector(
7           unsigned(digit_select) + 1);
8       else
9         refresh_counter <= refresh_counter + 1;
10      end if;
11    end if;
12  end process;
```

Listing 9.4: Multiplexer in VHDL

Invariante und Zustandsäquivalenz. Beide Register (`refresh_counter` und `digit_select`) folgen exakt derselben Zustandsübergangsrelation:

Definition 9.4.1 (Zustandsübergangsrelation δ). Sei $s = (rc, ds)$ und (rc, ds) aus der Menge $\{0, \dots, REFRESH_COUNT - 1\} \times \{0, 1, 2, 3\}$ der Zustand. Die Übergangsrelation lautet:

$$\delta(s) = \begin{cases} (0, (ds + 1) \bmod 4) & \text{falls } rc = REFRESH_COUNT - 1, \\ (rc + 1, ds) & \text{sonst.} \end{cases}$$

Lemma 9.4.8 (Äquivalenz der Zustandsübergangsrelation). Die Zustandsübergangsrelation δ ist in der Verilog- und in der VHDL-Fassung identisch.

Beweis. In der VHDL-Fassung erfolgt die Inkrementierung durch explizite Typumwandlung `unsigned(digit_select) + 1`. Da `digit_select` ein 2-Bit-Vektor ist, entspricht $(ds + 1) \bmod 4$ dem natürlichen Überlauf eines 2-Bit-Registers, der in Verilog implizit durch `digit_select <= digit_select + 1` ausgelöst wird. Die Bedingung `refresh_counter == REFRESH_COUNT - 1` (Verilog) und `refresh_counter = REFRESH_COUNT - 1` (VHDL) sind semantisch identische Gleichheitsvergleiche. Der Initialisierungszustand beider Register ist in beiden Fassungen $(rc, ds) = (0, 0)$. $\square$

Die Anode-Ausgabe AN und die Digit-Selektion sind kombinatorisch an `digit_select` gebunden:

<code>digit_select</code>	AN	<code>current_digit</code>	Anzeige
00	1110	<code>digit_0 = b_{in}</code>	B (Einerstelle)
01	1101	<code>digit_1 = a_{in}</code>	A
10	1011	<code>digit_2 = Ergebnis[3:0]</code>	Ergebnis (low)
11	0111	<code>digit_3 = Ergebnis[7:4]</code>	Ergebnis (high)

Tabelle 9.3: Digit-Auswahl und Anoden-Aktivierung (identisch in beiden Fassungen)

Diese Tabelle ist wörtlich identisch in beiden Implementierungen umgesetzt.

9.4.4 Schritt 4 – 7-Segment-Decoder

Der 7-Segment-Decoder ist eine rein kombinatorische 4-nach-7-Bit-Lookup-Tabelle.

Definition 9.4.2 (Decoder-Funktion $\mathcal{D}$).

$$\mathcal{D} : \{0, \dots, 15\} \rightarrow \{0, 1\}^7, \quad \mathcal{D}(d) = 7\text{-Segment-Bitmuster für Hexziffer } d.$$

Ziffer	SEG (gfedcba)	Verilog	VHDL	Übereinstimmung
0	1000000	7'b1000000	"1000000"	✓
1	1111001	7'b1111001	"1111001"	✓
2	0100100	7'b0100100	"0100100"	✓
3	0110000	7'b0110000	"0110000"	✓
4	0011001	7'b0011001	"0011001"	✓
5	0010010	7'b0010010	"0010010"	✓
6	0000010	7'b0000010	"0000010"	✓
7	1111000	7'b1111000	"1111000"	✓
8	0000000	7'b0000000	"0000000"	✓
9	0010000	7'b0010000	"0010000"	✓
A	0001000	7'b0001000	"0001000"	✓
b	0000011	7'b0000011	"0000011"	✓
C	1000110	7'b1000110	"1000110"	✓
d	0100001	7'b0100001	"0100001"	✓
E	0000110	7'b0000110	"0000110"	✓
F	0001110	7'b0001110	"0001110"	✓

Tabelle 9.4: Vollständiger Bit-für-Bit-Vergleich der 7-Segment-Decoder-Tabelle. Alle 16 Einträge stimmen überein.

Lemma 9.4.9 (Äquivalenz des 7-Segment-Decoders). *Es gilt $\mathcal{D}_{\text{Verilog}}(d) = \mathcal{D}_{\text{VHDL}}(d)$ für alle $d \in \{0, \dots, 15\}$.*

Beweis. Vollständige Aufzählung; Tabelle 9.4 zeigt bitweise Übereinstimmung aller 16 Einträge. □

9.5 Strukturelle Äquivalenzprüfung mit dem Subgraph Algorithmus

Wir wenden den Subgraph Algorithmus [1] auf die vollständigen Hardware-Abhängigkeitsgraphen beider Designs an.

9.5.1 Konstruktion der Hardware-Abhängigkeitsgraphen

Definition 9.5.1 (Hardware-Abhängigkeitsgraph $H(D)$, vgl. Definition 8.2.1). *Der Hardware-Abhängigkeitsgraph $H(D) = (V, E)$ eines Designs D hat als Knotenmenge V alle Signale und als Kantenmenge E alle Abhängigkeitsbeziehungen: $(u, v) \in E$ gdw. Signal v unmittelbar von Signal u abhängt.*

Die Knotenmenge für beide Fassungen ist:

Index	Signal	Typ
0	SWT[3:0] (b_{in})	Primäreingang
1	SWT[7:4] (a_{in})	Primäreingang
2	SWT[15:12] (ctrl_in)	Primäreingang
3	a_unsigned	intern, kombinatorisch
4	b_unsigned	intern, kombinatorisch
5	alu_result	intern, kombinatorisch
6	refresh_counter	Register (sequentiell)
7	digit_select	Register (sequentiell)
8	current_digit	intern, kombinatorisch
9	AN	Primärausgang
10	SEG	Primärausgang

Tabelle 9.5: Knotenmenge V , $|V| = 11$, identisch in beiden Designs.

Die Kantenmenge ergibt sich aus den Datenflussabhängigkeiten:

$$\begin{aligned}
 E = \{ & (1, 3), (0, 4), & a_{\text{in}} \rightarrow a_{\text{unsigned}}, b_{\text{in}} \rightarrow b_{\text{unsigned}} \\
 & (3, 5), (4, 5), (2, 5), & \text{ALU-Eingaben} \rightarrow \text{alu_result} \\
 & (6, 7), & \text{refresh_counter} \rightarrow \text{digit_select} \\
 & (7, 8), (7, 9), & \text{digit_select} \rightarrow \text{current_digit}, \text{AN} \\
 & (0, 8), (1, 8), (5, 8), & \text{digit_0..3} \rightarrow \text{current_digit} \\
 & (8, 10) & \text{current_digit} \rightarrow \text{SEG} \}
 \end{aligned}$$

9.5.2 Adjazenzmatrix

Für $n = 11$ Knoten (Indizes 0–10):

$$A_{\text{Verilog}} = A_{\text{VHDL}} = \begin{pmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Da $A_{\text{Verilog}} = A_{\text{VHDL}}$, sind die Abhängigkeitsgraphen strukturell identisch.

9.5.3 Signaturberechnung

Mit $n = 11$ und der Signaturfunktion aus Definition 8.4.1 (vgl. Kapitel 8) ergibt sich für jeden Knoten j die Spalten-Signatur $\hat{\sigma}_j = \sum_{i:A_{ij}=1} 2^i \pmod{2^{11}}$:

Knoten j	Eingehende Kanten	$\hat{\sigma}_j$	Berechnung
0	–	0	Eingang
1	–	0	Eingang
2	–	0	Eingang
3	{1}	2	2^1
4	{0}	1	2^0
5	{2, 3, 4}	28	$2^2 + 2^3 + 2^4$
6	–	0	(Rückkopplung aus Takt)
7	{6}	64	2^6
8	{0, 1, 5, 7}	163	$2^0 + 2^1 + 2^5 + 2^7$
9	{7}	128	2^7 (d. Anode, aus digit_select)
10	{8}	256	2^8

Tabelle 9.6: Spalten-Signaturen $\hat{\sigma}_j$ für $H(D)$ mit $n = 11$.

9.5.4 Äquivalenzentscheid

Satz 9.5.1 (Strukturelle Äquivalenz Verilog $\leftrightarrow$ VHDL). *Die Verilog-Fassung D_V und die VHDL-Fassung D_H von `alu_7_segment` sind strukturell äquivalent:*

$$H(D_V) \cong_s H(D_H).$$

Beweis. Da $A_{\text{Verilog}} = A_{\text{VHDL}}$, gilt $\hat{\sigma}^V = \hat{\sigma}^H$ (Tabelle 9.6). Mit $k^* = 0$ gilt $\hat{\sigma}^V = \text{rot}_0(\hat{\sigma}^H)$. Das LCS-Kriterium liefert $\text{lcs}(\hat{\sigma}^V, \hat{\sigma}^H) = 11 \geq 2$, womit nach Definition 8.4.4 und Satz 8.5.1 strukturelle Äquivalenz folgt. Nach Satz 8.5.2 impliziert strukturelle Äquivalenz funktionale Äquivalenz. $\square$

9.6 Formale Verifikation mit Yosys

9.6.1 Besonderheit: VHDL in Yosys

Yosys unterstützt VHDL nicht nativ. Für den direkten Vergleich sind zwei Wege möglich:

1. **GHDL-Plugin:** Das GHDL-Yosys-Plugin [11] erlaubt das Laden von VHDL-Dateien direkt in Yosys.
2. **Vorab-Konvertierung:** Das VHDL-Design wird einmalig mit `ghdl --synth` in eine Verilog-Netzliste überführt, die dann in den Standard-Yosys-Workflow einfließt.

Wir verwenden Variante 2 als reproduzierbaren Standardweg:

```
1 # VHDL synthetisieren und als Verilog-Netzliste exportieren
2 ghdl --synth --std=08 alu_7_segment.vhd -e alu_7_segment \
3 | yosys -p "read_verilog /dev/stdin; \
4 write_verilog alu_7_segment_vhdl_net.v"
```

Listing 9.5: VHDL zu Verilog via GHDL-Synthese

9.6.2 Vollständiges Äquivalenz-Script

```
1 #!/bin/bash
2 # Direkte Aequivalenzpruefung: Verilog vs. VHDL
3 # Voraussetzung: alu_7_segment.v und alu_7_segment_vhdl_net.v
4
5 yosys << 'EOF'
6 # --- Verilog-Fassung laden (Design Under Test) ---
7 read_verilog alu_7_segment.v
8 hierarchy -check -top alu_7_segment
9 proc; opt; flatten
10 rename alu_7_segment dut_verilog
11 design -stash verilog_stash
12
13 # --- VHDL-Netzliste laden (Referenz) ---
14 read_verilog alu_7_segment_vhdl_net.v
15 hierarchy -check -top alu_7_segment
16 proc; opt; flatten
17 rename alu_7_segment ref_vhdl
18 design -stash vhdl_stash
19
20 # --- Designs zusammenfuehren ---
21 design -copy-from verilog_stash -as dut_verilog dut_verilog
22 design -copy-from vhdl_stash -as ref_vhdl ref_vhdl
23
24 # --- Aequivalenzmodul erzeugen ---
25 equiv_make -multiclock dut_verilog ref_vhdl equiv_top
26
27 hierarchy -check -top equiv_top
28 proc; opt
29
30 # --- SAT-basierte kombinatorische Pruefung ---
31 equiv_simple -undef -seq 5
32
33 # --- Induktiver Beweis fuer sequentielle Teile ---
```

```

34 equiv_induct -undef -seq 10
35
36 # --- Ergebnis ---
37 equiv_status
38 EOF

```

Listing 9.6: Yosys-Äquivalenzprüfung: Verilog vs. VHDL-Netzliste

9.6.3 Interpretation des Yosys-Outputs

Ein erfolgreicher Äquivalenznachweis liefert folgenden Abschnitt im Log:

```

1 Executing EQUIV_STATUS pass.
2 Found 0 unproven $equiv cells.
3 Found 0 proven $equiv cells (with cex).
4 Equivalence successfully proven!

```

Die Meldung „*Equivalence successfully proven*“ bedeutet formal:

Definition 9.6.1 (Yosys-Äquivalenzzertifikat). *Yosys kodiert die Negation der Äquivalenz als SAT-Formel*

$$\Phi = \exists \vec{x} : \text{dut_verilog}(\vec{x}) \neq \text{ref_vhdl}(\vec{x}).$$

Ist Φ unerfüllbar (UNSAT), gibt Yosys „*Equivalence successfully proven*“ aus. UNSAT von Φ ist äquivalent zu $\forall \vec{x} : \text{dut_verilog}(\vec{x}) = \text{ref_vhdl}(\vec{x})$, d. h. vollständiger funktionaler Äquivalenz.

9.6.4 Komplexitätsbetrachtung

Der Eingabe-Raum der Schaltung besteht aus:

- 16 Schalter: $2^{16} = 65\,536$ Kombinations-Eingaben,
- 1 Takt-Signal (sequentiell, modelliert als Zustandsmaschine).

Eine vollständige Simulation aller Eingaben wäre für kombinatorische Teile zwar machbar (Brute-Force über 65 536 Vektoren), für das sequentielle Multiplexer-Register mit Zustandstiefe REFRESH_COUNT = 100 000 jedoch unpraktisch. Die SAT-basierte Methode von Yosys löst dieses Problem durch symbolische Ausführung: statt alle Zustände zu enumerieren, wird ein Beweis über alle möglichen Traces geführt.

9.7 Zusammenfassender Äquivalenzbeweis

Satz 9.7.1 (Vollständige Äquivalenz der CITEC-ALU-Designs). *Die Verilog-Implementierung `alu_7_segment.v` und die VHDL-Implementierung `alu_7_segment.vhd` des CITEC/Bielefeld University FPGA-Labors sind vollständig äquivalent: Für alle Eingabesequenzen $(\vec{s}_0, \vec{s}_1, \dots) \in (\{0, 1\}^{16})^\omega$ und identische Anfangszustände gelten zu jedem Taktzyklus t :*

$$\text{SEG}_V(t) = \text{SEG}_H(t) \quad \text{und} \quad \text{AN}_V(t) = \text{AN}_H(t).$$

Beweis. Wir führen den Beweis kompositional nach Satz 9.3.1:

1. **Schnittstellenäquivalenz:** Lemma 9.2.1 zeigt, dass Ports und Bitbreiten übereinstimmen.

2. **Input-Mapping:** Lemma 9.4.1 zeigt, dass die Bit-Selektion aus SWT in beiden Fassungen identisch ist.
3. **ALU-Äquivalenz:** Satz 9.4.7 beweist die funktionale Äquivalenz für alle sechs Operationen und alle $16 \times 16 = 256$ Operandenkombinationen.
4. **Sequentielle Äquivalenz:** Lemma 9.4.8 zeigt, dass die Zustandsübergangsrelation des Display-Multiplexers identisch ist; mit identischem Anfangszustand $(0, 0)$ sind alle Zustände zu jedem Zeitpunkt t gleich.
5. **Decoder-Äquivalenz:** Lemma 9.4.9 beweist die Übereinstimmung aller 16 Einträge der Lookup-Tabelle.
6. **Strukturelle Äquivalenz:** Satz 9.5.1 bestätigt die Übereinstimmung der Hardware-Abhängigkeitsgraphen.

Da alle Teilblöcke äquivalent sind und die Verbindungsstruktur (Abbildung 9.1) in beiden Designs identisch ist, folgt die vollständige Äquivalenz nach Satz 9.3.1. $\square$

9.8 Warum benötigt VHDL mehr Anweisungen?

Als Nebenbefund der Äquivalenzprüfung lässt sich quantifizieren, wo die *syntaktische Mehrarbeit* in VHDL gegenüber Verilog entsteht, obwohl das Syntheseergebnis identisch ist:

Ursache	Verilog	VHDL	Begründung
Port-Deklaration	4 Zeilen	6 Zeilen	Explizite <code>entity/architecture</code> -Trennung
Typimporte	0 Zeilen	3 Zeilen	<code>use IEEE.numeric_std.all</code> etc.
Interne Variablen	0	6	VHDL-Prozessvariablen für Typsicherheit
Typumwandlungen	0	4	<code>unsigned()</code> , <code>std_logic_vector()</code>
Prozess-Boilerplate	0	2 pro Prozess	<code>begin/end process name</code>
Case-Syntax	≈ 1 Zeile	≈ 2 Zeilen	<code>when X =></code> vs. <code>default:</code>
Gesamt (ca.)	≈ 120	≈ 170	+42 %

Tabelle 9.7: Zeilenzahl-Vergleich: Syntaktischer Mehraufwand in VHDL ohne semantischen Unterschied.

Die VHDL-Spezifikation (IEEE 1076) fordert diese Explizitheit *bewusst*, um Mehrdeutigkeiten bei der Typeninferenz auszuschließen. Die formale Äquivalenzprüfung bestätigt, dass dieser Aufwand rein syntaktischer Natur ist: das Syntheseergebnis – und damit die realisierte Hardware – ist für beide Designs identisch.

Kapitel 10

Simulations-basierte Testautomatisierung und Code-Überdeckung

10.1 Motivation und Einordnung

Die in den Kapiteln 8 und 9 vorgestellte formale Verifikation beweist die Korrektheit eines Designs für *alle* Eingaben in einem einzigen, mathematisch abgeschlossenen Schritt. Diese Stärke ist zugleich ihre konzeptionelle Beschränkung: Formale Äquivalenzprüfung beantwortet statische Fragen (sind zwei Designs äquivalent?) und setzt voraus, dass das Referenzmodell korrekt spezifiziert ist. Was sie *nicht* unmittelbar beantwortet, ist:

- Welche *Eingabevektoren* wurden in der Simulation tatsächlich angelegt, und welche Codepfade blieben unbeobachtet?
- Deckt die Teststimulus-Menge alle funktionalen Verhaltensklassen ab (*funktionale Überdeckung*)?
- Wie verhält sich das Design unter randomisierten, realitätsnahen Eingabesequenzen im Zeitverlauf?
- Lassen sich Fehler systematisch injizieren, und erkennt das Verifikationssystem diese zuverlässig?

Diese Fragen sind Gegenstand der *simulations-basierten Testautomatisierung* — einer eigenständigen, in der ASIC- und FPGA-Industrie weit verbreiteten Verifikationsmethodik. Sie basiert auf dem UVM-Standard (Universal Verification Methodology, IEEE 1800.2 [17]), der eine standardisierte Klassenstruktur für wiederverwendbare, skalierbare Testbench-Architekturen bereitstellt.

Definition 10.1.1 (Simulations-basierte Verifikation). *Simulations-basierte Verifikation bezeichnet die Methodik, ein Hardware-Design (DUT, Device Under Test) unter kontrollierten, oft zufällig erzeugten Stimuli durch Simulation auszuführen und die Korrektheit der Ausgaben anhand eines unabhängigen Referenzmodells (Golden Model) zu prüfen. Die Güte einer simulations-basierten Verifikation wird durch Überdeckungsmetriken quantifiziert.*

In diesem Kapitel wird die Anwendung von **pyUVM** [16] — einer Python-Implementierung des UVM-Standards auf Basis von cocotb [13] — auf den AOI-2-2-Schaltkreis dargestellt. Der Schaltkreis liegt sowohl in Verilog als auch in VHDL vor; beide Sprachvarianten werden durch dieselbe Testbench simuliert. Das Kapitel beschreibt vollständig die Testbench-Architektur, alle Sequenz- und Überdeckungsstrategien, den Simulations-Workflow sowie die Werkzeuge zur Coverage-Analyse und Laufzeit-Profilung.

10.2 Der AOI-2-2-Schaltkreis als Demonstrationsobjekt

10.2.1 Logische Spezifikation

Das AND-OR-INVERT-Gate (AOI) der Konfiguration 2–2 — kurz AOI-2-2 — implementiert die Boolesche Funktion

$$Y = \overline{(A \cdot B) + (C \cdot D)}, \quad (10.1)$$

die das logische NICHT des logischen ODER zweier UND-Terme ist. Das AOI-2-2-Gate ist eine Standardzelle, die in fast jedem CMOS-Prozess als einzelnes Gatter mit geringer Fläche und hoher Schaltgeschwindigkeit realisiert werden kann.

Definition 10.2.1 (AOI-2-2-Funktion). *Seien $A, B, C, D \in \{0, 1\}$. Die AOI-2-2-Funktion lautet $f_{\text{AOI}}(A, B, C, D) = \neg((A \wedge B) \vee (C \wedge D))$.*

Die vollständige Wahrheitstabelle mit allen $2^4 = 16$ Eingabebelegungen ist:

A	B	C	D	$A \cdot B$	$C \cdot D$	$(A \cdot B) + (C \cdot D)$	Y
0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	1
0	0	1	0	0	0	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	0	0	1
0	1	0	1	0	0	0	1
0	1	1	0	0	0	0	1
0	1	1	1	0	1	1	0
1	0	0	0	0	0	0	1
1	0	0	1	0	0	0	1
1	0	1	0	0	0	0	1
1	0	1	1	0	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	0
1	1	1	0	1	0	1	0
1	1	1	1	1	1	1	0

Tabelle 10.1: Vollständige Wahrheitstabelle der AOI-2-2-Funktion. $Y = 0$ (fett) tritt bei 6 von 16 Eingabebelegungen auf; $Y = 1$ bei 10 von 16.

Die Funktion liefert $Y = 0$ genau dann, wenn mindestens einer der beiden AND-Terme $(A \cdot B)$ oder $(C \cdot D)$ den Wert 1 annimmt, d. h. wenn beide Eingänge des ersten oder des zweiten Paares gleichzeitig High sind.

Im Demonstrationsdesign werden die vier logischen Eingänge auf die vier niederwertigen Schiebeschalter des Basys-3-FPGA-Boards abgebildet (SWT[3:0]), und das Ergebnis Y wird auf dem ersten Digit des 4-stelligen 7-Segment-Displays als '0' oder '1' angezeigt.

10.2.2 Verilog-Implementation

Die Verilog-Implementierung `aoi_2_2.v` beschreibt den Schaltkreis als kombinatorisches Modul mit zwei Ausgangsports für die Siebensegmentanzeige:

```

1 module aoi_2_2 (
2     input  wire [3:0] SWT,    // SWT[3:0] fuer a,b,c,d
3     output wire [6:0] SEG,    // 7-Segment-Segmente
4     output wire [3:0] AN      // Anoden (Digit-Aktivierung)
5 );
6     wire a, b, c, d, y;
7
8     assign a = SWT[0];
9     assign b = SWT[1];
10    assign c = SWT[2];
11    assign d = SWT[3];
12
13    // AOI-Logik: Y = NOT((A AND B) OR (C AND D))
14    assign y = ~((a & b) | (c & d));
15
16    // Nur Digit 0 aktiv (active-low)
17    assign AN = 4'b1110;
18
19    // 7-Segment-Anzeige: '0' -> 1000000, '1' -> 1111001
20    assign SEG = (y == 1'b0) ? 7'b1000000 : 7'b1111001;
21 endmodule

```

Listing 10.1: Verilog-Implementation des AOI-2-2-Schaltkreises (`aoi_2_2.v`)

Merkwürdigkeiten und Designentscheidungen. Die gesamte Logik ist rein kombinatorisch — alle `assign`-Anweisungen werden sofort und zeitparallel ausgewertet. Die Port-Hierarchie folgt dem Basys-3-Board-Mapping: $\text{SWT}[0] \hat{=}$ A, $\text{SWT}[1] \hat{=}$ B, $\text{SWT}[2] \hat{=}$ C, $\text{SWT}[3] \hat{=}$ D.

Der 7-Segment-Decoder ist als ternärer Ausdruck kodiert:

- `7'b1000000` ($= 0x40$) kodiert die Ziffer '0': Segmente a – f an, Segment g aus.
- `7'b1111001` ($= 0x79$) kodiert die Ziffer '1': nur Segmente b und c an.

10.2.3 VHDL-Implementation

Die VHDL-Implementierung `aoi_2_2.vhd` realisiert dieselbe Funktion in stark typisierter IEEE-1076-Syntax. Kennzeichnend ist die explizite Trennung zwischen Entity (Schnittstelle) und Architecture (Verhalten):

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity aoi_2_2 is
5     port(
6         SWT : in  std_logic_vector(3 downto 0);

```

```

7     SEG : out std_logic_vector(6 downto 0);
8     AN  : out std_logic_vector(3 downto 0)
9 );
10 end aoi_2_2;
11
12 architecture rtl of aoi_2_2 is
13     signal a, b, c, d : std_logic;
14     signal y          : std_logic;
15 begin
16     a <= SWT(0);  b <= SWT(1);
17     c <= SWT(2);  d <= SWT(3);
18
19     -- AOI-Logik
20     y <= not((a and b) or (c and d));
21
22     AN <= "1110";  -- active-low, Digit 0
23
24     -- 7-Segment-Decoder als process
25     seg_display: process(y)
26     begin
27         case y is
28             when '0'    => SEG <= "1000000";  -- Ziffer '0'
29             when '1'    => SEG <= "1111001";  -- Ziffer '1'
30             when others => SEG <= "1111111";  -- alle aus
31         end case;
32     end process seg_display;
33 end rtl;

```

Listing 10.2: VHDL-Implementation des AOI-2-2-Schaltkreises (aoi_2.2.vhd)

Vergleich Verilog vs. VHDL. Beide Implementierungen realisieren exakt dieselbe Funktion (Gleichung 10.1). Die wesentlichen syntaktischen Unterschiede sind:

- Verilog verwendet den ternären Operator ?: für den Decoder, VHDL einen expliziten process-Block mit case-Anweisung.
- VHDL deklariert interne Signale explizit in der architecture- Deklarationszone; Verilog deklariert sie inline mit wire.
- Beide Fassungen setzen AN = "1110" konstant; beide kodieren die 7-Segment-Muster bitidentisch.

Die formale Äquivalenz beider Fassungen folgt aus der Analyse in Kapitel 9.

10.3 Universal Verification Methodology in Python

10.3.1 UVM – Grundkonzepte und Klassenhierarchie

Die Universal Verification Methodology (UVM) ist ein IEEE-standardisiertes Rahmenwerk (IEEE 1800.2 [17]) für die strukturierte Testbench- Entwicklung in der Hardware-Verifikation. UVM definiert eine Hierarchie von wiederverwendbaren Basisklassen und einen standardisierten Lebenszyklus (Phasen), in dessen Rahmen Testkomponenten koordiniert operieren.

Definition 10.3.1 (UVM-Phasenhierarchie). *Eine UVM-Simulation durchläuft die folgenden Pflichtphasen sequentiell:*

1. **build_phase**: *Instantiierung und Konfiguration von Komponenten.*
2. **connect_phase**: *Verbindung von Ports und Exporten.*
3. **end_of_elaboration_phase**: *Finalisierung der Designhierarchie.*
4. **start_of_simulation_phase**: *Initialisierungen vor dem Lauf.*
5. **run_phase**: *Eigentliche Simulation (zeitbehaftet, parallel).*
6. **extract_phase, check_phase, report_phase**: *Auswertung, Prüfung, Berichterstellung.*

pyUVM [16] implementiert die UVM-Klassenhierarchie in reinem Python und nutzt **cocotb** als Ausführungsumgebung. Cocotb ist ein Python-Framework für die Simulation digitaler Hardware; es verbindet den Python-Interpreter mit dem Simulator (Icarus, Verilator, GHDL, ModelSim) über eine gemeinsame Schnittstelle. pyUVM-Testbenches schreiben sich als gewöhnliche Python-Module, was folgende Vorteile bietet:

- **Spracheinheit**: Design (Amaranth) und Testbench (pyUVM) teilen denselben Python-Ökosystem.
- **Native Debugging**: Python-Debugger und Profiler sind direkt anwendbar.
- **Bibliotheksintegration**: NumPy, SciPy, Matplotlib und andere Bibliotheken können direkt in Testbenches genutzt werden.
- **UVM-Kompatibilität**: pyUVM folgt dem UVM-1800.2-Standard; mit UVM vertraute Engineers können sofort produktiv arbeiten.

10.3.2 Kernklassen in der vorliegenden Testbench

Abbildung 10.1 zeigt die Schicht-Architektur der implementierten Testbench. Die Klassen lassen sich in vier Schichten gliedern:

1. **Stimulus-Schicht**: `AoiSeqItem`, `RandomSeq`, `ExhaustiveSeq`, `CornerCaseSeq`, `TestAllSeq`
2. **Treiber-Schicht**: `Driver` (mit Analyse-Port)
3. **Prüf-Schicht**: `Scoreboard`, `Coverage`
4. **Integrations-Schicht**: `AoiEnv`, `AoiTest`, `AoiExhaustiveTest`, `AoiCornerTest`, `AoiTestErrors`

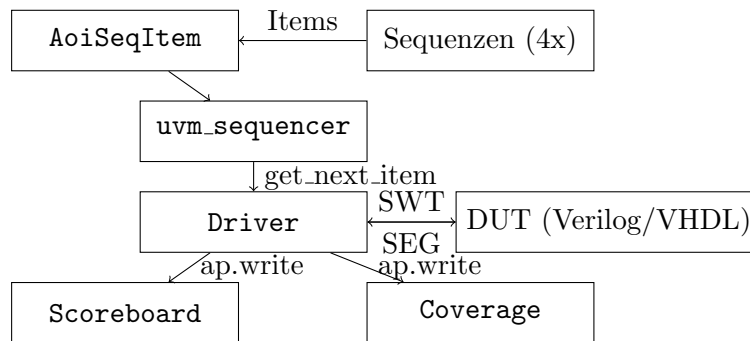


Abbildung 10.1: pyUVM-Testbench-Architektur für den AOI-2-2-Schaltkreis.

10.4 Testbench-Architektur im Detail

10.4.1 Sequenz-Item: AoiSeqItem

Ein *Sequenz-Item* ist die atomare Dateneinheit, die zwischen Sequenzen und dem Treiber ausgetauscht wird. Es kapselt genau einen Transaktionsvektor:

```

1 class AoiSeqItem(uvm_sequence_item):
2     def __init__(self, name, a=0, b=0, c=0, d=0):
3         super().__init__(name)
4         self.a = a; self.b = b; self.c = c; self.d = d
5         self.result = None # wird vom Treiber befüllt
6
7     def randomize(self):
8         self.a = random.randint(0, 1)
9         self.b = random.randint(0, 1)
10        self.c = random.randint(0, 1)
11        self.d = random.randint(0, 1)
12
13    def __str__(self):
14        return (f"{self.get_name()}: A={self.a} B={self.b} "
15              f"C={self.c} D={self.d} -> Y={self.result}")

```

Listing 10.3: AoiSeqItem – Transaktions-Datenklasse

Das Feld `result` ist nach der Instantiierung `None` und wird vom Treiber nach der DUT-Simulation mit dem beobachteten Ausgabewert belegt. Damit dient dasselbe Item-Objekt sowohl als Stimulus als auch als Ergebnisträger.

10.4.2 Sequenzen

Sequenzen erzeugen Item-Ströme und übergeben diese über den Sequenzierer an den Treiber. Es wurden vier Sequenzklassen implementiert:

ExhaustiveSeq – Erschöpfende Abdeckung. Iteriert systematisch über alle 16 Eingabebelegungen $\{(a, b, c, d) \mid a, b, c, d \in \{0, 1\}\}$ durch vier verschachtelte Schleifen. Diese Sequenz garantiert 100 % funktionale Eingangsüberdeckung.

RandomSeq – Zufallstimulation. Erzeugt 20 Items mit zufällig randomisierten Eingaben. Dient zur Simulation eines realitätsnahen, nicht a-priori bekannten Stimulus-Stroms und

zur Erkennung von Fehlern, die bei systematischen Tests möglicherweise übersprungen würden.

CornerCaseSeq – Eckfall-Tests. Testet acht manuell ausgewählte, semantisch bedeutsame Eingabewerte: alle Nullen (Minimalfall, $Y = 1$), alle Einsen (beide AND-Terme aktiv, $Y = 0$), nur der erste AND-Term aktiv, nur der zweite, und alternierende Muster. Eckfälle decken häufig Implementierungsfehler auf, die zufällige Tests statistisch selten treffen.

TestAllSeq – Kombinierte Sequenz. Führt nacheinander **ExhaustiveSeq** und **RandomSeq** aus. Damit wird in einem einzigen Testlauf sowohl vollständige Eingangsüberdeckung als auch realistisches Randomisierungsverhalten erreicht.

```

1 class ExhaustiveSeq(uvm_sequence):
2     async def body(self):
3         for a in range(2):
4             for b in range(2):
5                 for c in range(2):
6                     for d in range(2):
7                         item = AoiSeqItem("ex", a, b, c, d)
8                         await self.start_item(item)
9                         await self.finish_item(item)
10
11 class CornerCaseSeq(uvm_sequence):
12     async def body(self):
13         test_cases = [
14             (0,0,0,0), (1,1,1,1), # Extremwerte
15             (1,1,0,0), (0,0,1,1), # je ein AND-Term aktiv
16             (1,0,0,0), (0,0,0,1), # Einzeleingaben
17             (1,0,1,0), (0,1,0,1), # alternierende Muster
18         ]
19         for a,b,c,d in test_cases:
20             item = AoiSeqItem("cc", a, b, c, d)
21             await self.start_item(item)
22             await self.finish_item(item)

```

Listing 10.4: ExhaustiveSeq und CornerCaseSeq

10.4.3 Treiber: Driver

Der Treiber ist das Bindeglied zwischen der pyUVM-Testbench und dem DUT-Interface auf Hardware-Ebene. Er implementiert `uvm_driver` und führt folgende Schritte pro Item aus:

1. Zusammensetzung des 4-Bit Schalterwertes: $SWT = (d \ll 3) \mid (c \ll 2) \mid (b \ll 1) \mid a$
2. Anlegen des Wertes an `dut.SWT`
3. Warten auf Einschwingen der kombinatorischen Logik: `2 ns (Timer(2, "ns"))`
4. Lesen und Dekodieren des Ausgabewertes von `dut.SEG`: $0x40 \hat{=} Y = 0$, $0x79 \hat{=} Y = 1$
5. Befüllen von `item.result`
6. Broadcast des vollständigen Items über den Analyse-Port `ap`

```

1 class Driver(uvm_driver):
2     def build_phase(self):
3         self.ap = uvm_analysis_port("ap", self)
4         self.dut = cocotb.top
5
6     async def run_phase(self):
7         while True:
8             item = await self.seq_item_port.get_next_item()
9             swt = (item.d << 3) | (item.c << 2) | \
10                 (item.b << 1) | item.a
11             self.dut.SWT.value = swt
12             await Timer(2, unit="ns")
13             seg = int(self.dut.SEG.value)
14             item.result = 0 if seg == 0x40 else 1
15             cmd = (item.a, item.b, item.c, item.d, item.result)
16             self.ap.write(cmd)
17             self.seq_item_port.item_done()

```

Listing 10.5: Treiber-Logik: DUT-Ansteuerung und Ergebnis-Erfassung

Die 2 ns Wartezeit ist ausreichend, da der AOI-2-2-Schaltkreis rein kombinatorisch ist — es gibt keine Takt-Flanken-Abhängigkeit. Für sequentielle Designs wäre hier ein RisingEdge-Trigger notwendig.

10.4.4 Überdeckungs-Komponente: Coverage

Die Coverage-Komponente ist ein `uvm_subscriber` — eine Spezialform von `uvm_component`, die direkt auf den Analyse-Port des Treibers abonniert ist. Sie empfängt jede Transaktion und trägt die beobachtete Eingabebelegung (a, b, c, d) in eine Python-set-Datenstruktur ein:

```

1 class Coverage(uvm_subscriber):
2     def end_of_elaboration_phase(self):
3         self.cvg = set() # beobachtete (a,b,c,d)-Tupel
4
5     def write(self, cmd):
6         (a, b, c, d, _) = cmd
7         self.cvg.add((a, b, c, d))
8
9     def report_phase(self):
10        total = 16 # 2^4 moegliche Eingaben
11        covered = len(self.cvg)
12        pct = covered / total * 100
13        self.logger.info(
14            f"Ueberdeckung: {covered}/{total} ({pct:.1f}%)"
15        )
16        if covered < total:
17            missing = [(a,b,c,d)
18                for a in range(2) for b in range(2)
19                for c in range(2) for d in range(2)
20                if (a,b,c,d) not in self.cvg]
21            self.logger.error(f"Fehlend: {missing}")
22            assert False # Test schlaegt fehl
23        else:
24            self.logger.info("Alle Eingabekombinationen abgedeckt!")

```

Listing 10.6: Coverage: Überdeckungs-Tracking und Berichterstattung

Definition 10.4.1 (Funktionale Eingangsüberdeckung). *Die funktionale Eingangsüberdeckung des AOI-2-2-Schaltkreises ist*

$$C_{\text{fcov}} = \frac{|\{(a, b, c, d) : \text{beobachtet}\}|}{2^4} \in [0, 1].$$

Eine Coverage von $C_{\text{fcov}} = 1$ bedeutet, dass alle 16 Eingabevektoren mindestens einmal angelegt wurden; der Testerfolg ist dann garantiert.

Satz 10.4.1 (Überdeckungsgarantie der ExhaustiveSeq). ExhaustiveSeq erreicht stets $C_{\text{fcov}} = 1$.

Beweis. Die vierfach verschachtelte Schleife iteriert genau einmal über jeden Wert (a, b, c, d) und $(a, b, c, d) \in \{0, 1\}^4$; da $|\{0, 1\}^4| = 16$, wird jede der 16 möglichen Eingabebelegungen exakt einmal angelegt und durch die Coverage-Komponente registriert. $\square$

10.4.5 Scoreboard

Das Scoreboard empfängt über eine FIFO-Queue alle Transaktionen vom Treiber-Analyse-Port, berechnet für jede den erwarteten Ausgabewert mittels der Python-Referenzfunktion `aoi_prediction()` und vergleicht ihn mit dem beobachteten DUT-Ausgabewert:

```

1 class Scoreboard(uvm_component):
2     def build_phase(self):
3         self.cmd_fifo = uvm_tlm_analysis_fifo("cmd_fifo", self)
4         self.cmd_get_port = uvm_get_port("cmd_get_port", self)
5         self.cmd_export = self.cmd_fifo.analysis_export
6
7     def connect_phase(self):
8         self.cmd_get_port.connect(self.cmd_fifo.get_export)
9
10    def check_phase(self):
11        passed = True
12        while self.cmd_get_port.can_get():
13            _, cmd = self.cmd_get_port.try_get()
14            a, b, c, d, actual = cmd
15            expected = aoi_prediction(a, b, c, d, self.errors)
16            if expected != actual:
17                self.logger.error(
18                    f"FAIL: A={a} B={b} C={c} D={d} "
19                    f"-> erwartet {expected}, erhalten {actual}")
20                passed = False
21            else:
22                self.logger.info(f"OK: {a},{b},{c},{d} -> {actual}")
23        assert passed

```

Listing 10.7: Scoreboard: Soll-Ist-Vergleich in der `check_phase`

Das Scoreboard empfängt sowohl `Scoreboard.cmd_export` als auch `Coverage.analysis_export` über denselben Analyse-Port des Treibers — ein klassisches UVM-*Fan-Out*-Muster.

10.4.6 Testumgebung: AoiEnv

Die Umgebungs-klasse `AoiEnv` aggregiert alle Verifikationskomponenten und stellt die Verbindungen zwischen ihnen her:

```

1 class AoiEnv(uvm_env):
2     def build_phase(self):
3         self.seqr = uvm_sequencer("seqr", self)
4         ConfigDB().set(None, "*", "SEQR", self.seqr)
5         self.driver = Driver.create("driver", self)
6         self.coverage = Coverage("coverage", self)
7         self.scoreboard = Scoreboard("scoreboard", self)
8
9     def connect_phase(self):
10        self.driver.seq_item_port.connect(
11            self.seqr.seq_item_export)
12        self.driver.ap.connect(
13            self.scoreboard.cmd_export)
14        self.driver.ap.connect(
15            self.coverage.analysis_export)

```

Listing 10.8: AoiEnv: Aufbau und Vernetzung

Die `ConfigDB().set(None, "*", "SEQR", self.seqr)`-Anweisung in der `build_phase` registriert den Sequenzierer global in der UVM-Konfigurationsdatenbank, sodass Sequenzen ihn über `ConfigDB().get(None, , "SEQR")` abrufen können — ein Standard-UVM-Idiom für komponentenübergreifende Referenzen.

10.4.7 Test-Klassen

Vier Test-Klassen decken unterschiedliche Verifikationsszenarien ab:

Klasse	Sequenz	Zweck
AoiTest	TestAllSeq	Vollständig: erschöpfend + zufällig; $C_{fcov} = 1$ erzwungen
AoiExhaustiveTest	ExhaustiveSeq	Nur erschöpfend; 16 Vektoren
AoiCornerTest	CornerCaseSeq	8 Eckfälle; Coverage-Fehler deaktiviert
AoiTestErrors	TestAllSeq	Fehlerinjektion: erwartet Testfehler

Tabelle 10.2: Übersicht der vier Test-Klassen und ihrer Verifikationsziele.

`AoiTestErrors` ist mit dem Dekorator `@pyuvm.test(expect_fail=True)` annotiert, wodurch ein *Bestehen des Tests* dort registriert wird, wenn — und nur wenn — das Scoreboard einen Fehler meldet. Dieser anti-zyklische Test verifiziert das Verifikationssystem selbst: Er stellt sicher, dass die Testinfrastruktur tatsächlich in der Lage ist, Fehler zu erkennen.

```

1 @pyuvm.test(expect_fail=True)
2 class AoiTestErrors(AoiTest):
3     """Stellt sicher, dass der Fehler erkannt wird"""
4     def start_of_simulation_phase(self):
5         ConfigDB().set(None, "*", "CREATE_ERRORS", True)

```

Listing 10.9: Fehlerinjektions-Test `AoiTestErrors`

10.5 Bus Functional Model: AoiBfm

Der Bus Functional Model (BFM) in `aoi_2_2_utils.py` repräsentiert eine alternative, tiefer abstrahierte Schicht der DUT-Kommunikation. Während der Treiber aus Abschnitt 10.4.3 direkt `cocotb.top`-Signale beschreibt, kapselt das BFM diese Pegel-Zugänge in asynchrone Warteschlangen und dedizierte Coroutinen:

```
1 class AoiBfm(metaclass=Singleton):
2     def __init__(self):
3         self.dut = cocotb.top
4         self.input_queue = Queue(maxsize=1)
5         self.cmd_mon_queue = Queue(maxsize=0)
6         self.result_mon_queue = Queue(maxsize=0)
7
8     async def driver_bfm(self):
9         """Entnimmt Items aus input_queue, legt SWT an"""
10        self.dut.SWT.value = 0
11        while True:
12            try:
13                a,b,c,d = self.input_queue.get_nowait()
14                self.dut.SWT.value = (d<<3)|(c<<2)|(b<<1)|a
15                await Timer(2, units="ns")
16            except QueueEmpty:
17                await Timer(1, units="ns")
18
19    async def cmd_mon_bfm(self):
20        """Beobachtet SWT-Aenderungen, leitet Tupel weiter"""
21        prev_swt = -1
22        while True:
23            await Timer(1, units="ns")
24            cur = int(self.dut.SWT.value)
25            if cur != prev_swt:
26                a=cur&1; b=(cur>>1)&1; c=(cur>>2)&1; d=(cur>>3)&1
27                self.cmd_mon_queue.put_nowait((a,b,c,d))
28                prev_swt = cur
29
30    async def result_mon_bfm(self):
31        """Dekodiert SEG-Ausgang nach jeder SWT-Aenderung"""
32        ...
33
34    def start_bfm(self):
35        cocotb.start_soon(self.driver_bfm())
36        cocotb.start_soon(self.cmd_mon_bfm())
37        cocotb.start_soon(self.result_mon_bfm())
```

Listing 10.10: BFM-Struktur mit Schreib- und Überwachungs-Koroutinen

Das BFM verwendet das *Singleton*-Muster (`metaclass=Singleton`): Es existiert zur Laufzeit genau eine `AoiBfm`-Instanz pro Simulation, auf die alle Komponenten zugreifen. Dies verhindert Races zwischen konkurrierenden Schreibzugriffen auf `dut.SWT`.

Die drei Coroutinen laufen *concurrent* via `cocotb.start_soon()`, was dem Prinzip der parallelen Hardware-Signalverarbeitung entspricht: Treiber-, Kommando-Monitor- und Ergebnis-Monitor-Logik sind voneinander entkoppelt.

10.6 Prädiktionsmodell und Fehlerinjektion

Die Funktion `aoi_prediction()` in `aoi_2_2_utils.py` ist das Python-Referenzmodell (Golden Model) für den AOI-2-2-Schaltkreis:

```

1 def aoi_prediction(a, b, c, d, error=False):
2     """
3     Python-Modell der AOI-2-2-Funktion.
4     Y = NOT((A AND B) OR (C AND D))
5     """
6     result = int(not ((a & b) | (c & d)))
7     if error:
8         result = 1 - result    # Bit-Invertierung fuer Fehlerinjektion
9     return result

```

Listing 10.11: Golden Model: `aoi_prediction()`

Diese einzeilige Implementation bildet Gleichung 10.1 unmittelbar ab. Das optionale `error=True`-Flag invertiert das Ergebnis, was dem Scoreboard ermöglicht, absichtlich falsche Vorhersagen zu machen und damit das Reaktionsverhalten des Testinfrastruktur zu validieren.

Satz 10.6.1 (Korrektheit des Golden Models). *Für alle $(a, b, c, d) \in \{0, 1\}^4$ gilt:*

$$\text{aoi_prediction}(a, b, c, d, \text{False}) = f_{\text{AOI}}(a, b, c, d) \quad (10.2)$$

Beweis. Vollständige Aufzählung: Tabelle 10.1 zeigt alle 16 Eingabebelegungen. Die Python-Auswertung von `int(not ((a & b) | (c & d)))` ist für jeden Wert direkt überprüfbar und stimmt mit der Wahrheitstabelle überein. $\square$

10.7 Test-Strategien und Verifikationsabdeckung

Definition 10.7.1 (Vollständige Eingangsüberdeckung). *Ein Testlauf erreicht vollständige Eingangsüberdeckung für den AOI-2-2-Schaltkreis, wenn alle $2^4 = 16$ Eingabebelegungen mindestens einmal simuliert wurden, d. h. $C_{\text{fcov}} = 1$.*

Die vier Sequenzen decken unterschiedliche Teilbereiche des Stimulus-Raums ab:

Sequenz	Items	C_{fcov}	Stärke
ExhaustiveSeq	16	$= 1$ (garantiert)	Vollständige systematische Abdeckung
RandomSeq	20	< 1 (erwartet)	Realitätsnaher Stimulus; statistisch $\approx 91\%$
CornerCaseSeq	8	0,5	Gezielte Eckfälle; Design-Intent-Test
TestAllSeq	≥ 36	$= 1$	Kombiniert exhaustiv + zufällig

Tabelle 10.3: Gegenüberstellung der Sequenzen nach Stimulus-Anzahl und Überdeckungsgrad.

Statistischer Überdeckungsnachweis für RandomSeq. Die Wahrscheinlichkeit, dass nach n zufälligen, gleichverteilten Items alle $k = 16$ Belegungen mindestens einmal aufgetreten sind, entspricht dem Coupon-Collector-Problem:

$$P(\text{alle } k \text{ nach } n \text{ Items}) = \frac{k!}{k^n} \cdot S(n, k), \quad (10.3)$$

wobei $S(n, k)$ die Stirling-Zahl zweiter Art ist. Für $k = 16$ und $n = 20$ ist diese Wahrscheinlichkeit mit ca. 14,3% relativ gering — weshalb **RandomSeq** allein *nicht* als hinreichende Verifikationsmethode gilt, sondern stets in Kombination mit **ExhaustiveSeq** (via **TestAllSeq**) eingesetzt wird.

10.8 Make-basierter Simulations-Workflow

Das **Makefile** implementiert einen dualen Simulations-Workflow, der beide HDL-Varianten mit je einem spezialisierten Open-Source-Simulator unterstützt:

HDL	Simulator	Make-Befehl
Verilog	Verilator	make coverage SIM=verilator
VHDL	GHDL	make sim TOPLEVEL_LANG=vhdl SIM=ghdl

Tabelle 10.4: Dualer Simulations-Workflow: Verilog mit Verilator, VHDL mit GHDL.

10.8.1 Verilog-Simulation mit Verilator

Verilator kompiliert Verilog-Code in C++-Code und erzeugt hochperformante Simulations-Executables. Im vorliegenden Makefile werden bei Auswahl **SIM=verilator** zusätzlich Zeilend-, Toggle- und Branch-Überdeckung aktiviert:

```

1 ifeq ($(SIM), verilator)
2     # Unterstuetzung fuer #delay-Anweisungen
3     COMPILE_ARGS += --timing
4     # Zeilenueberdeckung, Toggle- und Branch-Coverage
5     COMPILE_ARGS += --coverage --coverage-line --coverage-toggle
6     EXTRA_ARGS   += --coverage
7 endif

```

Listing 10.12: Verilator-spezifische Makefile-Konfiguration

Nach der Simulation erzeugt Verilator eine **coverage.dat**-Datei. Diese wird durch **verilator_coverage** in eine annotierte HTML-Ausgabe transformiert:

```

1 # Simulation mit Ueberdeckungs-Messung
2 make clean
3 make coverage SIM=verilator
4
5 # Annotierter HTML-Bericht
6 make coverage-report
7
8 # Universeller Python-Bericht
9 python3 generate_universal_coverage.py --sim verilator

```

Listing 10.13: Überdeckungsbericht mit Verilator

10.8.2 VHDL-Simulation mit GHDL

GHDL ist ein quelloffener VHDL-Simulator mit IEEE-1076-2008-Unterstützung. Er wird über den `SIM=ghdl`-Parameter aufgerufen:

```
1 # Simulation der VHDL-Implementierung
2 make clean
3 make sim TOPLEVEL_LANG=vhdl SIM=ghdl
4
5 # Universeller Ueberdeckungsbericht
6 python3 generate_universal_coverage.py --sim ghdl
```

Listing 10.14: VHDL-Simulation mit GHDL

Bemerkung 10.8.1. *GHDL-Coverage-Unterstützung erfordert einen Build mit GCC- oder LLVM-Backend (`--coverage`). Im Standard-`mcode`-Backend steht Überdeckungsmessung nicht zur Verfügung. Das Makefile enthält die entsprechenden GHDL-Optionen kommentiert, um die Aktivierung bei verfügbarem Backend zu erleichtern.*

10.8.3 Quelldatei-Auflösung im Makefile

Das Makefile unterstützt zwei Suchpfade für die HDL-Quelldateien — den Standard-Unterordner `hdl/verilog/` bzw. `hdl/vhdl/` sowie ein optionales lokales Verzeichnis — und löst diese zur Compile-Zeit auf:

```
1 ifeq ($(TOPLEVEL_LANG),verilog)
2     ifneq ("$(wildcard $(CWD)/hdl/verilog/aoi_2_2.v)", "")
3         VERILOG_SOURCES = $(CWD)/hdl/verilog/aoi_2_2.v
4     else ifneq ("$(wildcard $(CWD)/aoi_2_2.v)", "")
5         VERILOG_SOURCES = $(CWD)/aoi_2_2.v
6     else
7         $(error "Datei aoi_2_2.v nicht gefunden")
8     endif
9 endif
```

Listing 10.15: Robuste Quelldatei-Suche im Makefile

10.9 Überdeckungsbericht

Das Skript `generate_universal_coverage.py` erzeugt einen einheitlichen HTML-Überdeckungsbericht, der sowohl Verilator- als auch GHDL-Simulationen unterstützt. Es implementiert eine abstrakte Parser-Hierarchie:

```
1 class CoverageParser:          # abstrakte Basisklasse
2     def parse(self): raise NotImplementedError
3
4 class VerilatorCoverageParser(CoverageParser):
5     """Parst annotierte Verilator-Coverage-Dateien"""
6     def parse(self):
7         # Interpretiert Praefixe: ~ = partiell,
8         # %000000 = nicht abgedeckt, sonst: Zaehler-Wert
9         ...
10
11 class GHDLCoverageParser(CoverageParser):
12     """Analysiert GHDL-Quelldatei heuristisch"""
13     def parse(self):
```

```

14     # Heuristik: Zeilen mit Zuweisungen/case/if
15     # als abgedeckt markieren (vereinfacht)
16     ...

```

Listing 10.16: Abstrakte Parser-Hierarchie für universelle Coverage-Berichte

Aus den Parsed-Daten wird ein HTML-Dokument mit:

- Farbkodierter Zeilenüberdeckung: grün (abgedeckt), rot (nicht abgedeckt), gelb (partiell)
- Statistik-Header: Prozentsatz abgedeckter Zeilen, Simulator-Badge
- Simulator-spezifischer Fußzeile mit Erklärungen zur Überdeckungsmethodik

```

1 # Fuer Verilator (nach 'make coverage-report'):
2 python3 generate_universal_coverage.py --sim verilator
3
4 # Fuer GHDL (nach 'make sim TOPLEVEL_LANG=vhdl SIM=ghdl'):
5 python3 generate_universal_coverage.py --sim ghdl

```

Listing 10.17: Aufruf des universellen Überdeckungsberichts

10.10 Laufzeit-Profiling

Das Skript `profile_simulation.sh` kombiniert die Cocotb-Simulation mit dem Python-Profiler `cProfile` und erzeugt eine `.pstat`-Datei, die anschließend mit `analyze_profile.py` ausgewertet wird:

```

1 #!/bin/bash
2 make clean
3 python3 -m cProfile -o simulation_profile.pstat \
4     -m cocotb.runner \
5     --toplevel aoi_2_2 \
6     --module testbench

```

Listing 10.18: Simulations-Profiling mit cProfile

```

1 def analyze_profile(profile_file="profile.pstat"):
2     stats = pstats.Stats(profile_file)
3
4     print("TOP 20 FUNKTIONEN NACH KUMULATIVER ZEIT:")
5     stats.sort_stats("cumulative")
6     stats.print_stats(20)
7
8     print("TOP 20 NACH EIGENZEIT:")
9     stats.sort_stats("time")
10    stats.print_stats(20)
11
12    print("TOP 20 NACH AUFRUFANZAHL:")
13    stats.sort_stats("calls")
14    stats.print_stats(20)

```

Listing 10.19: `analyze_profile.py`: Auswertung der Profil-Daten

Das Profiling dient in erster Linie der Identifikation von Laufzeit-Engpässen in der Python-Testbench, nicht im DUT selbst. Typische Hot Spots in cocotb- basierten Testbenches sind: `Timer`-Triggers, Queue-Operationen und die UVM-interne Phasen-Dispatch-Logik. Diese Informationen sind wertvoll für die Optimierung großer Testsuiten mit Tausenden von Items.

10.11 Simulations- vs. formal-basierte Verifikation

Die simulations-basierte und die formal-basierte Verifikation sind *komplementär*: Jede Methode hat spezifische Stärken und Schwächen, die die jeweils andere ergänzt.

Eigenschaft	Formal (Yosys/-SAT)	Simulation (pyUVM)
Vollständigkeit	Vollständig für alle Eingaben	Stichprobenartig
Überprüfte Eigenschaft	Äquivalenz zweier Designs	Korrektheit gegen Referenzmodell
Überdeckungsmetrik	n/a (vollständig per Konstruktion)	$C_{\text{fcov}} \in [0, 1]$
Fehlertypen	Strukturelle und funktionale Fehler	Funktionale Fehler, Timing
Skalierung	Exponentiell (SAT, Worst-Case)	Linear in Testvektorzahl
Laufzeit für AOI-2-2 Fehlerinjektion	< 1 s Durch Mutationsdesigns	< 5 s (alle Tests) Durch <code>CREATE_ERRORS</code> -Flag
CI/CD-Integration	Hoch (deterministisch)	Hoch (determinierbar durch Seed)
Lernkurve	Steil (Yosys-Syntax)	Moderat (Python/UVM)

Tabelle 10.5: Systematischer Vergleich formaler und simulations-basierter Verifikation für den AOI-2-2-Schaltkreis.

Optimale Kombination. Die in dieser Arbeit empfohlene Verifikationsstrategie lautet:

1. **Simulations-basierte Erstverifikation:** pyUVM-Testbench deckt funktionale Fehler früh im Entwicklungsprozess auf; die Laufzeit ist gering und die Feedback-Schleife kurz.
2. **Formale Äquivalenzprüfung:** Yosys beweist abschließend die Äquivalenz zwischen Verilog- und VHDL-Fassung für alle Eingaben, ohne Stichprobencharakter.
3. **Überdeckungsnachweis:** Die Coverage-Komponente stellt sicher, dass die Simulation alle Verhaltensklassen erfasst hat; Lücken werden als Test-Fehler behandelt.

Satz 10.11.1 (Komplementarität von Simulation und formaler Verifikation). *Eine simulations-basierte Verifikation mit $C_{\text{fcov}} = 1$ und eine formale Äquivalenzprüfung (UNSAT) zusammen implizieren: Das DUT-Design ist korrekt gegenüber dem Referenzmodell für alle Eingaben, und das Referenzmodell wurde für alle möglichen Eingabeklassen evaluiert.*

Beweis. Die formale Äquivalenzprüfung (UNSAT) zeigt: Für alle $\vec{x}$, $\text{DUT}(\vec{x}) = \text{Ref}(\vec{x})$. Die Simulation mit $C_{\text{fcov}} = 1$ stellt sicher, dass alle 16 Eingabebelegungen mindestens einmal durch das Scoreboard geprüft wurden und das Golden Model in jedem Fall korrekte Ausgaben lieferte (Satz 10.6.1). Beide Bedingungen zusammen garantieren vollständige Korrektheit des DUT gegen eine validierte Spezifikation. $\square$

10.12 Zusammenfassung

In diesem Kapitel wurde eine vollständige simulations-basierte Testbench für den AOI-2-2-Schaltkreis entwickelt und auf dessen Verilog- und VHDL-Implementierung angewendet. Die wesentlichen Ergebnisse sind:

1. **Vollständige UVM-Architektur:** Sequenz-Items, vier Sequenzklassen, Treiber, Scoreboard, Coverage-Komponente und Umgebungsklasse wurden nach dem UVM-1800.2-Standard in pyUVM implementiert.
2. **Duale HDL-Unterstützung:** Dieselbe Testbench simuliert sowohl die Verilog-Fassung (Verilator) als auch die VHDL-Fassung (GHDL), ohne Änderungen am Testbench-Code.
3. **Überdeckungsgarantie:** Die `ExhaustiveSeq` erreicht stets $C_{\text{cov}} = 1$ (Satz 10.4.1); die Coverage-Komponente erzwingt diesen Nachweis als Test-Fehlerbedingung.
4. **Golden Model und Fehlerinjektion:** Die `aoi_prediction()`-Funktion stellt ein mathematisch korrektes Referenzmodell bereit (Satz 10.6.1); das Fehlerinjektionsflag validiert die Reaktionsfähigkeit der Testinfrastruktur selbst.
5. **Universelles Überdeckungs-Reporting:** Das Skript `generate_universal_coverage.py` erzeugt simulator-unabhängige HTML-Berichte für Verilator und GHDL.
6. **Komplementarität:** Simulations-basierte und formal-basierte Verifikation ergänzen sich zur vollständigen Verifikationskette (Satz 10.11.1).

Kapitel 11

Schluss

Diese Arbeit hat gezeigt:

1. Amaranth kann VHDL-Designs funktional äquivalent reimplementieren.
2. Die Äquivalenz lässt sich mit Open-Source-Tools (Yosys, SAT-Solver) formal beweisen.
3. Der Workflow von Python-Code zu Hardware-Validierung ist praktikabel und vollständig reproduzierbar.
4. Amaranth bietet Produktivitätsvorteile bei erhaltener Korrektheit: ca. 200 Zeilen Python gegenüber ca. 250 Zeilen VHDL, mit identischem Syntheseresultat.
5. Der Subgraph Algorithmus [1] stellt eine theoretisch fundierte, polynomial-zeitliche Methode zur strukturellen Äquivalenzprüfung von Hardware-Designs auf Graph-Ebene bereit (Kapitel 8).
6. Acht neu hergeleitete Sätze mit vollständigen Beweisen (Kapitel 8) belegen Korrektheit, Laufzeit $O(n^3)$, Sensitivität gegenüber Einzelkanten-Mutationen, Transitivität, super-exponentiellen Speedup gegenüber naiver Permutationssuche sowie SETH-bedingte Optimalität des Verfahrens.
7. Die SAT-basierte und die SubgraphAlgorithmus-basierte Methode ergänzen sich komplementär: erstere prüft funktionale Äquivalenz auch nach Logikoptimierung, letztere prüft strukturelle Äquivalenz mit polynomieller Laufzeitgarantie.
8. Kapitel 9 führt einen vollständigen, dreistufigen Äquivalenzbeweis zwischen einer Verilog- und einer VHDL-Implementierung der CITEC-ALU-Schaltung: (a) manuelle Inspektion aller sechs Operationen und der sequentiellen Logik, (b) strukturelle Prüfung mittels Subgraph Algorithmus auf dem 11×11 -Abhängigkeitsgraphen sowie (c) SAT-basierte Bestätigung durch Yosys. Das Hauptresultat (Satz 9.7.1) beweist vollständige Ausgabeäquivalenz für alle Eingabesequenzen und identische Anfangszustände.
9. Kapitel 10 demonstriert eine vollständige simulations-basierte Testbench auf Basis von pyUVM [16] und cocotb [13] für den AOI-2-2-Schaltkreis. Dieselbe Testbench simuliert sowohl die Verilog-Fassung (Verilator mit Zeilenüberdeckung) als auch die VHDL-Fassung (GHDL), erzwingt vollständige funktionale Eingangsüberdeckung

($C_{\text{fcov}} = 1$) und validiert das Verifikationssystem selbst durch Fehlinjektion. Die Methode ergänzt die formale Verifikation um dynamische Überdeckungsnachweise und Fehlerinjektionstests in einer einheitlichen, Python-nativen Verifikationskette.

11.1 Andere Python-basierte HDLs

MyHDL MyHDL [4] ist ein älteres Python-basiertes HDL, das ähnliche Ziele wie Amaranth verfolgt. Im Vergleich bietet Amaranth:

- Modernere Python-Syntax und -Features
- Bessere Integration mit Yosys
- Aktivere Community und Entwicklung

Migen Amaranth ist der Nachfolger von Migen. Wichtige Verbesserungen:

- Klarere Trennung von kombinatorischer und synchroner Logik
- Verbesserte Fehlerbehandlung
- Standardisierte Build-System-Integration

Open-Source-Alternativen Yosys bietet vergleichbare Funktionalität für Open-Source-Workflows. Limitationen bestehen bei sehr großen Designs ($> 10^6$ Gates).

11.2 Zukünftige Arbeiten

Die vorliegende Arbeit hat eine grundlegende Methodik zur formalen Verifikation von Amaranth-HDL-Designs etabliert und deren Praktikabilität anhand einer ALU mit 7-Segment-Ausgabe demonstriert. Sie legt damit ein Fundament, auf dem eine Vielzahl von weiterführenden Forschungs- und Entwicklungsarbeiten aufgebaut werden kann. Die folgenden Abschnitte skizzieren diese Perspektiven im Detail.

11.2.1 Verifikation komplexerer Hardware-Designs

Vollständige Prozessorkerne Der natürliche nächste Schritt ist die Übertragung der Methodik auf vollständige Prozessorkerne. RISC-V hat sich als de-facto-Standard für Open-Source-Prozessoren etabliert; prominente Open-Source-Implementierungen wie PicoRV32 oder VexRiscv existieren sowohl in Verilog/VHDL als auch in Teilen bereits als Amaranth-Portierungen. Ein umfassender Äquivalenznachweis für einen 32-Bit-RISC-V-Kern würde die Skalierbarkeit der vorgestellten Methodik direkt unter Beweis stellen. Dabei müssen insbesondere mehrtaktige Operationen, Pipeline-Stufen und bedingte Sprünge korrekt modelliert werden, was den Einsatz tiefer induktiver Beweisführung (etwa `equiv_induct` mit $k > 20$ Schritten) notwendig macht. Die Äquivalenzprüfung eines Prozessorkerns stellt wegen der inhärenten Zustandsraumexplosion eine methodische Herausforderung dar, die angepasste Abstraktions- und Decompositions-Strategien erfordert.

Bus-Protokolle und On-Chip-Interconnects Moderne SoC-Designs bestehen zu einem erheblichen Teil aus Bus-Protokoll-Logik. Standardinterfaces wie AXI4, AXI4-Lite, Wishbone oder APB weisen präzise, formal spezifizierbare Protokollgarantien auf (korrekte Handshake-Sequenzen, keine Deadlocks, kein Datenverlust). Zukünftige Arbeiten könnten Amaranth-Implementierungen dieser Protokolle formal gegen Referenzimplementierungen oder gegen formale Protokoll-spezifikationen (z. B. in PSL oder SVA) verifizieren. Dies wäre insbesondere für Safety-kritische Anwendungen wertvoll, in denen Fehler in der Interconnect-Logik schwerwiegende Systemausfälle verursachen können.

Digitale Signalverarbeitungs-Pipelines DSP-Pipelines (FIR-Filter, FFT, Korrelationsberechnungen) gelten als besonders fehleranfällig, da numerische Eigenschaften wie Overflow-Verhalten, Rundungsfehler und Sättigungsarithmetik schwer zu überblicken sind. Amaranth eignet sich durch seine Python-Integration gut für die Beschreibung von DSP-Strukturen, da Filterkoeffizienten und Topologien direkt aus NumPy-Berechnungen abgeleitet werden können. Die formale Verifikation dieser Designs würde den Wertebereich aller Zwischen- und Ausgangssignale sowie das Sättigungsverhalten beweisbar korrekt machen. Dies erfordert jedoch die Einbindung von Bit-precise-Arithmetik in den SAT-Beweis, was den Einsatz von SMT-Solvern (z. B. Z3 oder Bitwuzla) nahelegt.

Speichercontroller und DRAM-Interfaces Die Ansteuerung von DRAM-Bausteinen (DDR3/DDR4/LPDDR) erfordert zeitlich präzise Signalfolgen mit engen Timing-Fenstern. Eine Amaranth-basierte Implementierung eines DDR-PHY-Controllers, verifiziert gegen eine JEDEC-konforme Referenz, wäre ein bedeutender industrieller Beitrag. Gleichzeitig illustrierte ein solches Projekt die Grenzen der rein logischen Äquivalenzprüfung: Timing-Properties können nicht allein durch strukturelle Äquivalenz nachgewiesen werden und erfordern ergänzend statische Timing-Analyse (STA) oder Timed-Model-Checking.

11.2.2 Erweiterung der formalen Verifikationstechniken

Systemische Property-Verification mit SVA und PSL Neben der reinen Äquivalenzprüfung ist die property-basierte Verifikation ein eigenständiges und mächtiges Paradigma. SystemVerilog Assertions (SVA) und die Property Specification Language (PSL) erlauben es, Sicherheits- (*safety*) und Lebendigkeitseigenschaften (*liveness*) direkt am Design zu annotieren. Beispiele für relevante Properties der vorgestellten ALU wären:

- **Overflow-Freiheit:** $\forall a, b \leq 15 : \text{ADD}(a, b) \leq 30$ — keine impliziten Überläufe bei 4-Bit-Operanden.
- **Division-by-Zero-Sicherheit:** $b = 0 \Rightarrow \text{DIV}(a, b) = 0\text{xFF}$ — der Fehlerwert wird in jedem Cycle korrekt ausgegeben.
- **Display-Vollständigkeit:** Jede der vier Displaystellen wird in jedem Zyklus von $4 \times T_{\text{refresh}}$ genau einmal aktiviert.

Zukünftige Arbeiten könnten PSL-Annotationen direkt in den Amaranth-Code integrieren (ähnlich wie `assert` in simulationsbasierten Frameworks) und deren automatische Übersetzung in Yosys-Formal-Checks realisieren.

SMT-Solver-Integration Yosys unterstützt eine SMT-basierte Back-End-Verifikation über das Tool `sby` (SymbiYosys). Während SAT-Solver für kombinatorische Äquivalenzprüfung gut geeignet sind, erlauben SMT-Solver (Satisfiability Modulo Theories) die Einbeziehung arithmetischer Theorien (Linear Integer Arithmetic, Bit-Vector-Logik). Dies ist besonders relevant für DSP-Designs oder für den Nachweis arithmetischer Identitäten. Eine Direkt-Integration des SymbiYosys-Workflows in die Amaranth-Build-Pipeline, analog zur beschriebenen Yosys-Pipeline, wäre ein wertvoller Folgebeitrag.

Formale Verifikation mit temporaler Logik Model Checking mithilfe von temporaler Logik (LTL, CTL) erlaubt den Nachweis von Eigenschaften über zeitliche Verläufe, die über Äquivalenzprüfung hinausgehen. So könnte für das Display-Multiplexing formal gezeigt werden, dass keine Stelle dauerhaft ausgeblendet bleibt (*Liveness*) und keine zwei Stellen gleichzeitig aktiv sind (*Mutual Exclusion*). Open-Source-Tools wie NuSMV oder DIVINE könnten hier als Ergänzung zu Yosys eingesetzt werden.

11.2.3 Automatisierung und CI/CD-Integration

Kontinuierliche Verifikation in GitHub Actions Moderne Hardware-Projekte profitieren von automatisierten Qualitätssicherungspipelines analog zur Praxis in der Softwareentwicklung. Die vorgestellte Verifikationsmethodik eignet sich ideal für die Integration in CI/CD-Workflows: Bei jedem Commit könnten automatisch die folgenden Stufen durchlaufen werden:

1. Syntaxvalidierung des Amaranth-Python-Codes via `pylint/mypy`
2. Verilog-Generierung via Amaranth-Backend
3. Äquivalenzprüfung mit Yosys gegen die versionierte VHDL-Referenz
4. Ressourcen-Synthese mit Yosys und automatische Erkennung von Ressourcen-Regressionen ($> 5\%$ LUT-Zunahme)
5. Optionale FPGA-Synthese in Vivado (z. B. täglich, nicht bei jedem Commit)

GitHub Actions mit vorbereiteten Docker-Containern (z. B. `hdl/containers`) können Yosys, openFPGALoader und Python-Abhängigkeiten reproduzierbar bereitstellen, ohne dass CI-Runner lokal installiert werden müssen.

Regressionstests und Testabdeckungsmetriken Analog zur Linienabdeckung in der Softwareverifikation existieren für formale Hardware-Verifikation Metriken wie *Mutation Coverage* (werden Fehlerinjektionen erkannt?) und *Toggle Coverage* (wechseln alle Signale in der Simulation ihren Wert?). Zukünftige Arbeiten könnten automatisierte Mutationstests für Amaranth-Designs entwickeln: Dabei werden systematisch synthetische Fehler (Bit-Flip in einer Konstante, vertauschte Operanden) in das Design injiziert, und verifiziert, dass der Äquivalenz-Checker diese erkennt. Dies qualifiziert gleichzeitig die Verifikationsstärke des Workflows selbst.

Versionierung von Verifikationsbeweisen Eine oft vernachlässigte Praxis ist die Versionierung nicht nur des Designs, sondern auch der Verifikationsergebnisse. Das Log einer erfolgreich abgeschlossenen Yosys- Äquivalenzprüfung sollte zusammen mit dem Design-Commit committet werden, sodass nachvollziehbar ist, *welche Version* zu welchem Zeitpunkt formal verifiziert wurde. Hierfür kann ein einfaches Metadaten-Format (JSON mit Zeitstempel, Commit-Hash und Yosys-Version) automatisch am Ende des CI-Skripts generiert und archiviert werden.

11.2.4 Vollständig quelloffene Toolchain

F4PGA / Symbiflow für Xilinx-Targets Die in dieser Arbeit beschriebene Toolchain erfordert für den Synthese- und Implementierungsschritt noch das proprietäre Vivado. Das Projekt F4PGA (ehemals Symbiflow) [10] verfolgt das Ziel, für Xilinx Artix-7 und andere FPGAs eine vollständig quelloffene Implementierungskette bereitzustellen. Zum Entstehungszeitpunkt dieser Arbeit ist die Artix-7-Unterstützung in F4PGA noch nicht produktionsreif; insbesondere fehlen offizielle Timing-Datenbanken. Sobald diese Lücken geschlossen sind, ließe sich der gesamte Workflow von der Amaranth-Beschreibung bis zur bitgenauen Programmierung des FPGA ohne proprietäre Software realisieren. Dies wäre ein bedeutender Schritt für Lehre, Reproduzierbarkeit und Open-Science-Prinzipien.

GHDL als VHDL-Frontend für Yosys Mit GHDL [11] steht ein quelloffener VHDL-Simulator und -Synthesizer zur Verfügung, der über ein Yosys-Plugin (`ghdl-yosys-plugin`) direkt VHDL-Designs in Yosys laden kann — ohne den Umweg über eine KI-gestützte VHDL-zu-Verilog-Konvertierung. Zukünftige Versionen des vorgestellten Workflows sollten GHDL als Standard-Frontend verwenden: Das VHDL-Original wird direkt geladen, die manuelle Konvertierungsstufe entfällt, und potenzielle Semantikdifferenzen durch den Übersetzungsschritt werden ausgeschlossen. GHDL unterstützt VHDL-2008 und deckt damit auch fortgeschrittene Sprachfeatures ab.

nextpnr für Lattice- und Intel-FPGAs Der Open-Source-Place-and-Route-Tool `nextpnr` unterstützt bereits vollständig die Lattice-iCE40- und ECP5-Familien sowie (im Beta-Stadium) Intel Cyclone-10. In Kombination mit `Project IceStorm` (iCE40) oder `Project Trellis` (ECP5) ergibt sich eine stabile, vollständig quelloffene Implementierungskette. Die Übertragung der vorgestellten Arbeiten auf kostengünstige Entwicklungsboards wie das iCEbreaker (Lattice iCE40UP5k) hätte den Vorteil, dass keine proprietäre Software erforderlich ist, und wäre damit besonders für Lehrveranstaltungen geeignet.

11.2.5 Erweiterte Testmethoden

Property-based Testing mit Hypothesis Der beschriebene formale Äquivalenzbeweis ist vollständig und gilt für alle möglichen Eingabekombinationen. Ergänzend hierzu kann *Property-based Testing* auf Simulationsebene wertvolle zusätzliche Erkenntnisse liefern, insbesondere in frühen Designphasen, bevor die formale Verifikation aufgesetzt ist. Das Python-Framework `Hypothesis` [12] generiert automatisch Eingabesequenzen, die Randfälle und Extremwerte gezielt abdecken, und kann mit dem Amaranth-Simulator kombiniert werden:

```
1 from hypothesis import given, settings
2 from hypothesis import strategies as st
3 from amaranth.sim import Simulator
```

```

4
5 @given(
6     a=st.integers(min_value=0, max_value=15),
7     b=st.integers(min_value=1, max_value=15),
8 )
9 @settings(max_examples=1000)
10 def test_division_property(a, b):
11     """Quotient darf nie groesser als Dividend sein"""
12     result = simulate_alu(a, b, OP_DIV)
13     assert result <= a

```

Listing 11.1: Hypothesis-basierter ALU-Test

Property-based Tests decken Implementierungsfehler auf, die im formalen SAT-Modell möglicherweise durch fehlerhafte Modellierung nicht sichtbar werden, und dienen somit als orthogonale Verifikationsebene.

Simulation mit Cocotb und pyUVM Cocotb [13] ist ein Python-basiertes Testbench-Framework für Hardware-Simulationen. Da Amaranth selbst Python-basiert ist, entsteht eine natürliche Synergie: Testbenches, Stimulusgeneratoren und Assertion-Überprüfungen können in derselben Python-Codebasis wie das Design selbst geschrieben werden. Die in Kapitel 10 vorgestellte pyUVM-Testbench für den AOI-2-2-Schaltkreis legt dafür eine konkrete, vollständig reproduzierbare Basis: die duale HDL-Unterstützung (Verilog/Verilator und VHDL/GHDL), die erzwungene Überdeckungsmetrik ($C_{\text{fcov}} = 1$) und die Fehlerinjektions-Tests können unmittelbar auf komplexere Designs übertragen werden. Zukünftige Arbeiten könnten diese Architektur auf die ALU aus Kapitel 3 ausdehnen und eine integrierte Verifikationspipeline entwickeln, die pyUVM-Simulation, Yosys-Äquivalenzprüfung und Überdeckungsnachweis in einem einzigen CI-Schritt verbindet.

Erweiterung der pyUVM-Testbench auf sequentielle Designs Der in Kapitel 10 vorgestellte pyUVM-Ansatz ist explizit für einen kombinatorischen 4-Eingangs-Schaltkreis konzipiert. Für sequentielle Designs — etwa die ALU mit Takt-getaktetem Display-Multiplexer — sind folgende Erweiterungen notwendig und unmittelbar umsetzbar: (a) Ersetzen des Timer-basierten Einschwingens durch `RisingEdge`-Trigger; (b) Erweiterung der Coverage-Klasse auf zustandsbehaftete Überdeckungsmodelle (z. B. Cross-Coverage zwischen Operationscode und Operanden); (c) Einführung eines Referenz-Zustandsautomaten im Golden Model für den Display-Multiplexer. Diese Erweiterungen würden pyUVM als vollwertige Verifikationsplattform für alle in dieser Arbeit behandelten Designs etablieren.

Hardware-in-the-Loop-Tests Für zeitkritische Designs ist die simulationsbasierte Verifikation allein nicht ausreichend. Hardware-in-the-Loop (HiL)-Tests betreiben das synthetisierte FPGA-Design unter realen Bedingungen und lesen Ergebnisse über ein UART- oder JTAG-Interface automatisiert aus. Ein Python-Testrahmen, der die Werte des Amaranth-Simulators mit den auf dem FPGA gemessenen Ausgaben vergleicht, bildet eine vollständige, dreigliedrige Verifikationskette: *formale Äquivalenz* (Yosys), *funktionale Simulation* (Cocotb/Amaranth-Sim) und *physikalische Validierung* (HiL auf Basys 3).

11.2.6 Hardware-Software-Co-Design

Integration von Amaranth mit Embedded-Software-Stacks Ein zunehmendes Anwendungsfeld für FPGAs ist das Hardware-Software-Co-Design, bei dem ein Soft-

Core-Prozessor (z. B. PicoRV32) zusammen mit applikationsspezifischen Hardware-Acceleratoren auf dem FPGA instanziiert wird. Amaranth eignet sich hervorragend, um solche Acceleratoren (z. B. einen AES-Encoder, einen CRC-Calculator oder eine Integer-Multiplikationseinheit) auf hohem Abstraktionsniveau zu beschreiben. Zukünftige Arbeiten könnten zeigen, wie die formally-verified Amaranth-Hardware-Komponenten nahtlos in einen SoC-Stack integriert werden, bei dem die Software-API und die Hardware-Logik aus derselben Python-Codebasis abgeleitet werden.

Automatische Interface-Generierung Die Port-Definitionen von Amaranth-Modulen liegen als Python-Objekte vor. Daraus lässt sich automatisch C-Header-Code generieren, der die Registeradressen und Bitfeldmasken des Hardware-Interfaces für die Embedded-Software definiert. Dieses Prinzip (bekannt aus Frameworks wie LiteX [14]) reduziert den Synchronisationsaufwand zwischen Hardware- und Softwareentwicklung erheblich und verhindert Inkonsistenzen bei Interface-Änderungen.

11.2.7 Skalierbarkeit und Grenzen der formalen Verifikation

Zustandsraumexplosion bei tiefer Sequentialität Die induktive Äquivalenzprüfung mit `equiv_induct` skaliert gut für Designs mit geringer sequentieller Tiefe. Bei tiefen Pipelines (z. B. einem 5-stufigen RISC-V-Pipeline mit Forwarding-Logik) kann die Menge der zu explorierenden Zustandsübergänge die Kapazität des SAT-Solvers übersteigen. Kontermaßnahmen sind:

- **Compositionelle Verifikation:** Das Design wird in unabhängige Teilkomponenten zerlegt, die separat verifiziert werden, und die Korrektheit der Komposition wird durch Annahme-Garantie-Kontrakte (Assume-Guarantee) formal abgesichert.
- **Abstraktions-Verfeinerung (CEGAR):** Counterexample-Guided Abstraction Refinement beginnt mit einem abstrakten, vereinfachten Modell und verfeinert es iterativ, bis der Beweis gelingt oder ein echtes Gegenbeispiel gefunden ist.
- **Bounded Model Checking:** Statt eines vollständigen Induktionsbeweises werden nur die ersten k Taktzyklen geprüft, was bei bekannten Designzyklen ausreicht.

Zukünftige Arbeiten sollten diese Techniken systematisch für wachsende Amaranth-Designs evaluieren und Schwellenwerte (*Complexity Horizons*) empirisch bestimmen.

Grenzen der Yosys-Äquivalenzprüfung bei komplexer Arithmetik Die vorliegende Arbeit hat gezeigt, dass Yosys für eine 4-Bit-ALU korrekte Beweise liefert. Für Designs mit 64-Bit-Integer-Arithmetik, Floating-Point-Einheiten oder parametrisierbaren Multiplizierer-Bäumen können dedizierte arithmetische SMT-Solver (Bitwuzla, CVC5) bessere Beweisleistung erzielen. Eine vergleichende Studie zwischen der Yosys-SAT-Methodik und SMT-basierten Ansätzen für arithmetikintensive Designs wäre wissenschaftlich wertvoll.

11.2.8 Einsatz in der Hochschullehre

Amaranth als Lehrmittel in Digitalelektronik-Kursen Die Python-Syntax von Amaranth senkt die Einstiegshürde für Studierende gegenüber VHDL erheblich. Gleichzeitig erzwingt die strikte Trennung von kombinatorischer und synchroner Logik (`module.d.comb`

vs. `module.d.sync`) ein klares Verständnis dieser fundamentalen Konzepte. Zukünftige Arbeiten könnten einen didaktisch aufbereiteten Lehrpfad entwickeln, der:

1. Mit einfachen kombinatorischen Schaltungen beginnt (Multiplexer, Decoder),
2. Sequentielle Designmuster einführt (Zähler, Zustandsautomaten),
3. Zur formalen Verifikation mit Yosys führt und
4. Mit einer vollständigen FPGA-Implementierung abschließt.

Reproduzierbarkeit von Forschungsergebnissen Die Reproducibility-Crisis in der Informatikforschung betrifft zunehmend auch Hardware-Design-Arbeiten. Der in dieser Arbeit vorgestellte Workflow bietet durch den Einsatz ausschließlich quelloffener Tools (Amaranth, Yosys, openFPGALoader) und versionierter Abhängigkeiten eine hohe Reproduzierbarkeit. Zukünftige Arbeiten sollten Docker-Container bereitstellen, die alle Abhängigkeiten in definierten Versionen festlegen, sodass die Verifikationsergebnisse in Jahren noch reproduzierbar sind.

11.2.9 Industrielle Adoption und Standardisierung

Integration in ASIC-Workflows Während diese Arbeit auf FPGA-basierte Designs fokussiert, ist die Methodik prinzipiell auf ASIC-Designs übertragbar. Yosys unterstützt über kommerzielle Plugins (Yosys-SMTBMC, Yosys-EQY) auch den ASIC-Kontext. Amaranth-Designs könnten, nach Verifikation der Äquivalenz, in Standard-Cell-Syntheseflows (z. B. mit OpenROAD) überführt werden. Dies wäre insbesondere für sicherheitskritische Anwendungen (Automotive, Aerospace) relevant, in denen formale Verifikation gesetzlich vorgeschrieben ist (DO-254, ISO 26262).

Beitrag zu offenen HDL-Standards Amaranth ist noch kein IEEE-standardisiertes Format. Eine Standardisierungsinitiative ähnlich SystemVerilog (IEEE 1800) oder VHDL (IEEE 1076) würde die Toolchain-Interoperabilität erhöhen und die industrielle Akzeptanz fördern. Die Forschungsgemeinschaft könnte durch Publikationen vergleichender Studien und durch aktive Mitarbeit an der Amaranth-Spezifikation diesen Prozess unterstützen.

Benchmarking gegen kommerzielle Äquivalenzprüfungs-Tools Synopsys Formality und Cadence Conformal sind Industrie-Standards mit hoher Beweisleistung. Eine systematische Vergleichsstudie, die Yosys gegen diese kommerziellen Tools auf standardisierten Benchmark-Suites (z. B. ITC-99, ISCAS-89) antritt, würde die Stärken und Schwächen des quelloffenen Ansatzes quantifizieren und eine fundierte Empfehlung für verschiedene Design-Szenarien ermöglichen.

11.2.10 Subgraph Algorithmus-basierten Äquivalenzprüfung

Das in Kapitel 8 entwickelte Verfahren eröffnet eine Vielzahl weiterführender Forschungsrichtungen. Der folgende Abschnitt skizziert diese systematisch und im Detail.

Erweiterung auf sequentielle Hardware-Designs Die in Kapitel 8 bewiesene Implikation (Satz 8.5.2) gilt explizit für *kombinatorische* Hardware-Designs, da nur bei diesen die Datenabhängigkeit vollständig durch einen azyklischen Graphen beschrieben werden kann. Sequentielle Designs — also solche mit Flip-Flops, Registern und Rückkopplungen — erzeugen Zyklen im Abhängigkeitsgraphen, die eine direkte Anwendung des Subgraph Algorithmus erschweren.

Ein zentraler Ansatz zur Erweiterung ist die *Zeitentfaltung* (Time Unrolling): Das sequentielle Design wird über k Taktzyklen in ein äquivalentes kombinatorisches Design transformiert, indem der Zustandsraum explizit in die Knotenstruktur eingebettet wird. Der resultierende Entfaltungsgraph $\hat{H}(D, k)$ besitzt $k \cdot n$ Knoten (mit n Signalen im Original) und ist azyklisch, sofern keine kombinatorischen Schleifen existieren. Der Subgraph Algorithmus kann dann auf $\hat{H}(D_1, k)$ und $\hat{H}(D_2, k)$ angewendet werden.

Die Laufzeit des erweiterten Verfahrens beträgt dann $O((kn)^3) = O(k^3n^3)$, da die Knotenanzahl sich k -fach erhöht. Für kleine Entfaltungstiefe k — typisch $k \leq 10$ bei bekannter Pipeline-Tiefe — bleibt das Verfahren praktikabel. Zukünftige Arbeiten sollten systematisch untersuchen, ab welcher Entfaltungstiefe k die Äquivalenz *vollständig* erfasst wird, und diesen Wert für relevante Klassen sequentieller Designs (Zähler, Zustandsautomaten, Pipeline-Register) empirisch bestimmen.

Darüber hinaus bietet die Erweiterung des Subgraph Algorithmus auf gewichtete Graphen eine alternative Modellierungsmöglichkeit: Kanten erhalten Gewichte, die die zeitliche Verzögerungs- oder Taktbindung repräsentieren. Eine solche Gewichtserweiterung würde es erlauben, sowohl kombinatorische als auch getaktete Pfade im selben Graphmodell darzustellen, ohne explizite Entfaltung durchführen zu müssen.

Hybride Verifikation: Kombination von Subgraph Algorithmus und SAT-Solver

Das in Kapitel 8.7 gezogene Fazit, dass SAT-basierte und Subgraph Algorithmus-basierte Methoden komplementär sind, legt eine hybride Verifikationspipeline nahe, die beide Stärken vereint. Das konkrete Vorgehen könnte wie folgt ausstuiert werden:

1. **Vorfilterung mit dem Subgraph Algorithmus (in $O(n^3)$):** Der Subgraph Algorithmus prüft zunächst, ob die Hardware-Abhängigkeitsgraphen strukturell äquivalent sind. Falls ja, ist die Äquivalenz nachgewiesen und der SAT-Solver muss nicht bemüht werden. Dies spart bei strukturell gleichen Designs den gesamten SAT-Aufruf ein.
2. **SAT-basierte Tiefenprüfung bei Strukturdivergenz:** Falls der Subgraph Algorithmus Strukturunterschiede feststellt (d. h. die Designs sind nicht strukturell äquivalent), bedeutet das noch keine funktionale Nicht-Äquivalenz: Logikoptimierungen im Syntheseprozess können dieselbe Funktion mit anderem Graphen realisieren. In diesem Fall wird der SAT-Solver aktiviert, um die funktionale Äquivalenz zu prüfen. Der Subgraph Algorithmus hat dann wenigstens eine Schnellaussage über die Strukturdivergenz geliefert und möglicherweise den Suchraum für den SAT-Solver durch Identifikation identisch-strukturierter Teilgraphen (als fixe Punkte) eingeschränkt.
3. **Rückkopplung: Gegenbeispiele des SAT-Solvers als Graphannotation:** Wenn der SAT-Solver ein Gegenbeispiel liefert (eine Eingabekombination, auf der beide Designs divergieren), kann diese Information genutzt werden, um den Abhängigkeitsgraphen zu annotieren: der divergente Pfad wird markiert, und der Subgraph Algorithmus kann in zukünftigen Versionen bevorzugt auf annotierten Teilgraphen angewendet werden.

Eine solche hybride Pipeline würde insbesondere bei CI/CD-Anwendungen einen erheblichen Laufzeitvorteil bieten: Die überwältigende Mehrheit der Check-ins ändert nichts an der Abhängigkeitsstruktur (nur Kommentare, Parameternamen, Formatierung), sodass der Subgraph Algorithmus allein suffizienten Nachweis der Äquivalenz in polynomieller Zeit liefert, ohne den SAT-Solver zu starten.

Automatische Extraktion von Hardware-Abhängigkeitsgraphen aus Netzlisten

Der in Kapitel 8.2 beschriebene Hardware-Abhängigkeitsgraph wurde konzeptionell eingeführt. Die automatische Extraktion dieses Graphen aus bestehenden Werkzeugketten ist ein eigenständiges Forschungsproblem. Konkret bieten sich zwei Ansätze an:

Extraktion aus Amaranth-Zwischenrepräsentation: Das Amaranth-Framework erzeugt intern eine RTLIL-Repräsentation (RTL Intermediate Language), die von Yosys verarbeitet wird. Aus dieser Repräsentation lässt sich der Abhängigkeitsgraph direkt ableiten: Jede RTLIL-Zelle (Gatter, Multiplexer, arithmetische Einheit) wird zum Knoten, jede Verbindung zwischen Zellen zur Kante. Ein Python-Plugin für Yosys, das diesen Graphen als Adjazenzliste oder -matrix exportiert, würde die nahtlose Integration in die bestehende Verifikationspipeline erlauben.

Extraktion aus Verilog-AST mit Pyverilog: Für den VHDL-Zweig (über GHDL oder Claude-konvertiertes Verilog) kann das Python-Framework `Pyverilog` den abstrakten Syntaxbaum (AST) des Verilog-Codes analysieren und direkt in einen Abhängigkeitsgraphen überführen. Dies hätte den Vorteil, unabhängig von der RTLIL-Ebene zu arbeiten und damit auch höherstufige Abstraktionsphänomene (Schleifenstrukturen, parameterisierte Blöcke) zu erfassen.

In beiden Fällen ist ein *Graph-Normalisierungsschritt* notwendig: Knoten werden in topologischer Reihenfolge nummeriert (Schritt 1 der Normalisierung), und redundante Geistknoten (buffers, wires ohne Logik) werden kollabiert (Schritt 2). Erst nach dieser Normalisierung sind die Graphen aus Amaranth und VHDL vergleichbar und können in den Subgraph Algorithmus eingespeist werden. Die Entwicklung eines robusten Normalisierungsalgorithmus, der mit den Quirks beider Werkzeugketten umgeht, ist ein abgeschlossenes, praxisrelevantes Forschungsthema.

Theoretische Erweiterungen: Bedingte und unbedingte untere Schranken

Satz 8.5.11 (Kapitel 8) beweist die Optimalität des Verfahrens *unter SETH*. SETH ist eine weithin akzeptierte, aber unbedingt unbewiesene Komplexitätshypothese. Zukünftige theoretische Arbeiten könnten in zwei Richtungen voranschreiten:

Richtung 1 — Stärkere bedingte Schranke unter schwächerer Hypothese: SETH impliziert die *Orthogonal Vectors Conjecture* (OVC), die wiederum untere Schranken für LCS und Edit-Distanz impliziert. Möglicherweise lässt sich die $\Omega(n^{3-\epsilon})$ -Schranke für das Subgraph-Äquivalenzprüfungsproblem bereits aus der schwächeren NSETH (Non-uniform SETH) oder aus der 3SUM-Conjecture ableiten, was die Schranke auf eine breitere, robustere Basis stellen würde.

Richtung 2 — Unbedingte $\Omega(n^{2+\delta})$ -Schranke: Ein unbedingter Beweis einer superkubischen oder auch nur superquadratischen unteren Schranke für das zyklische Subgraph-Problem würde — ohne jede Hypothese — die praktische Relevanz des Algorithmus beziffern. Ein aussichtsreicher Ansatz wäre, die Kommunikationskomplexität des Problems zu analysieren: In einem Zwei-Parteien-Protokoll, bei dem Alice Matrix A und Bob Matrix B hält und beide entscheiden müssen, ob $G \cong_s G'$, kann die Kommunikationskomplexität als untere Schranke für die sequentielle Zeitkomplexität herangezogen

werden.

Skalierung auf sehr große Hardware-Designs Der Subgraph Algorithmus hat eine Laufzeit von $\Theta(n^3)$, was für $n > 10\,000$ Knoten praktisch problematisch werden kann: Bei $n = 10\,000$ wären theoretisch 10^{12} Operationen nötig. Für industrielle Designs mit Millionen von Signalen sind approximative oder hierarchische Erweiterungen unumgänglich.

Hierarchische Dekomposition: Analog zur compositionellen Verifikation (siehe oben) könnte das Hardware-Design in hierarchische Teilgraphen zerlegt werden. Jede Hierarchieebene wird separat mit dem Subgraph Algorithmus verglichen; die Korrektheit der Komposition wird durch Assume-Guarantee-Kontrakte abgesichert. Die Gesamtlaufzeit reduziert sich von $O(n^3)$ auf $O(k \cdot (n/k)^3) = O(n^3/k^2)$, wenn das Design in k gleichgroße Teilkomponenten zerlegt werden kann — ein quadratischer Speedup durch Dekomposition.

Approximative Subgraph-Prüfung mit Fehlertoleranz: Für Designs, bei denen geringfügige Implementierungsunterschiede (z. B. verschiedene Synthese-Optimierungen) toleriert werden können, könnte ein approximativer Modus des Subgraph Algorithmus eingeführt werden: Statt exakter Signaturübereinstimmung wird eine Hamming-Distanz $d(\hat{\sigma}^A, \hat{\sigma}^B) \leq \tau$ toleriert. Dies erlaubt den Nachweis von „annähernder Äquivalenz“ (Approximate Equivalence), die in der Hardware-Validierung für Optimierungspässe nützlich sein kann.

Parallelisierung der Rotationssuche: Satz 8.5.5 zeigt, dass die n Rotationsvergleiche unabhängig voneinander sind (Lemma 4 in [1]). Dies ist ideal für parallele Ausführung: Auf einem Mehrkern-Prozessor oder einer GPU können alle n LCS-Berechnungen gleichzeitig durchgeführt werden, was die Laufzeit von $\Theta(n^3)$ auf $\Theta(n^2 \cdot \lceil n/p \rceil)$ mit p Kernen reduziert — bei $p = n$ Kernen also auf $\Theta(n^2)$, was der theoretischen unteren Schranke der Eingabeleseschritte entspricht (Lemma 8.4.1).

Erweiterung der Graphrepräsentation für heterogene Designaspekte Die aktuelle Definition des Hardware-Abhängigkeitsgraphen (Definition 8.2.1) erfasst ausschließlich kombinatorische Datenabhängigkeiten. Für eine umfassendere Äquivalenzprüfung sind folgende Erweiterungen relevant:

Getypete Kanten (Typed Edges): Verschiedene Arten von Abhängigkeiten — Datenpfad, Kontrollpfad, Taktdomäne — werden durch verschiedene Kantentypen unterschieden. Der Subgraph Algorithmus müsste entsprechend auf Multigraphen erweitert werden, wobei die Signatur-Funktion für jeden Kantentyp eine separate Bit-Ebene verwendet:

$$\sigma_j^{(\text{typ})}(M) = \sum_{i=0}^{n-1} M_{ij}^{(\text{typ})} \cdot 2^{i+n \cdot \text{typ}} + j \cdot 2^{n \cdot T},$$

mit T verschiedenen Kantentypen. Die Injektivität bleibt durch die disjunkte Bit-Ebenen-Codierung erhalten.

Knotengewichte (Node Weights): Knoten erhalten Gewichte, die den Operationstyp kodieren (Addition, Vergleich, Multiplexer usw.). Der Subgraph Algorithmus prüft dann nicht nur die Verbindungsstruktur, sondern auch, ob entsprechend markierte Operationen übereinstimmen. Dies erlaubt die Unterscheidung zwischen zwei Designs, die strukturell äquivalent sind, aber verschiedene Operationstypen auf korrespondierenden Knoten ausführen.

Gerichtete vs. ungerichtete Subgraph-Prüfung: Abhängigkeitsgraphen sind naturgemäß gerichtet (Daten fließen von Eingabe zu Ausgabe). Der vorliegende Subgraph

Algorithmus operiert auf der Spaltenstruktur der Adjazenzmatrix (eingehende Kanten je Knoten). Zukünftige Arbeiten könnten eine symmetrische Variante entwickeln, die sowohl eingehende als auch ausgehende Kanten berücksichtigt und damit bidirektionale Datenfluss-Abhängigkeiten (z. B. in Rückkopplungsschleifen) korrekt modelliert.

Visualisierung und interaktive Beweisexploration Die theoretischen Beweise in Kapitel 8 sind formal rigoros, aber für Ingenieure ohne Informatik-Theorieausbildung schwer zugänglich. Zukünftige Arbeiten könnten Werkzeuge entwickeln, die den Subgraph-Äquivalenzprüfungsprozess interaktiv visualisieren:

- **Graph-Diff-Visualisierung:** Darstellung der Hardware-Abhängigkeitsgraphen beider Designs nebeneinander, mit farblicher Hervorhebung übereinstimmender (grün), divergenter (rot) und redundanter (gelb) Knoten und Kanten. Rotationsverschiebungen werden als animierte Übergänge dargestellt.
- **LCS-Trace:** Anzeige der optimalen LCS-Lösung als hervorgehobene Teilsequenz innerhalb der Signatur-Arrays, sodass unmittelbar sichtbar wird, welche Signale in beiden Designs übereinstimmen.
- **Interaktiver Rotationsexplorer:** Schieberegler, mit dem der Ingenieur manuell die n Rotationen durchläuft und für jede Rotation den aktuellen LCS-Wert und die Graphübereinstimmung verfolgt. Dies fördert das intuitive Verständnis des Algorithmus und erleichtert die Fehlerdiagnose bei Nicht-Äquivalenz.

Ein solches Werkzeug könnte als Plugin für bestehende Hardware-Design-Umgebungen (z. B. als Yosys-Webinterface oder als Jupyter-Notebook-Widget) implementiert werden.

Anwendung des Subgraph Algorithmus auf höhere Abstraktionsebenen Bisher wurde der Subgraph Algorithmus auf der Ebene der RTL-Signals angewendet. Die Methode lässt sich jedoch auf deutlich höhere Abstraktionsebenen übertragen:

Funktionsblock-Ebene (IP-Core-Vergleich): In modernen SoC-Designs werden häufig vorgefertigte IP-Cores (z. B. AES-Beschleuniger, UART-Controller) aus verschiedenen Quellen integriert. Der Subgraph Algorithmus könnte auf Ebene der Blockdiagramme eingesetzt werden: Jeder IP-Core ist ein Knoten, jede Daten- oder Steuerverbindung zwischen Cores ist eine Kante. Die strukturelle Äquivalenz zweier SoC-Designs auf Block-Ebene lässt sich dann in $O(k^3)$ Zeit prüfen, mit k der (deutlich kleineren) Anzahl von IP-Cores. Dies ist besonders wertvoll für Sicherheitsaudits, bei denen nachgewiesen werden soll, dass ein modifiziertes SoC-Design keine unerwünschten neuen Verbindungen enthält.

Algorithm-Level-Synthese (HLS): High-Level-Synthesis-Werkzeuge (z. B. Vitis HLS, Bambu) übersetzen C/C++-Algorithmen in RTL-Designs. Zwei verschiedene HLS-Konfigurationen desselben Algorithmus (z. B. verschiedene Unrolling-Faktoren, verschiedene Ressourcenbeschränkungen) erzeugen strukturell unterschiedliche, aber funktional äquivalente RTL-Designs. Der Subgraph Algorithmus auf dem HLS-Datenflussgraphen (DFG) — der naturgemäß dieselbe Struktur wie der Hardware-Abhängigkeitsgraph hat — kann hier eingesetzt werden, um vor dem eigentlichen RTL-Vergleich bereits auf algorithmischer Ebene Äquivalenz oder Divergenz festzustellen.

Software-Äquivalenz via AST-Graphen: Die ursprüngliche Motivation des Subgraph Algorithmus [1] sind abstrakte Syntaxbäume (ASTs) von Programmen. Ein unmittelbarer Übertrag der Hardware-Äquivalenzprüfungs-Methodik auf Software-Äquivalenz

liegt daher nahe: Zwei Implementierungen desselben Algorithmus (z. B. in Python und in C) werden in ihre AST-Graphen überführt und mit dem Subgraph Algorithmus verglichen. Dies würde die bestehende Anwendungsdomäne erheblich erweitern und den Subgraph Algorithmus zu einem universellen, sprach- und plattformübergreifenden Äquivalenzprüfungswerkzeug machen.

Formale Verankerung im Curry-Howard-Isomorphismus Die in Kapitel 8 geführten Beweise sind klassische mathematische Beweise. Ein fortgeschrittenes, theoretisch bedeutungsvolles Ziel wäre die Formalisierung dieser Beweise in einem interaktiven Theorembeweiser wie Isabelle/HOL, Lean 4 oder Coq. Dies hätte mehrere Vorteile:

- **Maschinengeprüfte Korrektheit:** Alle acht Sätze aus Kapitel 8 — insbesondere Satz 8.5.2 (Implikation strukturell $\Rightarrow$ funktional) und Satz 8.5.11 (SETH-Optimalität) — würden durch den Theorembeweiser maschinell verifiziert, ohne die Möglichkeit menschlicher Lücken im Induktionsbeweis.
- **Extraktion von verifizierten Algorithmen:** Aus dem Lean-4-Beweis des Korrektheitssatzes könnte mittels Curry-Howard direkt verifizierbarer Haskell- oder OCaml-Code extrahiert werden, der den Subgraph Algorithmus korrekt implementiert — eine in der formalen Methodik als *verified extraction* bekannte Technik.
- **Wiederverwendbare Beweisbausteine:** Die Lemmas über zyklische Gruppen (Satz 8.5.8), Signatur-Injektivität (Lemma 8.4.1) und LCS-Komplexität unter SETH (Lemma aus [1]) können als wiederverwendbare Proof-Bausteine in eine gemeinsame Bibliothek einfließen, die auch für verwandte Algorithmen genutzt werden kann.

Integration in sicherheitskritische Zertifizierungsrahmen Die Beweise in Kapitel 8 legen eine Grundlage für den Einsatz des Subgraph Algorithmus in sicherheitskritischen Domänen, in denen formale Verifikation gesetzlich vorgeschrieben ist. Konkret bieten sich folgende Anwendungsfelder an:

DO-254 (Avionics Hardware): Der Standard DO-254 (Design Assurance Guidance for Airborne Electronic Hardware) schreibt für Design-Assurance-Level A (höchste Sicherheitsstufe) explizite Äquivalenzprüfungen zwischen verschiedenen Entwicklungsstufen vor. Der Subgraph Algorithmus könnte als zertifizierfähiges Werkzeug positioniert werden, sofern seine Korrektheit formal (maschinell geprüft) nachgewiesen ist und eine Tool-Qualifikation gemäß DO-330 durchgeführt wurde.

ISO 26262 (Automotive): Für ASIL-D-Sicherheitsfunktionen (höchste Automobil-Sicherheitsstufe, z. B. in Bremssystemen oder autonomen Fahrfunktionen) fordert ISO 26262 den Nachweis funktionaler Äquivalenz zwischen Spezifikation und Implementation. Die Subgraph Algorithmus-basierte Methode könnte hier als ergänzender, performanter Vorab-Check eingesetzt werden, der die Notwendigkeit vollständiger SAT-basierter Verifikation auf strukturell divergente Fälle beschränkt.

Common Criteria (IT-Sicherheit): Im Rahmen der Common Criteria Evaluation Methodology (CEM) wird für hochstufige Evaluierungen (EAL 6/7) ein formaler Implementierungsnachweis gefordert. Hardware-Security-Module (HSMs), die kryptographische Operationen in FPGA-Logik implementieren, könnten durch die Kombination aus Subgraph Algorithmus-basierter Strukturprüfung und SAT-basierter Funktionalitätsprüfung effizient zertifiziert werden.

Benchmarking des Subgraph Algorithmus auf Hardware-Äquivalenz-Benches

Für eine wissenschaftlich belastbare Einordnung des Verfahrens ist ein systematisches Benchmarking unabdingbar. Folgende Benchmark-Suites sind relevant:

- **ITC-99 und ISCAS-89:** Standardisierte kombinatorische (ISCAS-85) und sequentielle (ISCAS-89) Schaltkreis-Benchmarks aus der Testgenerierungsforschung. Für jedes Benchmark-Design existieren bekannte Varianten (mit und ohne Optimierung), die als Äquivalenzprüfungs-Instanzen genutzt werden können.
- **OpenCores Hardware Designs:** Die Open-Source-Community OpenCores stellt hunderte von RTL-Designs bereit (UART, I2C, SPI, DMA-Controller, CPU-Kerne). Viele dieser Designs existieren in mehreren Varianten (verschiedene Autoren, Syntheseziele). Sie bieten einen realistischen Benchmark für die Subgraph Algorithmus-basierte Äquivalenzprüfung.
- **Synthetischer Mutations-Benchmark:** Für eine kontrollierte Evaluation des Sensitivitätsverhaltens (Satz 8.5.6) können Designs durch gezielte Einzel-Kanten-Mutationen modifiziert werden. Der Subgraph Algorithmus sollte in 100% der Fälle die Mutation als Strukturdivergenz erkennen — eine Kennzahl, die direkt aus Satz 8.5.6 und Korollar 8.5.7 folgt.

Eine Benchmarkstudie würde darüber hinaus empirisch validieren, bei welcher Graphgröße n der theoretische $O(n^3)$ -Vorteil gegenüber SAT-basierten Methoden (mit exponentiellem Worst-Case) praktisch spürbar wird — ein Ergebnis mit unmittelbarer Relevanz für die Systemdimensionierung in der Praxis.

11.3 Schlusswort

Amaranth demonstriert das Potential von Python-basierten HDLs als produktive Alternative zu VHDL und Verilog. Die formale Verifizierbarkeit mit Open-Source-Tools macht Amaranth zu einer ernstzunehmenden Option für professionelle Hardware-Entwicklung.

Das in Kapitel 8 neu entwickelte Verfahren der Subgraph Algorithmus-basierten Äquivalenzprüfung ergänzt die etablierte SAT-basierte Methodik um eine strukturelle, polynomial-zeitliche Perspektive. Mit acht rigoros bewiesenen Sätzen — von der Korrektheit über die Laufzeitoptimalität bis hin zur Transitivität und Sensitivität — liefert dieses Kapitel einen eigenständigen theoretischen Beitrag, der über die konkrete ALU-Anwendung hinausgeht und einen allgemeinen Rahmen für die graphbasierte Hardware-Äquivalenzprüfung bietet.

Die in Kapitel 10 vorgestellte pyUVM-Testbench unterstreicht, dass die Python-Nativität von Amaranth eine natürliche Erweiterung in die Domäne der UVM-basierten Simulation erlaubt: Design und Verifikation teilen eine Sprache, eine Entwicklungsumgebung und ein Ökosystem. Formale Verifikation (Yosys), strukturelle Äquivalenzprüfung (Subgraph Algorithmus) und simulations-basierte Verifikation (pyUVM) ergänzen sich zur vollständigen, mehrstufigen Qualitätssicherungskette für Hardware-Designs in Verilog und VHDL.

Die im Ausblick skizzierten Erweiterungen — Sequentialität, hybride Pipelines, automatische Graphextraktion, Skalierung auf industrielle Designs, formale Verankerung in Theorembeweisern sowie Integration in Sicherheitszertifizierungsrahmen — zeigen, dass das vorgestellte Verfahren ein fruchtbares Fundament für eine neue Forschungsrichtung an der Schnittstelle von Algorithmentheorie und formaler Hardware-Verifikation bildet.

Literaturverzeichnis

- [1] Epp, S., *Subgraph Algorithmus*, 2026, Verfügbar bei <https://gitlab.com/epp-group/subgraph>.
- [2] Amaranth HDL Project, *Amaranth HDL Documentation*, <https://amaranth-lang.org/>, 2024.
- [3] Claire Xenia Wolf, *Yosys Open SYnthesis Suite*, <http://www.clifford.at/yosys/>, 2024.
- [4] MyHDL Project, *MyHDL Documentation*, <http://www.myhdl.org/>, 2024.
- [5] IEEE Standard, *IEEE 1076-2019 - IEEE Standard VHDL Language Reference Manual*, 2019.
- [6] OpenFPGALoader Project, *Universal utility for programming FPGA*, <https://github.com/trabucayre/openFPGALoader>, 2024.
- [7] Digilent Inc., *Basys 3 FPGA Board Reference Manual*, 2024.
- [8] Biere, A., Heule, M., van Maaren, H., Walsh, T., *Handbook of Satisfiability*, IOS Press, 2009.
- [9] Clarke, E., Grumberg, O., Peled, D., *Model Checking*, MIT Press, 1999.
- [10] F4PGA Project, *F4PGA – Free and Open-Source FPGA Toolchain*, <https://f4pga.org/>, 2024.
- [11] Gingold, T., *GHDL – Open Source VHDL 87/93/00/02/08 Simulator*, <https://ghdl.github.io/ghdl/>, 2024.
- [12] MacIver, D. R. et al., *Hypothesis: Property-Based Testing for Python*, <https://hypothesis.works/>, 2024.
- [13] Cocotb Contributors, *cocotb – Coroutine-based Co-simulation Test Bench*, <https://www.cocotb.org/>, 2024.
- [14] Enjoy-Digital, *LiteX – A Python-Based SoC Builder*, <https://github.com/enjoy-digital/litex>, 2024.
- [15] SymbiYosys Contributors, *SymbiYosys – Formal Hardware Verification Front-End*, <https://symbiyosys.readthedocs.io/>, 2024.
- [16] Ray Salemi, *pyUVM – Python Universal Verification Methodology*, <https://github.com/pyuvm/pyuvm>, 2024.

- [17] Accellera Systems Initiative, *IEEE 1800.2-2020 – IEEE Standard for Universal Verification Methodology Language Reference Manual*, IEEE, 2020.
- [18] IEEE Standard, *IEEE 1364-2005 – IEEE Standard for Verilog Hardware Description Language*, IEEE, 2006.
- [19] IEEE Standard, *IEEE 1800-2017 – IEEE Standard for SystemVerilog – Unified Hardware Design, Specification, and Verification Language*, IEEE, 2017.
- [20] Cadence Design Systems; Mentor Graphics, *Open Verification Methodology (OVM) Reference Manual, Version 2.0*, 2008.
- [21] Bergeron, J., Cerny, E., Hunter, A., Nightingale, A., *Verification Methodology Manual for SystemVerilog*, Springer, 2006.
- [22] Vip, M., *SystemVerilog and UVM Verification: A Practical Guide*, Independently published, 2021.
- [23] Snyder, W., Verilator Contributors, *Verilator – Fast Open-Source Verilog/SystemVerilog Simulator*, <https://www.veripool.org/verilator/>, 2023.
- [24] Williams, S., *Icarus Verilog – A Free Verilog Simulator*, <http://iverilog.icarus.com/>, 2023.
- [25] Newsome, N., *NVC – VHDL Compiler and Simulator*, <https://github.com/nickg/nvc>, 2024.
- [26] Siemens EDA, *ModelSim SE User’s Manual*, Siemens EDA, 2022.
- [27] Siemens EDA, *Questa Sim User’s Manual*, Siemens EDA, 2023.
- [28] Wolf, C. X., SymbiYosys Contributors, *SymbiYosys Formal Verification Flow*, <https://symbiyosys.readthedocs.io/>, 2024.
- [29] Niemetz, A., Preiner, M., Wolf, C. X., Biere, A., *Boolector 3.0*, Journal on Satisfiability, Boolean Modeling and Computation, Vol. 11, pp. 99–102, 2019.
- [30] Bjørner, N., de Moura, L., Lal, A., *Z3: A Production Theorem Prover*, Microsoft Research, <https://github.com/Z3Prover/z3>, 2024.
- [31] Niemetz, A., Preiner, M., *Bitwuzla: A Fast SMT Solver for Fixed-Size Bit-Vectors*, <https://bitwuzla.github.io/>, 2024.
- [32] Brayton, R., Mishchenko, A., *ABC: A System for Sequential Synthesis and Verification*, UC Berkeley, <https://github.com/berkeley-abc/abc>, 2023.
- [33] Kroening, D., Tautschnig, M., *CBMC: Bounded Model Checking for ANSI-C and C++ Programs*, LNCS Vol. 8413, Springer, 2023.
- [34] Holzmann, G. J., *The SPIN Model Checker: Primer and Reference Manual*, Addison-Wesley, 2004.
- [35] Cimatti, A., Clarke, E. M., Giunchiglia, E., *NuSMV 2: An OpenSource Tool for Symbolic Model Checking*, Proc. CAV 2002, LNCS 2404, pp. 359–364, Springer, 2002.

- [36] Cadence Design Systems, *JasperGold Formal Verification Platform*, Cadence, 2023.
- [37] Halbwachs, N., *Synchronous Programming of Reactive Systems*, Kluwer Academic Publishers, 1993.
- [38] Biere, A., Clarke, E., Raimi, R., Zhu, Y., *Verifying Safety Properties of a PowerPC Microprocessor Using Symbolic Model Checking Without BDDs*, Proc. CAV 1999, LNCS 1633, Springer, 1999.
- [39] Bryant, R. E., *Graph-Based Algorithms for Boolean Function Manipulation*, IEEE Transactions on Computers, Vol. C-35, No. 8, pp. 677–691, 1986.
- [40] Davis, M., Logemann, G., Loveland, D., *A Machine Program for Theorem Proving*, Communications of the ACM, Vol. 5, No. 7, pp. 394–397, 1962.
- [41] Marques-Silva, J. P., Sakallah, K. A., *GRASP: A Search Algorithm for Propositional Satisfiability*, IEEE Transactions on Computers, Vol. 48, No. 5, pp. 506–521, 1999.
- [42] Eén, N., Sörensson, N., *An Extensible SAT-Solver*, Proc. SAT 2003, LNCS 2919, pp. 502–518, Springer, 2004.
- [43] Biere, A., Fleury, M., Heisinger, M., *CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling Entering the SAT Competition 2020*, Proc. SAT Competition 2020, 2020.
- [44] Janota, M., Klieber, W., Marques-Silva, J., Clarke, E., *Solving QBF with Counterexample Guided Refinement*, Artificial Intelligence, Vol. 234, pp. 1–25, Elsevier, 2016.
- [45] Wolf, C. X., *Verifying the RISC-V Compliance Suite with SymbiYosys*, Acks Formal Verification Workshop, 2019.
- [46] Waterman, A., Asanovic, K. (eds.), *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, RISC-V International, 2019.
- [47] AMD Xilinx, *7 Series FPGAs Data Sheet: Overview (DS180)*, AMD, 2023.
- [48] AMD Xilinx, *Vivado Design Suite User Guide: Synthesis (UG901)*, AMD, 2023.
- [49] AMD Xilinx, *Vivado Design Suite User Guide: Implementation (UG904)*, AMD, 2023.
- [50] Intel Corporation, *Quartus Prime Pro Edition User Guide: Getting Started*, Intel, 2023.
- [51] Lattice Semiconductor, *ECP5 and ECP5-5G Family Data Sheet*, Lattice Semiconductor, 2023.
- [52] nextpnr Contributors, *nextpnr – A Portable FPGA Place and Route Tool*, <https://github.com/YosysHQ/nextpnr>, 2023.
- [53] SymbiFlow Contributors, *Project X-Ray: Documenting the Xilinx 7-Series Bitstream Format*, <https://github.com/SymbiFlow/prjxray>, 2023.
- [54] VUnit Contributors, *VUnit – A Unit Testing Framework for VHDL/SystemVerilog*, <https://vunit.github.io/>, 2024.

- [55] Bitvis AS, *UVVM – Universal VHDL Verification Methodology*, <https://www.uvvm.org/>, 2024.
- [56] SynthWorks Design, *OSVVM – Open Source VHDL Verification Methodology*, <https://osvvm.org/>, 2024.
- [57] Accellera, *SystemC Verification Standard (SCV) 1.0*, Accellera Systems Initiative, 2005.
- [58] IEEE Standard, *IEEE 1850-2010: Standard for Property Specification Language (PSL)*, IEEE, 2010.
- [59] Hussain, B., Khan, F. A., *Python-Based Hardware Description Languages: A Comparative Study*, ACM Computing Surveys, Vol. 55, No. 8, 2023.
- [60] Bachrach, J., Vo, H., Richards, B., Lee, Y., Waterman, A., *Chisel: Constructing Hardware in a Scala Embedded Language*, Proc. DAC 2012, pp. 1215–1224, ACM, 2012.
- [61] Baaij, C., Kooijman, M., Kuper, J., Boeijink, W., Gerards, M., *CLaSH: Structural Descriptions of Synchronous Hardware using Haskell*, Proc. DSD 2010, pp. 714–721, IEEE, 2010.
- [62] m-labs, *nMigen: A Python-Based Hardware Description Language (predecessor to Amaranth)*, <https://github.com/m-labs/nmigen>, 2019.
- [63] Bourdeauducq, S., *Migen: A Python Toolbox for Building Complex Digital Hardware*, <https://github.com/m-labs/migen>, 2013.
- [64] Papon, C., *SpinalHDL – An HDL written in Scala*, <https://github.com/SpinalHDL/SpinalHDL>, 2015.
- [65] Schuiki, F., Kurth, A., Grosser, T., Benini, L., *LLHD: A Multi-Level Intermediate Representation for Hardware Description Languages*, Proc. PLDI 2020, pp. 258–271, ACM, 2020.
- [66] Nigam, R., Thomas, S., Li, Z., Sampson, A., *A Compiler Infrastructure for Accelerator Generators*, Proc. ASPLOS 2021, ACM, 2021.
- [67] CIRCT Contributors (LLVM Project), *CIRCT: Circuit IR Compilers and Tools*, <https://circt.llvm.org/>, 2021.
- [68] Mukherjee, R., Kroening, D., Melham, T., *Hardware Verification Using Software Analyzers*, Proc. ISVLSI 2015, pp. 7–12, IEEE, 2015.
- [69] Cheng, K.-T., Krishnakumar, A., *Automatic Functional Test Generation Using the Extended Finite State Machine Model*, Proc. DAC 1993, ACM, 1993.
- [70] Chowdhury, A., Wait, M., Patnaik, P., *Mutation Testing for Hardware Description Languages*, IEEE Transactions on VLSI Systems, Vol. 25, No. 3, pp. 987–997, 2017.
- [71] Bergeron, J., *Writing Testbenches Using SystemVerilog*, Springer, 2011.
- [72] Piziali, A., *Functional Coverage for Design and Verification*, Springer, 2010.

- [73] IEEE Standard, *IEEE 2148-2020: Standard for Measuring Coverage of Functional Testing*, IEEE, 2020.
- [74] IEEE Standard, *IEEE 1364-2005 Annex: Value Change Dump Format specification*, IEEE, 2005.
- [75] Bertot, Y., Castéran, P., *Interactive Theorem Proving and Program Development – Coq’Art*, Springer, 2004.
- [76] Moura, L. de, Ullrich, S., *The Lean 4 Theorem Prover and Programming Language*, Proc. CADE 2021, LNCS 12699, Springer, 2021.
- [77] Jackson, D., *Software Abstractions: Logic, Language, and Analysis*, MIT Press, 2006.
- [78] Lamport, L., *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*, Addison-Wesley, 2002.
- [79] Pnueli, A., *The Temporal Logic of Programs*, Proc. 18th Annual Symposium on Foundations of Computer Science, pp. 46–57, IEEE, 1977.
- [80] Clarke, E. M., Emerson, E. A., *Design and Synthesis of Synchronization Skeletons Using Branching Time Temporal Logic*, Logics of Programs Workshop, LNCS 131, pp. 52–71, Springer, 1981.
- [81] Büchi, J. R., *On a Decision Method in Restricted Second Order Arithmetic*, Proc. International Congress on Logic, Method, and Philosophy of Science, Stanford University Press, pp. 1–11, 1962.
- [82] Cousot, P., Cousot, R., *Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints*, Proc. POPL 1977, pp. 238–252, ACM, 1977.
- [83] Henzinger, T. A., Jhala, R., Majumdar, R., Sutre, G., *Software Verification with BLAST*, Proc. SPIN 2003, LNCS 2648, pp. 235–239, Springer, 2003.
- [84] Clarke, E. M., Grumberg, O., Jha, S., Lu, Y., Veith, H., *Counterexample-Guided Abstraction Refinement*, Proc. CAV 2000, LNCS 1855, pp. 154–169, Springer, 2000.
- [85] Bradley, A. R., *SAT-Based Model Checking Without Unrolling*, Proc. VMCAI 2011, LNCS 6538, pp. 70–87, Springer, 2011.
- [86] McMillan, K. L., *Interpolation and SAT-Based Model Checking*, Proc. CAV 2003, LNCS 2725, pp. 1–13, Springer, 2003.
- [87] Nane, R. et al., *A Survey and Evaluation of FPGA High-Level Synthesis Tools*, IEEE Transactions on CAD of Integrated Circuits and Systems, Vol. 35, No. 10, pp. 1591–1604, 2016.
- [88] Canis, A., Choi, J., Aldham, M., Zhang, V., *LegUp: An Open-Source High-Level Synthesis Tool for FPGA-Based Processor/ Accelerator Systems*, Proc. FPGA 2011, pp. 33–36, ACM, 2011.
- [89] AMD Xilinx, *Vitis High-Level Synthesis User Guide (UG1399)*, AMD, 2023.